

VECTOR

144 Pages of High-Value APL

- Clark on DNA Fingerprinting 38
- Wilson on the UK National Lottery 51
- AP207 and OpenGL Graphics 58,64
- HTML Basics for APLers — Tables 83
- Pearson on Namespaces 94
- Lescasse on Object Oriented +Win 106
- Burke on Operators in APL and J 129



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433

www.vector.org.uk

Vol.14 No.1 July 1997

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette (any format). APL code can be accepted as camera-ready copy, in workspaces from I-APL, APL*PLUS, IBM APL2/PC or Dyalog APL/W, or in documents from Windows Write (use the Vector TrueType font, available free from Vector Production), and Winword-2.

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 1997-98

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£14	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	10	5
Overseas Corporate	£135	10	
Sustaining	£430	10	5
Non-voting Member (Student, OAP, unemployed)	£6	1	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or JCB, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates (excl VAT) are £250 per full page, £125 for half-page or less (there is a £75 surcharge per page if spot colour is required).

Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to: Gill Smith, Brook House, Gilling East, YORK YO6 4JJ. Tel: 01439-788385 CompuServe: 100331,644

Contents

		Page
Editorial	Duncan Pearson	3
APL NEWS		
Quick Reference Diary		5
Correspondence		
Norman Thomson <i>et al</i>		7
British APL Association News		11
AGM and Annual Accounts		
News from Sustaining Members	Gill Smith	15
The Education Vector		33
Jottings 13	Norman Thomson	33
DNA Fingerprinting in APL	Ian Clark	38
Do APL2 and J Come up to Scratch?	Norman Thomson	46
The UK National Lottery	Alan Wilson	51
APL Product Guide	Gill Smith	19
GENERAL ARTICLES		
OpenGL Graphics and J	Chris Burke	58
Emulation of AP207 in Dyalog APL	Skomorokhov & Kornilovsky	64
The IML Matrix Language:		
A Belated Introduction	Donald B. Pittenger	78
HTML Basics for APLers — Tables	Adrian Smith	83
TECHNICAL SECTION		
Namespaces in Causeway	Duncan Pearson	94
Object Oriented Programming with APL+Win	Eric Lescasse	106
APL and J (5): Operators — Inner Product,		
Each, Commute, Compose	Chris Burke	129
At Play with J: Oh, No, Not Eigenvalues Again!	Gene McDonnell	135
The API Problem as a		
Particular Recurrence Problem	Andrei Buzin	140
Index to Advertisers		143

Enterprise software for today's business needs.

Providing high performance systems
and applications software designed to
meet the changing needs of
customers and the changing
face of Information Technology.

SHARP APL

a highly productive application development
environment for MVS mainframes and
UNIX platforms

Historically based in main-
frame technologies, today
Soliton's product line is integrated with
industry standard operating
environments using both
standard and proprietary network solutions.

ViewPoint

a powerful, flexible, yet easy-to-use data
manager and report generator

TimeSquare

a UNIX-based database management
system, optimized for financial timeseries
data, integrated with standard UNIX and
PC desktops and with popular relational
database managers

For more information,

Soliton Associates Limited

44 Victoria Street, Suite 2100

Toronto, Ontario, Canada M5C 1Y2

Tel: (416) 364-9355 Fax: (416) 364-6159

In Europe

Groot Blankenberg 53
1082 AC Amsterdam
The Netherlands
Tel: + 31-20-646-4475

In the U.S.

951 President Street, #3F
Brooklyn
New York, USA 11215
Tel: (718) 783-9330

1100 University Avenue, Suite 111
Rochester
New York, USA 14607
Tel: (716) 256-6466

www.soliton.com



**SOLITON
ASSOCIATES**

Enterprise Software Solutions

Editorial

by Duncan Pearson

This issue of Vector will be distributed to the delegates to the first annual world APL conference that is not organised under the auspices of the ACM SIGAPL.

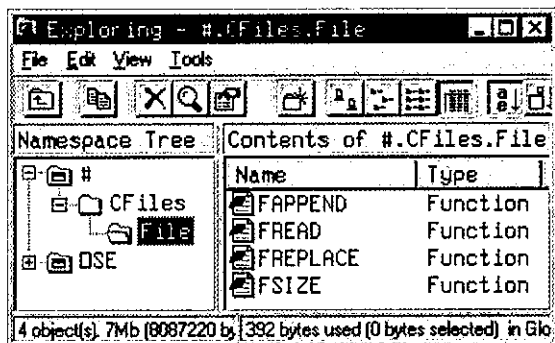
Due mainly to the lack of volunteer support SIGAPL has converted to a publications-only SIG. The lack of support has been in evidence for some time with Mike Kent having taken an increasing share of the organisational burden on his broad shoulders over the last two years. He deserves high praise and thanks for having done so. Anyway the first visible result of this change has been the appearance simultaneously of the conference proceedings for APL96 and the next issue of Quote-Quad. It is good to see Quote-Quad back on the stands and the latest issue is excellent. I am looking forward to more of the same.

SIGAPL is not the only volunteer organisation to suffer from the lack of support. The annual election to the committee of the British APL Association is rarely much more than a reshuffling of posts among an increasingly disillusioned band of stalwarts. The rôle of the Association is unclear with little support from among the membership for the few meetings that are organised and an increasing proportion of the membership from overseas. The opening of the Vector web pages has been a clear success and the work on finding practical ways to publish APL on the web is clearly of benefit to the entire community and to APL itself. Also the printed magazine continues to be read, judging from the feedback, not all positive, that we receive.

It is in the area of meetings however that the real challenge lies. The Toronto SIG are taking a fair risk in the organisation of APL97. They will be free of many of the restrictions that ACM imposed on previous conferences and the content and structure seems as a result to be more market led than before. There are many who see the annual conference as a means of gaining valuable experience of other fields of endeavour within the APL world and increasingly to do this one must have interactive sessions with participation enhancing the learning process. There are many APL organisations in the world with sufficient funds, if pooled, that we could ensure that this kind of event continued. Is this a better use of our time and resources than national meetings? Is it now time for greater international co-operation or should we contract and concentrate on the publications just as SIGAPL have done?

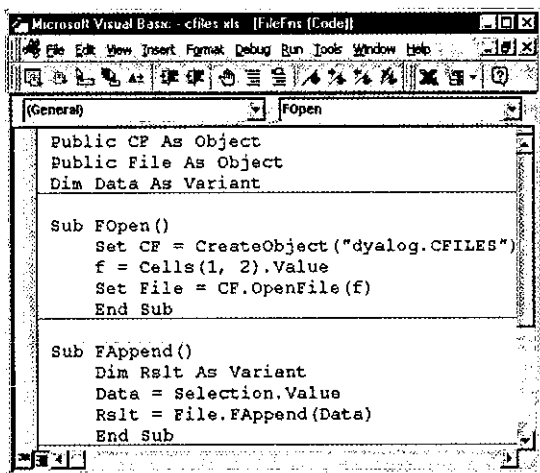
dyalog APL

The Definitive APL for Windows™



Have you ever
wanted to call your
APL functions from
Excel, Visual Basic,
or C++ ?

Well now you can!



That's why Dyalog APL/W remains the professional choice. For further information, contact Dyadic or your local distributor today.

Dyadic Systems Limited, Riverside View, Basing Road, Old Basing,
Basingstoke, Hants. RG24 7AL, United Kingdom.
Tel: +44 1256 811125 Fax: +44 1256 811130 Email: sales@dyadic.com

Microsoft is a registered trademark and Windows and the Windows Logo are trademarks of Microsoft Corporation



Quick Reference Diary 1997

Date	Venue	Event
August 17-20	University of Toronto	1997 International Conference on APL "Share Knowledge, Share Success"
November 3-5	Orlando, Florida	APL2000 User Conference

Conference News

The 1997 APL2000 Conference will be held Monday November 3, 1997 through Wednesday November 5, 1997 at the Sheraton World Resort in Orlando Florida. Presentation and Training schedules will be posted as soon as they are available.

Dates for Future Issues of VECTOR

	Vol.14 No.2	Vol.14 No.3	Vol.14 No.4
Copy date	5th September	5th December	6th March
Ad booking	12th September	12th December	13th March
Ad Copy	19th September	19th December	20th March
Distribution	October 97	January 98	April 98

Internet Information Server Auxiliary Processor for Dyalog APL for Windows

Microsoft.

**Windows NT.
Server 4.0**

with

Microsoft

**Internet
Information Server**



The ultimate tool for
developing
server-side Internet
applications using
these great products

dyalog APL

ISAP:

- Is a multithreaded DLL that provides full integration of Dyalog APL by Dyadic Systems Ltd. with the Microsoft Internet Information Server. APL applications run as IIS extensions, not as CGI programs. Zero interpreter loading time. Multiple applications can be run in parallel with only one copy of Dyalog APL loaded.
- Uses the Microsoft Internet Information Server just like other ActiveX modules do. The standard server's functionality remains available.
- Allows the use of all features of the Internet server--administration, maintenance, security, and secure connections--with no programming. (It's like having Bill Gates working for you for free.)
- Supports on-the-fly image creation using the standard graphical functions of Dyalog APL (GIF and x-bitmap formats).
- Offers true asynchronous event-driven programming.
- Easy to install. Simple to program. No sockets programming at all.



Lingo Allegro USA, Inc.

1105 Chicago Avenue, Suite 155
Oak Park, IL 60302, USA

1-(800)-LINGO-A1

71303.3224@CompuServe.com

all trademarks acknowledged

visit our server at www.lingo.com for live demo

CORRESPONDENCE

Optimizing the Antecedent Precipitation Index

From: Norman Thomson

12 May 1997

Am I missing something, or is Bob Bernecky unaware of `decode` (see Vector Vol.13 No.4 pp.7-11)? This solves the above problem in six characters, viz: `x1"`, `\y` and incidentally runs about four times as fast on APL2/PC as either of Bob's `api1` or `api3`. I described the analogous J version, namely `>@(#. &. ><\)`, in Vector Vol.13 No.3 p. 12.

[Ed - see also Andrei Buzin's note on page 140]

APL and the rank operator

From: Norman Thomson

12 May 1997

Chris Burke in his article *APL and J(4) : Function Application and the Axis Operator* does APL an injustice when he dismisses the axis operator as a "patchwork of special cases". He says that the APL interpreter gets the error messages wrong — he does not say *which* interpreter he is talking about — IBM interpreters correctly give *AXIS ERROR* for the two "wrong" messages he cites for transpose and rotate. He also says that "we would expect that $(\rho, [\theta]A) \leftrightarrow (\rho A), 1$ ". This is precisely what happens in APL2, that is $(\rho, [10]A) \leftrightarrow (\rho A), 1$.

While it is true that the rank operator is a unifying concept, and that the APL axis notation has always been something of an inconsistency, it is unfair to condemn the APL syntax for transpose and rotate which were standardised long before the rank operator was conceived. In both cases, the values necessary to differentiate the various possible cases can be provided with total adequacy as function parameters, and there is logically no reason why such definitions should be altered to an operator form just because this route has been chosen in J.

As a historical footnote, Chris says of dyadic APL functions that "In theory... a semi-colon could be used to delimit the ranks of left and right axes". The first experimental version of the IBM APL2 interpreter did just that, but the idea was abandoned.

The Enclose of an Empty Array

From: Joseph De Kerf

1 May 1997

ISO APL and the proposed standard for Extended APL define an empty array A as an array, the count of which is 0, the count being the product of the shape list of the array ($\times/\rho A$). APL books and manuals translate this by defining an empty array as an array of rank 1 or more with at least one length of its axes being 0.

So, empty scalars do not exist. The basic idea seems to be that, as a scalar has no dimensions, it cannot be empty. But what about the enclose of an empty array, for instance the enclose of the empty numeric vector ($\langle \langle \rangle \rangle$) or the enclose of the empty character vector ($\langle \langle ' ' \rangle \rangle$)? It's a scalar: its rank is 0. On the other hand, it has no value. Should we call such a scalar a 'pseudo empty scalar' or a 'non-valued scalar'?

Joseph De Kerf
Roosenberg 72
B-2570 Duffel
Belgium

Year 2000 — why the Consternation?

From: Bob Brown

23 March 1997

There have been several articles in the press about major problems with computers dealing with the year "00". I haven't paid much attention because the way I have designed my applications during the past 40 years means that I know that "00" means 2000, not 1900 (or even 1800) and "01" means 2001.

Apparently more fuel has been added to the fire by ISO9000 "century compliant" standards. So I spent something under an hour modifying a couple of standard utilities that are prevalent in my application.

Internal representation

CALJUL takes any date and returns an internal representation based on the days since 31 December 1900. *CALJUL* 32597 32599 32501 yields 35149 35879 36609. The internal representation is not something that a person can read and make any sense of, but it does make it very easy to figure out how many days are between any pair of calendar dates.

With a bit of modification *CALJUL* 3251997 3251999 3252001 gives the same internal representation. Incidentally this function is in the application because back in the mid 60s (1960s that is) I was about to leave to make a

presentation in England and suddenly realized that I wanted to be able to read and write dates as dd/mm/yy as well as mm/dd/yy. The user can set a switch to determine which way makes dates more familiar in his environment.

External representation

The other function *DTFMT* takes any internal representation and generates a formatted date, with two- or four-digit years.

DTFMT 35149 35879 36609 generates
3/25/1997 3/25/1999 3/25/2001.

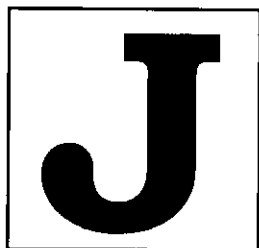
The user can flip a switch so that the result comes out 25/3/1997 25/3/1999 25/3/2001. It obviously doesn't matter to the output how the internal representation was generated. Since it took so little time to modify two very simple utility functions that always deal with all dates throughout the application, why are consultants making so much money addressing this consternation?

Vector Back Numbers

Back numbers of Vector are available from:

British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO6 4JJ

Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.



Express Yourself...

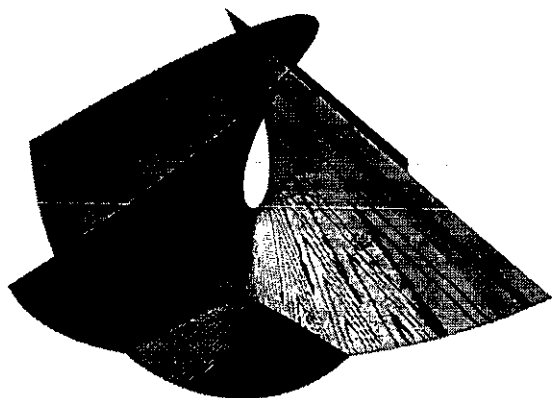
Jsoftware is a high-level, functional, multi-dimensional array processing language that is ideal for expressing mathematical algorithms.

Unlike traditional programming languages, J is based on simple and consistent rules that work directly on arrays of any size or dimension. J lets you think about the algorithms without thinking about the implementation.

J includes complex numbers, arbitrary precision integers, 2D and 3D OpenGL graphics, and a powerful GUI interface.

Coordinate matrices for Enneper's surface, shown below:

```
u=. steps _2.4 2.4 25
x=. (u - (u^3)%3) + u */ u^2
y=. |: x
z=. -/~ u^2
```



Produced by:
Iverson Software Inc.
Distributed by:
Strand Software Inc.
19235 Covington Court
Shorewood, Minnesota
USA 55331
Tel (612) 470-7345
Fax (612) 470-9202
info@jsoftware.com

www.jsoftware.com

British APL Association News

Minutes of Committee Meeting

Held before the AGM Friday 23 May 1997

The Agenda was as follows:

1. Apologies
 2. Financial Report
 3. Membership Report
 4. Management Committee for 1997/8 (possible amalgamation of roles with the Vector Working Group)
 5. Meetings: Vendor Forum, Internet cafe, speakers for BCS Southampton
 6. Education Vector
 7. Vector content: articles on J etc
 8. Insurance/future of BAA portable computer
 9. Subscription Rate
 10. AOB
-
1. Ray Cannon and Duncan Pearson sent apologies. Those present were Alan Mayer (Chair), Nicholas Small (Treasurer), Sylvia Camacho (Secretary), Ian Clark, Marc Griffiths, Ajay Askoolum, Anthony Camacho, Jon Sandles, Adrian Smith (late).
 2. Nicholas's accounts show a small loss of about £3,000 shortfall of receipts over outgoings. It was agreed that this is sustainable for the near future and that a large part of the difference would be avoided if the Education Vector mailing was ended. (see item 6)
 3. The number of Vectors mailed was 709 as against 750 last year. The loss of members is from the UK. There was the usual discussion about the call for APL expertise. The difference this year is that there is a requirement for APL programmers to vet old code for the millenium problem.
 4. It was agreed that the difficulty of filling Committee posts might be answered by asking all recipients of this e-mail to be prepared to be co-opted to do jobs as the need arises and to be prepared to discuss general management business that could not be settled over the net, if necessary, at a Vector planning meeting. Therefore only four management posts would be specifically assigned, *Chairman, Treasurer, Secretary and Editor*. It was agreed to drop the title of Education Officer (see 6 below).

5. It was agreed that meetings had proved difficult to organise and had attracted fewer punters each year. If a Vendor Forum was to be held this must be at the instigation of the Vendors themselves. The Internet cafe had aroused interest but not many people joined in on the night. BCS Southampton group had suggested that we provide "Introduction to APL Speakers" for the benefit of their members (and ours). This matter was deferred for consideration later.
6. The card included in the last issue of Education Vector, asking people to return it to confirm their continuing interest, has resulted in only about 100 returns so far. This does not justify the print or postage of Education Vector.

It was thought preferable to send copies of Vector itself for the rest of the year with the suggestion that a regular subscription should be taken thereafter.

This matter will be referred to Duncan as Vector Editor. Ian Clark endorsed this position, pointing out that he is no longer involved in the education world and that he has detected no enthusiasm to learn new skills among teachers because they are under too much pressure in other directions.

7. The balance of material in Vector was discussed with respect to the space given to papers of interest to professional APL programmers, enthusiasts for J and general more philosophical material such as Langlet articles tended to be.

It was agreed that the Editor should be careful not to overload the journal with minority interest material. It was recognised that he depends in part upon the balance of the material sent to be considered for publication

At this point, the time being close to 3.30, the meeting was joined by four members answering the invitation to be present at the AGM. Items 8 & 9 were therefore not taken.

Minutes of Annual General Meeting

1. The Minutes of the AGM of 17 May 1996 were agreed.
2. The Chairman reported on the year. Only one Committee meeting was held (19 July 1996), There was one Vendor Forum after the 1996 AGM and the Internet cafe event. The Conference held at Lancaster had been successful and had made a profit. It was supported to a small extent by the British APL Association but not run by them and they received none of the profit. The *Proceedings* have not yet been published. It is hoped that *Quote Quad*, at least, will survive.
3. The Treasurer highlighted the interesting aspects of the accounts. This did not give rise to any queries.
4. The Accounts were circulated. Nicholas pointed out that they had not yet been audited.

5. Two officers were required to be replaced, having served 3 years. Anthony Camacho was elected Chairman (proposed Alan Mayer, seconded Ian Clark). Ajay Askoolum was elected Secretary (proposed Alan Mayer, seconded Anthony Camacho). Alan Mayer agreed to take on Anthony's previous role as Publicity and Projects Officer. All other Committee members expressed their willingness to continue serving.
6. The Treasurer confirmed that John Sullivan is willing to continue as Auditor and his appointment was agreed.

At this point there was a brief pause for coffee. The new Chairman then reconvened the meeting, welcoming the members attending the AGM.

7. There was some discussion of job advertisements. Could they go on the Website? What makes a credible APL programmer? Several suggestions for initiatives were made. It was agreed that APL interpreters are not "user friendly" to new, inexperienced programmers. Adrian suggested that some form of front end could be devised, and was asked to put this forward as a possible project.
8. No decision was made on whether the BAA portable should be insured.
9. It was agreed that, as we would be spending about £3,000 less next year by not mailing Education Vector, there was no urgent reason to increase the subscription.

Sylvia Camacho

Summary of Annual Accounts 1996/97

Notes to the Accounts.

Pence figures have been omitted, so columns may not add exactly.

The value of stocks of Vector and the Software Library have not been assessed, nor has the value of the Association's computing hardware and software.

For 1996/97, figures are shown both as income and expenditure, i.e. revenues strictly relating to the activities of that year, and as receipts and payments, i.e. what goes in and out of our bank account. The comparative figures for earlier years relate to income and expenditure.

Nicholas Small
Hon. Treasurer

Summary of income and expenditure/receipts and payments:

	1996/97 (R&P) £	1996/97 (I&E) £	1995/96 (I&E) £
Income/Receipts			
Subscriptions	10592	11364	1871
BCS services	106	106	317
Bank interest	3022	3022	2900
Vector advertising (incl. VAT)	4149	4605	3240
Other	381	351	200
Total receipts	18250	19448	18529
Expenditure/Payments			
Meetings	358	358	1086
Administration	1493	2135	2340
BCS services	106	106	317
Vector production and despatch	15053	15315	15458
Education Vector supplement	3371	3446	3462
Projects	1049	1117	729
Other	569	670	488
Total payment	20149	23148	23880
Assets summary:			
Bank and other balances		52021	55771
Debtors		2188	1217
Creditors		(6151)	(5230)
Written off			(250)
Net assets		48059	51759

News from Sustaining Members

Compiled by Gill Smith

Insight Systems / Adaytum Software

Now available: APL Pipes ActiveX Client

For two years, APL Pipes has been used by customers across the world to connect APL systems together across TCP networks. APL Pipes has made it possible for APL systems from APL2000, Dyadic, IBM and Soliton to share data and make remote function calls to each other under Windows, OS/2 and Unix.

Now, we have added an ActiveX client to the family. This means that Visual Basic, Delphi, C++, Powerbuilder or just about any Windows development environment can join the party and be a client of APL-based servers. For example, the new Excel interface for our budgeting and planning application, Adaytum Planning, will use the ActiveX to make function calls to the Adaytum Planning engine.

The functionality of the APL Pipes ActiveX is similar to the DLL and OCX interfaces which are available for J and other APL products directly from the respective vendors. The difference is that the APL Pipes ActiveX works over the network without requiring Network OLE, and will support Unix, OS/2 or even Windows 3.1 servers in addition to Windows 95 and NT.

The APL Pipes ActiveX Client has been developed as a joint project between Insight Systems and Causeway Graphical Systems. In addition to the ActiveX, a simpler DLL version is also available. Both interfaces are available in 16- and 32-bit versions. At the time of sending this article to Vector, the ActiveX is in Beta Test; we expect it to be generally available in time for APL97.

Under Development: SQAPL Client Version 2.1

After numerous postponements, we have started work on a new version of the SQAPL or APL Link PRO Client. The list of features has not been set in concrete, but we expect that important new features will be support for a number of new ODBC calls: "ExecDirect", "GetInfo", "PutInfo" and "BrowseConnect". For advanced users, we will make the Environment, Connection and Statement handles available so that you can extend the product from APL by making direct DLL calls. We are also working on support for Bulk Input, but do not know yet that this can be included in version 2.1.

Adaytum Software

Picking up the theme for APL97, we are happy to share some success: The financial year 1996/1997 looks as if it will be another year of rapid growth for Adaytum. We are now established in the USA and our office in Minneapolis has grown to about 15 people in the first year. There are now more than 300 customers world wide. We have signed our first OEM agreement with a Canadian Data Warehouse vendor, and expect to open up several additional channels for distribution of our products during 1997. We expect to be recruiting a number of new developers, technical writers, designers and testers during the next year. If you have a knowledge of APL and C, and are willing to relocate to Denmark, please contact mkrom@adaytum.dk or check our Web site: www.adaytum.dk.

During the summer, we will release version 2.1 of our Windows product, which amongst other things contains support for text cells, annotation, audit trail, and support for larger models. Our Excel interface will make it possible to use our modelling engine from within the world's most popular entry-level budgetting environment: the spreadsheet. A Lotus 1-2-3 interface is likely before year end, as is the release of our own Web server product.

Causeway Graphical Systems Ltd

Firstly, we would like to welcome Marc Griffiths to Causeway. As our NewLeaf users will know, Marc helped us out with the tutorial section of the manual, but they may not be aware of the number of small flaws and inconsistencies that he turned up (and got fixed) before the product went live. Marc has come to us with a long and successful history of APL development work on major systems, but he also brings a deep knowledge of HCI (Human-Computer Interaction) principles, and an extreme thoroughness when it comes to the specification and testing of new software.

His arrival brings us close to our long-term goals in terms of team strength and skills base; we now have the ability to turn out professional code in all flavours of APL (including J), C++, Visual Basic for Applications and Java. We also have unparalleled experience in interface design, and Marc will continue his involvement in the world HCI community to ensure we stay with the leading edge, wherever it goes.

We are also delighted to announce that we have begun a long-term development agreement with Adaytum KPS to provide them with development resource and design expertise. This was announced by their managing director in his keynote speech at this year's Adaytum User Conference, as was the Excel

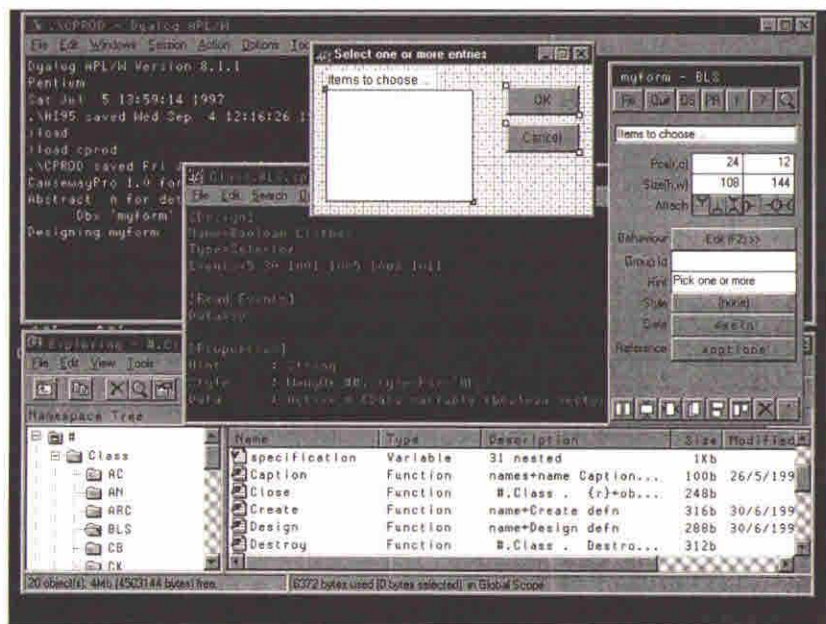
interface to *Adaytum Planning* which was demonstrated in prototype form to considerable acclaim. This work is under the technical direction of Morten Kromberg, and we expect to work increasingly closely alongside Insight Systems, sharing expertise and development resource as the need arises.

We now have an immediate and urgent need for *CausewayPro*, as we are about to begin conversion of a very complex sales-forecasting application from mainframe APL*PLUS to Dyalog/W. As reported in the last Vector, the 'engine room' of *CausewayPro* is proving fast and reliable; the current task is completing the conversion of the class-library and subjecting the converted classes to the kind of comprehensive QA checks that they never had in the first place! Marc has already been a great help here, and we shall be mailing the first beta set to all our potential users at the end of the first week of July. If you want to know more about the 'inside story' of the way CPro works with Dyalog namespaces, you can read Duncan Pearson's detailed technical description on page 94 in this Vector.

It is good to report that only very minor modifications were needed to allow our *RainPro* and *NewLeaf* namespaces to run on Dyalog/M on the Sun and RS/6000 platforms. This opens up the Unix market for us, but probably a better way of looking at it would be to say that it opens up the possibility of producing high-quality charts and reports to a very large base of existing Dyalog/M customers. As most of these sites already rely on PostScript printers, they will get the maximum benefit from *Causeway's* PostScript approach with no change to their hardware or network setup. In order to check and support our software on the Unix platform, we will shortly be taking delivery of an RS/6000 machine running Dyalog/M.

We have recently issued an interim 'bug fix' release to both *RainPro* and *NewLeaf* which is posted on the web site as 4 passworded ZIP files. The *RainPro* release adds chart subheadings and multi-line X-Axis captions; the *NewLeaf* release simply consolidates all the fixes posted on the web site since September 1996. If you haven't already updated your copy, you may download the new version (updated help file is included) and contact us for the password. If you don't have internet access, mail or fax us and we will mail you a diskette free of charge.

CausewayPro



A Development Toolkit for the APL Professional

- full integration with the Dyalog Explorer
- zero name-pollution (guaranteed)

Available only for Dyalog APL/W and APL/M

Price £400 (\$600) includes on-line help, printed
User's Manual, and one year's telephone support.

Causeway Graphical Systems Ltd.
The Maltings, Castlegate
MALTON, North Yorkshire
YO17 0DP

Tel: +44 (0) 1653 696760
Fax: +44 (0) 1653 697719
Web: www.causeway.co.uk
causeway@compuserve.com

APL Product Guide

compiled by Gill Smith

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete APL Systems (Hardware & Software)
- APL Interpreters
- APL-based Packages
- APL Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses
- World Wide Web and FTP Sites

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

All contributions and updates to the APL Product Guide should be sent to Gill Smith, at Brook House, Gilling East, York, YO6 4JJ. Tel: 01439-788385, Email: 100331.644@Compuserve.com

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Dyadic	IBM RS/6000 MD320	11,736	APL POWERstation (Greyscale) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 19" 1280x1024 Greyscale Graph Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	13,817	APL POWERstation (Colour) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	22,656	Advanced APL POWERstation 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD520	37,114	APL POWERsystem (8-users) 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape CD-ROM Drive, 18 Ports AIX, Dyalog APL (2-8 user licence)
	IBM RS/6000 MD530	72,054	APL POWERsystem (18-users) 34.5 MIPS, 10.9 Mflops RISC Processor 32Mb RAM, 1.34Gb Disk, 2.3Gb Tape CD-ROM Drive, 18 Ports AIX, Dyalog APL (8+ user licence)
	IBM RS/6000 MD540	122,842	APL POWERsystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
Optima	IBM Compatible	poa	Complete networked or stand-alone solutions including configuration installation, maintenance and commissioning.

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL Software	APL*Plus/PC Release 10	450	STSC's APL for IBM PCs & compatibles. Upgrades from earlier releases also available.
	Run-time	poa	Closed version of APL*Plus/PC which prevents user exposure to APL.
	APL*Plus II	1,395	All the features of mainframe APL*Plus for your 386PC!
	Run-time Dyalog APL	poa 1000-10,000	2nd generation APL for Unix systems
	APL2/PC	poa	IBM's APL 2 for the PC.
Beautiful Systems	Dyalog APL/W for Windows	poa	US Distributor of Dyalog APL products from Dyadic.
	Dyalog APL for Unix	poa	See Dyadic listing for product details.
The Bloomsbury Software Company	APL+PC Version 11	260	Upgrade to version 11 gives free runtime (£120 from any version)
	APL+Win v1.8	1350	A 32-bit Windows-hosted interpreter that runs under all Windows platforms including Windows 95. Note: Any user purchasing APL+Win during 1996 will receive free updates to Vn 1.8 and Vn 2.0 (user to pay carriage)
	Upgrade to Version 1.8	540	From earlier versions of APL+Win. Free update to Version 2.0 (user to pay carriage)
	Migration to APL+Win	620	from APL*PLUS II versions 4/5. Free update to Version 2.0 (user to pay carriage)
		750	from earlier versions of APL*PLUS II
	APL+DOS	1300	APL*PLUS II DOS is renamed to APL+DOS.
	Migration to APL+DOS	620 / 390	from APL*PLUS/PC or APL*PLUS II

	APL*PLUS II for UNIX	poa	APL2000's 2nd generation APL for all major Sparc and Risc Unix workstations.
	APL*PLUS VMS	poa	2nd generation APL for DEC VAX computers under VMS.
	APL*PLUS Mainframe	poa	Enhances VS APL with many high performance, high productivity features. For VM/CMS and MVS/TSO offers simple upgrade from VS APL.
Dinosoft Oy	Dyalog APL/W for Windows	poa	Finnish distributor of Dyalog APL products.
	Dyalog APL for Unix	poa	See Dyadic's listing for product details.
Dyadic	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL/W for Windows	995	As above, plus object-based GUI development tools. Requires Windows 3.0 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Altos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.
IAC/Human Interfaces			
	I-APL/Mac	13	Macintosh version of I-APL.
I-APL Ltd	I-APL/PC or clones	8	ISO conforming interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
	I-APL/BBC Master	8	As above
	I-APL/Archimedes	8	As above
	Strand Software Inc		Strand Software Inc has the sole selling rights to Iverson Software Inc products. I-APL stocks a few of these (mainly APLJWIN and the personal J products and books), but is no longer an agent.
IBM APL Products	TryAPL2	free	APL2 for educational or demonstration use. Write, fax or Email to APL Products; specify disk size desired.
	APL2 PC (US Version)	\$630	Product No. 5799-PGG. PRPQ Number RJ0411. Order from 1-800-IBM-CALL.
	APL2 PC (European Version)	£348	Product No. 5604-260. Part number 38F1753. From all IBM dealers, including MicroAPL.
	APL2 for OS/2 Entry Edition	\$185	Part No 89G1556.
	APL2 for OS/2 Advanced Edition	\$650	Part No 89G1697. Contains all facilities of the Entry Edition plus: DB2 interface; co-operative processing TCP/IP Interface; tools for writing APIs; TIME facility
	APL2 for Sun Solaris	\$1500	Product No. 5648-065.
	APL2 for AIX 6000	poa	Product No. 5765-012.
	APL2 Version 2	poa	Product No. 5688-228. Full APL2 system for S/370 and S/390
	APL2 Application Envt Vn2	poa	Product No. 5688-229. Runtime environment for APL2 packages
Insight Systems	APL*PLUS/PC	poa	APL systems marketed and supported ...
	Dyalog APL	poa	from: Dyadic, Manugistics, IBM
	APL2	poa	under: Windows, OS2 and Unix
Iverson Software Inc.	J on the Web online registration ...		
	J Educational Edition	\$50	
	J Standard Edition	\$150	
	J Professional	\$325	
	Books and accessories (discounts for reg users)		
	J Dictionary	\$50	
	J User Manual	\$50	

	J Phrases	\$40	
	J Primer	\$40	
	Set of the above 4 books	\$160	
	Concrete Math	\$40	
	Fractals, Visualization & J	\$50	
	Exploring Math	\$50	
	J User Conference Proceedings	\$35	
	Mugs, T-shirts, Mousepads	\$10 each	
	APLWIN	\$70	For 386/PC under Windows 3.1
J Austria	J	poa	Distributor for Austria and Switzerland
	Dyalog APL	poa	Distributor
	Causeway Products	poa	Distributor
	Structural Analysis Software	poa	Complete package by IG Zerkner&Handel to perform structural analysis/engineering calculations. Also suitable for dynamic problems, e.g. earthquake simulation.
Lescasse Consulting	APL+PC	poa	Lescasse Consulting is the exclusive APL2000 distributor in France and also
	APL+Unix	poa	distributes in Switzerland and Belgium. Call for price quotes.
	APL+DCS	poa	
	APL+Win	poa	
	Dyalog APL/W	poa	French distributor for Dyalog
MasterWork Software	Manugistics Products and ISI	poa	New Zealand distributor
MicroAPL	APL68000 Level I	2000	First generation APL with numerous enhancements. Multi-user version (Unix, Mirage, MCS).
	APL68000 Level II	2500	Second generation APL. Nested arrays, user defined operators, selective specification etc. Multi-user version (Unix, Mirage, MCS)
	APL68000/X	1500-6000	Second-generation APL. Nested arrays, user defined operators, selective specification, etc. Multi-user AIX version with full OSF/Motif support.
	APL68000 Level I Mac, ST, Amiga	87	First generation APL. Single user, full windowing interface, software floating point support.
	Mac, Amiga	260	First generation APL. Single user, full windowing interface, hardware floating point.
	APL68000 Level II ST	170	Second generation APL. Full windowing interface, software floating point support.
	Amiga	260	Second generation APL. Full windowing interface. Hardware and software floating point support.
	Mac	520	Second generation APL. Full windowing interface. Hardware and software floating point support.
	APL*PLUS Rel 10	450	
	APL*PLUS II V 4.0	1395	
Oasis Systems	Dyalog APL	poa	Dyadic Systems
	APL*PLUS	poa	Manugistics
	APL68000	poa	MicroAPL Ltd
	APL2	poa	IBM
Optima	Dyalog APL/W	995	Fully fledged Windows development environment.
RE Time Tracker Oy	APL+PC (APL*PLUS/PC)	poa	Complete APL+ and Statgraphics product range and links to various 3rd party products.

	APL+DOS (APL*PLUS II)		
	APL+Win (APL*PLUS III), APL+Link		
	APL+UNIX		
	APL*PLUS Sharefile		
Soliton Associates	SHARP APL for MVS	poa	for IBM MVS mainframes
	SHARP APL for Unix	poa	for IBM RS/6000 and Sun SPARC
Strand Software	Canada		
	All APL*PLUS Products	poa	All APL*PLUS products including upgrades and educational.
	Dyadic and ISI products	poa	
	USA		
	Dyadic and ISI products	poa	

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
ADAPTA Software	MPS - Master Production Schedulingpos FBS - Forecasting and Budgeting System DRP - Distribution Requirements Planning		
Adaptable Systems	FLAIR	poa	Finite loader and interactive rescheduler. Customisable full-function scheduling system. (Available outside Australia by special arrangement only.)
Adaytum Software	Adaytum Planning	poa	Full-featured Budgeting and Financial Planning system for medium to large enterprises.
The APL Group	Qualedl	\$1500-4000	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
APL Software Ltd (mainframe)	RDS	poa	Relation Data Base System
	IPLS	poa	Project Management System
	REGGPAK	poa	Regression Analysis Package
(microcomputer)	POWERTOOLS	295	Assembler written replacement function for commonly used CPU-consuming APL functions, includes a Forms Processor.
	REGGPAK	poa	Regression Analysis Package
	RDS	990	Relational Database System
Beautiful Systems	ASF_FILE	\$399	Dyalog APL/W auxiliary processor for access to APL*PLUS/PC APL component files (*.ASF).
	NAT_FILE	\$299	Dyalog APL/W auxiliary processor which emulates the APL*PLUS/PC quad-N native file subsystem for access to the DOS file system.
	DBF_FILE	\$299	Dyalog APL/W auxiliary processor for efficient block mode access to dBASE format files. Designed to get large amounts of data in and out of dBASE. Not suited for random access to small amounts of data (it does not handle keys).
	SF_READ	poa	Dyalog APL/W functions to read APL*PLUS data objects of any type or structure from *.SF style component files created by APL*PLUS II or III.
The Bloomsbury Software Company (for VSAPL)	Enhancements & Sharefile	poa	Component files, quad-functions & nested arrays for VSAPL under VM/CMS & MVS/TSO
	Compiler	poa	The First APL compiler!

(for APL2)	Sharefile/AP	poa	STSC's shared access component file system for APL2. Comparable to all APL*PLUS file systems: multi-user storage of APL2 arrays with efficient disk usage.
Causeway	Causeway for Dyalog/W	35/\$50	Registration for the Causeway platform supplied with Dyalog APL/W as shareware.
	Rain Graphics (shareware)	35/\$50	Annual registration covers full on-line documentation (Windows Help) and handy-reference for the Rain graphics workspace supplied with Dyalog/W.
	RainPro Graphics (professional)	250	The ultimate graphics toolkit for the APL developer. Adds 3D charting capability, Web publishing and clipboard support to the shareware product. Charts can be included in NewLeaf reports. Functionally compatible across Dyalog/W and APL+Win.
	NewLeaf for Dyalog and +Win	400	Frame-based reporting tool with comprehensive table-generation and text-flow support. Offers multiple master-page capability, bitmap wrap-around and on-screen preview with pan and zoom. Fully supported on Dyalog/W and APL+Win (1.8 and above)
Cinerea AB	ORCHART	250	Organization chart package for IBM APL2/PC. Full & heavily commented source code included - free integration into other applications. NB: ASCII output with line-drawing (semi-graphic) characters for boxes.
CODEWORK	HELM	poa	Decision Support system for top management. Handles large multi-dimensional tables, data analysis, EIS presentations; generates HTML and Latex output. Platforms: DOS, APL+II, Windows 3.1/95 for Dyalog APL, LAN support. Ideal for APL customisation, more than 100 installed.
H.M.W.	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping
	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling
	Menu-Bar	poa	pull-down menu for APL*PLUS/PC
HRH Systems	APL Utilities	poa	Software to transfer workspaces between APL*PLUS and Sharp, and between APL*PLUS and I-APL. Software to import IBM .ATF files to APL*PLUS.
	APL*PLUS Utilities		Public domain software, unlock locked fns, a user-friendly alternative to locking. Ins of mathematical physics, menus, and others.
IAC/Human Interfaces	SPARKS	poa	Educational simulation of electric circuit (for Apple Mac.)
	EPIDEMIC	poa	Educational simulation of spreading infection (for Apple Mac.)
	COINS	poa	Educational simulation (KS3) of coin-tossing experiment with simple stats (for Apple Mac.)
	FIBONA	poa	Educational simulation of Fibonacci's rabbits (for Apple Mac.)
I-APL Ltd	Educational workspaces	5	PC format disks with the examples from: Thomson, Espinasse (Kits 1-4), Kromberg, Jizba & FinnAPL. All the examples to save your fingers!
IBM APL Products	A Graphical Statistical System	\$250	for DOS, Product Number 5764-009
	(AGSS)	\$500	for Workstations (OS/2, Aix, Solaris), Product Number 6764-092
		\$2500	for CMS, Product Number 5764-011
INFOTROY	APL*PLUS/Xbase Interface (II/386 Version 2)	\$198	Complete package written in C. Comparable with the data, index & memo files of FoxPro, dBASE, & Clipper. Multi-user support. No DBMS license required.
	(PC Version 2)	\$98	As above for APL*PLUS/PC.
	(DLL Version 1)	\$198	The same in a DLL form! Gives your Windows applications all advantages of DLLs.
Insight Systems	IUTILS/XP	20-95	Cross-platform utility library including simple OS calls (DIR, COPY, DEL, RENAME) and DATE functions. For APL*PLUS II, APL2 and Dyalog APL under Windows, OS/2 and Unix.

	ASI	95	APL Spreadsheet Interface. "Device-Independent" spreadsheet driver supporting Excel, 123 and Quattro-Pro for Dyalog APL/W
	WinCom	95	Asynchronous comms package for Dyalog APL/W
	S2D,22D,X2X	poa	Advanced APL syntax analysis and conversion packages from Sharp and APL2 to Dyalog, and between any two APLs
	SQAPL Client	poa	Interface from APL*PLUS II, APL2 and Dyalog (Windows, OS/2 or Unix) to most SQL databases over most networks.
	SQAPL Server	poa	Makes APL*PLUS II, APL2 or Dyalog APL (Unix) available as SequelLink servers. Can be called from SQAPL clients or other applications such as Excel, C++, Smalltalk, Visual Basic.
JAD Software	JAD SMS	150-500	Software management system for APL*PLUS II based on hierarchical databases; includes full-screen interface and stand-alone functions. Price depends on number of users.
Lescasse Consulting	APL+Win Monthly Training Program	\$600	Download 50+ page document about APL+ programming each month. You also get one or more workspaces full of re-usable APL code and sometimes additional files or products.
	Advanced Windows Programming ...	\$95	200-page book plus companion disk on interfacing APL and Delphi. Contains full coverage of Delphi-2, +Win and Dyalog.
	DLL parser for +Win	\$250	Parse any Visual Basic DLL declaration file into a set of quadNA definitions. Turn constants and structures into APL variables. Available for APL+Win and Dyalog/W.
	Delphi Forms Translator	\$195	Design forms with Delphi and turn them automatically into APL programs which recreate the same form (+Win and Dyalog/W).
	APL+Link Pro	poa	ODBC Interface for APL+Win
	SQAPL Pro	poa	ODBC Interface for Dyalog APL/W
	RainPro	poa	Highly customisable 2D and 3D publication graphics for APL+Win and Dyalog APL/W
	NewLeaf	poa	Page layout and printing tools for APL+Win and Dyalog
	GraphX and ChartFX	poa	High-quality business graphics for APL+Win
	Formula One and Dyalog APL	\$95	100-page book + companion disk on how to use the Formula One VBX with Dyalog APL/W
RE Time Tracker Oy	UIT/W	poa	Comprehensive high-level Windows User Interface library for APL+Win and +I! v 5.1. Comprehensive spreadsheets, replicated fields, special field types, etc. 16 and 32 bit versions available.
	AJGRAPH	poa	Graphpak-compatible 2D graphics package for +Win and +DOS. Includes multi-window support, print and metafile support. No DLLs required.
	ECCO PRO with APL	poa	Leading group and personal information management system with comprehensive customising. Supplied with sample +Win workspace to interface to ECCO databases via DDE.
	NEWT TCP/IP SDK with APL	poa	Lead TCP/IP SDK with interfaces to all protocols. Supplied on 3 CD ROMs together with a sample +Win workspace.
	DB+	poa	Database Interface for APL+DOS under Windows. Allows combining character-based APL applications with ODBC-compliant databases such as Oracle and SQL-server..
Soliton Associates	LOGOS	poa	Application Development Environment
	MAILBOX	poa	Electronic Mail
	VIEWPOINT	poa	Report generator with interfaces to DB2 and MVS data
Warwick University	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.
Zark	APL Tutor (PC)	\$299	APL computer-based training. Available for APL*PLUS PC & APL*PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.

Zark ACE	\$99	APL continuing education. APL tutor news and hotline phone support.
APL Advanced Techniques....	\$59.95	488pp. book, (ISBN 0-9619067-07) including 2-disk set of utility functions (APL*PLUS PC format).
Communications	\$200 pc, \$500 mf	Move workspaces or files between APL environments.

APL CONSULTANCY AND DEVELOPMENT

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation, of APL products in all APL environments
Andrews	Consultancy	poa	APL programming and analysis, specialises in tree-processing algorithms.
APL Solutions Inc	Consultancy	poa	APL systems design, development, maintenance, documentation, testing and training. Providing APL solutions since 1969.
AUSCAN Software	Consultancy	poa	APL software development, training
Bloomsbury Software	Consultancy	300-750+VAT	
Camacho	Consultancy	poa	Manuals; feasibility reports and estimates; analysis and programming; APL and MS Windows applications; Sharp, ISI APL, APL*PLUS, APL2/PC and other APLs spoken. Fixed price systems a speciality
Ray Cannon	Consultancy	poa	APL, C, Assembler, Windows, Graphics: PC and mainframe
Causeway	Consultancy and Training	poa	On-site training for Causeway, RainPro and NewLeaf. Customisation and enhancement to meet local needs. Code review and pre-implementation check of Causeway applications.
Paul Chapman	Consultancy	250-500	24-hour programmer: APL, Smalltalk, C; Windows front end design a speciality.
CODEWORK	Consultancy	poa	Development, maintenance, migration, documentation of APL applications. Speciality: info systems for top executives, internet applications.
Dinsoft Oy	Consultancy	poa	Specialised in very large databases.
Dyadic	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
Evestic AB	Consultancy	poa	Excellent track record from 15+ years of APL applications in banking, insurance, and education services. All dialects, platforms and project phases. SQL expertise.
General Software	Consultancy	from 120	
GodIn London Inc	Software Development	poa	We have applications in the food manufacturing field, travel agency and airline bookings field and in product lease management.
Entropy Software Ltd	Consulting	poa	Company reporting, business graphics, Windows applications with Dyalog APL/W.
H.M.W.	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Hoekstra Systems Ltd	Consultancy	poa	APL consultancy, programming, etc. Also UNIX system administration
Michael Hughes	Consultancy	poa	Consultant with 10+ years experience with various APL Interpreters and C.
IAC/Human Interfaces	Consultancy	poa	APL on Macintosh & PC. HCI design. VDU ergonomics: EC/Health & Safety compliance.
	Documentation	100-200	On-line assistance, product demos & mock-ups, manual writing; foreign language software localization.
	Training	poa	Using I-APL for courseware & distance learning materials; Mac programming in C, APL & HyperCard.

INFOSTROY	Consultancy	poa	Moving applications between platforms. Client/server development. Multilingual user interface.
Insight Systems	Consultancy	poa	Experts in APL conversions between any combination of: APL*PLUS, APL2, Dyalog APL and Sharp APL. We are also experienced right-sizers, comfortable with networks and relational databases (that also means when NOT to use SQL) and client/server development in APL, C and Visual Basic.
JAD Software	Consultancy	poa	Systems design and development, project management, technical manuals, financial and actuarial expertise in APL.
Phil Last	Consultancy	poa	
Lescasse Consulting	Consultancy	poa	A range of consultants, experts in Windows programming, with APL+Win and Dyalog APL/W. More than 100 major APL applications already developed. We all have additional expertise in Formula One and Delphi.
Mackay Kinloch Ltd	Consultancy	200-400	Design, analysis and programming for banking, insurance, financial planning and modelling, corporate performance and legal reporting
MicroAPL	Consultancy	poa	Technical & applications consultancy.
Ellis Morgan	Consultancy	250-500	Business Forecasting & APL Systems.
Oasis Systems	Consultancy	poa	Expertise in APL system design, Project management, conversion, migration, tuning; for all APL versions (10+ years experience)
Object Oriented Ltd	Consultancy	poa	General APL consulting, code recycling — mainframe to PC, performance tuning.
Optima	Consultancy	poa	A range of consultants specialising in all areas of pharmaceutical, industrial and financial systems with 5-15 yrs experience on both PC and mainframe.
RE Time Tracker Oy	Consultancy	poa	APL application conversions, APL Windows Interfaces, APL to API-level interfacing to any system under Windows, TCP/IP network and database connectivity.
Rex Swain	Consultancy	poa	Independent consultant, 20 years experience. Custom software development & training, PC and/or mainframe.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
Shepp & Associates	Consultancy	poa	APL applications development and consulting, especially in the travel industry, especially on small computers. 25 years experience in APL programming.
Snake Island Research Inc	Consultancy	poa	APL interpreter and compiler enhancements, intrinsic functions, performance consulting. APL parallel compiler APEX is giving very good initial performance tests with convolution somewhat faster than FORTRAN.
Strand Software	Consultancy	poa	Advice on migrating to and from all flavours of APL and hardware platforms. Full-screen interface implementation, APL utilities, benchmarking, efficiency analysis, actuarial software, system development tools, valuation, pricing and modelling systems.
Sykes Systems Inc	Consultancy	poa	Complete APL services specialising in audit, optimisation and conversion of APL systems. Excellent design skills. All dialects and platforms. 17-23 years experience.
Stephen Wynn	Consultancy	poa	Most experience of financial planning, and mathematical areas: operational research, quality control, experimental design.

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Employment	poa	Contractors and permanent employees
APL-385	Typefaces	poa	Variants of the APL2741 typeface available to specification.
Bloomsbury Software Training		poa	Contact the company for details.

ComLog	Comic-Logger	\$25.95+p&p	APL*PLUS II comic-book inventory system. Shareware version available on America OnLine.
HMW	Employment	poa	Contractors and permanent employees placed.
HRH Systems	APL lessons		On-screen Interactive APL lessons for APL*PLUS, TryAPL2, Sharp and I-APL — In English or French.
	The BBS\APL:	\$24 p.a.	703-528-7617, 1200-14400b, N-8-1, 24 hours. APL educational material is downloadable free. An additional 30 megs of APL software for APL*PLUS, PLUS II, IBM, Sharp & I-APL is available to subscribers (cost is \$24/yr). Selection available on disk for \$15 post-paid. Free on-disk catalogue.
I-APL Ltd	An APL Tutorial	3	45pp by Alvord & Thomson
	An Encyclopaedia of APL (2d Ed)	6	228pp by Helzer
	APL in Social Studies	3	36pp by Traberman
	I-APL Instruction Manual (2d Ed)	3	55pp by Camacho & Ziemann
	APL Programs for the Mathematics Classroom (Springer-Verlag)	16	185pp by Thomson
	Programming in J	10	75pp by Ken Iverson
	Arithmetic	12	118pp by Ken Iverson
	Tangible math	8	36pp by Ken Iverson
	Sharp APL Reference Manual	42	349pp by Berry
	APL Press Books	poa	A comprehensive selection of early APL literature
	<i>Please note there is a packing charge of £3 per order</i>		
Oasis Systems	Training	poa	Introductory courses in APL Advanced courses for different APL versions
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.
Soliton Associates	MVSLINK	poa	Interface from Sharp APL (Unix & MVS) to non-APL data and software in the MVS environment.
	SSQL	poa	High-performance DB2 Interface for Sharp APL (Unix and MVS).

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$15
APL Club Austria	Austria	-	Quarterly Meetings	200AS(indiv), 1000AS(corp)
APL Club Germany	Germany	APL Journal	Semi-annual meetings	DM60
Ass. Francophone pour la promotion d'APL	France	Les Nouvelles d'APL		FF350 (private) FF2800 (Company)
BACUS	Belgium	APL-CAM	Conferences & Seminars	£18 (\$30)
Capital PCUG	Washington, D.C.	Monitor	Monthly meetings, occasional classes	free
Danish SIG	Denmark			
Dutch APL Assoc.	Holland	-	Mini-congress, APL ShareWare Initiative	
FinnAPL	Helsinki, Finland	FinnAPL Newsletter	Seminars on APL	100FIM(private), 30(student), 1000 (Co)
Rome/Italy SIG	Roma, Italy			
RusAPL	Moscow, Russia	APL Club	Seminars and Annual Meeting	100,000R (students 20,000)
SE APL Users Grp	Atlanta, Georgia	SEAPL Newsletter	Quarterly meetings	\$10
SovAPL	Obrninsk, Russia			
SwedAPL	Sweden	SwedAPL Nytt	Semi-annual meetings, seminars	SEK 75
SWAPL	Texas, USA	SWAPL		\$18
Swiss APL (SAUG)	Bern	Part of Qdly SI-Info		SF60 (SI) + SF20 (SAUG)
Sydney APLUG	Sydney, Australia	Epsilon	Monthly Meetings	
Toronto SIG	Toronto, Canada	Gimme Arrays!	Monthly Meetings, APL skills database, J SIG, Toronto Toolkit	\$25

ADDRESSES

ORGANISATION	CONTACT	ADDRESS, TELEPHONE, FAX, EMAIL etc.
ADAPTA Software GmbH	Michael Baas	Marlenhoehe 86, 25451 Quickborn, Germany Tel: +49 4106 60977 Fax: +49 4106 67869 Email: 101523.1757@compuserve.com
Adaptable Systems	Lola & Richard Hill	49 First Street, Black Rock 3193, Australia. Tel: +61 3 9589 5578 Fax: +61 3 9589 3220 Email: adsys@ibm.net
Adaytum Software	Douglas Rowley	13 Great George Street, BRISTOL BS1 5RR UK Tel: 0117-921 5555
Adfee	Bernard Smoor	Dorpsstraat 50, 4128 BZ Lexmond, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342 Email: adfee@concepts.nl
Andrews	Dr Anne D Wilson	23, The Green, Accomb, YORK YO2 5LL, UK Tel: 01904-792670
APL-385	Adrian Smith	Brook House, Gilling East, York YO6 4JJ UK. Tel: 01439-788385 Fax: 01439-788194 Email: 100331.644@compuserve.com
APL Bay Area APLBUG	Lewis H. Robinson (Sec)	1100 Gough St, Apt 14A, San Francisco, CA 94109, USA Tel: +1 (415) 928-2059 Email: frgp21a@prodigy.com
APL Club Austria	Harald F. Nelson	c/o N-TECH, Slebenbrunnerfeldg. 4-6, A-1050 Wien, Austria. Tel: +43 1 5458063 Fax: +43 1 5458063-17
APL Club Germany	Dieter Lattemann	Rheinstraße 23, D-69190 Walldorf, Germany. Tel: +49 6227-63469 Compuserve: 100332.1461
The APL Group Inc	Stuart Sawabini	644 Danbury Road, WILTON, CT 06897 USA. Tel: +1 (203) 762-3933 Fax: +1 (203) 762-2108 Email: ssawabini@aol.com, eshaw@aplgroup.com
APL Solutions Inc	Eric Landau	1107 Dale Drive, Silver Spring, MD 20910-1607 USA Tel: +1 (301) 589-4621 Fax: +1 (301) 589-4618 Email: elandau@cals.com
Association Francophone pour la promotion d'APL	Ludmila Lemagnen	174 Boulevard de Charonne, F-75020 Paris, FRANCE Email: lemagnen@aol.com
AUSCAN Software Ltd	Richard Procter	8 Springmount Ave, Toronto, Ontario M6H 2Y4 Canada Tel: +1-416-651-4037 Email: rjp@interlog.com Web: http://www.interlog.com/~rjp/auscan/
BACUS	Joseph De Kerf	Rocinberg 72, B-2570 Duffel, Belgium. Tel: +32 15 31 47 24
Beautiful Systems, Inc.	Jim Goff	308 Old York Road, Suite 5, Jenkintown, PA 19048, USA Tel: +1 (215) 886-2636; Fax: +1 (215) 886-4888
Bloomsbury Software	Peter Day	3-6 Alfred Place, Bloomsbury, London WC1E 7EB UK. Tel: 0171-436 9481; Fax: 0171-436 0524; CompuServe: 100010.1467
Camacho	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS UK. Tel: 0117-9730038. email: acamacho@dx.compulink.co.uk Reutemet (Sharp): ACAM
Ray Cannon		21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS UK Tel: 01252-674697 Email: 100430.740@compuserve.com
Paul Chapman		51B Lambs Conduit Street, London WC1N 3NB UK. Tel: 0171-404 5401. Compuserve: 100343.3210
Causeway Graphical Systems Ltd	Adrian Smith	5 The Maltings, Castlegate, MALTON, North Yorks YO17 0DP UK Tel: 01653-696760 Fax: 01653-697719 Compuserve: 100265.1564
Cinerea AB	Rolf Kornemark	Skyttegatan 25, S-193 00 Sigtuna, Sweden.
CODEWORK Italia srl	Mauro Guazzo	Corso Cairoli 32, 10123 Torino, Italy. Tel: +39 11 685168 Fax: +39 11 812 2652 Email: codework@inrete.it
ComLog Software	Jeff Pedneau	PO Box 5570, Derwood, MD 20855 USA Tel: +1 (301) 990-7063 Email: jeff@softmed.com
CPCUG	Lynne Sturtz	Capital PC User Group, 51 Monroe Street, Suite PE-2, Rockville, Maryland 20850-2421, USA. Tel: +1 (301) 762-9372 Fax: (301) 762-9375. Email: gjerlov@ibm.net
Danish User Group	Per Gjerlov	Lönnrothkatu 21C, 00120 Helsinki, FINLAND. Tel: +358 9 70028820 Fax: +358 9 70028824 Email: dlnsoft@dlnsoft.fi
Dinosoft Oy	Pertti Kalliojärvi	
Dutch APL Association	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342
Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL UK. Tel: 01256-811125 Fax: 01256-811130
Entropy Software Ltd	George MacLeod	Bartrum House, Ravens Lane, Berkhamstead, Herts, HP24 2DY UK Tel: 01442-878065 Email: gml@slmcorp.co.uk
Evestic AB	Olle Evero	Berteliusvagen 12A, S-146 38 Tullinge, Sweden Tel&Fax: +46 778 4410 Email: olle.evero@mailbox.swipnet.se

FinnAPL		Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland
General Software Ltd	M.E. Martin	22 Russell Road, Northholt, Middx, UB5 4QS UK. Tel/fax: 0181-854 9537
Godin London Incorporated	Gaëtan Godin	12 Gerrard St., London, Ontario, Canada N6C 4C5 Tel: +1 (519) 679-8290 Fax: +1 (519) 438-6381 Email: info@godin.on.ca
H.M.W.Trading Systems Ltd		Hamilton House, 1 Temple Avenue, Victoria Embankment, London EC4Y 0HA UK. Tel: 0171-353 8900; Fax: 0171-353 3325; Email: 100020.2632@compuserve.com
Hoekstra Systems Ltd	Bob Hoekstra	5 Thorsden Court, Guildford Road, Woking, Surrey, GU22 7GS UK Tel: 01483-771028 Email: bob@khamsln.demon.co.uk
HRH Systems	Dick Holt	3802 N Richmond St, Suite 271, Arlington, VA 22207 USA Tel: +1 (703) 528-7624; Email: dick.holt@acm.org
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Leics. LE16 8QL UK. Tel: 01536-770998 Email: 101740.1203@compuserve.com
IAC/Human Interfaces	Ian A. Clark	9 Hill End, Frosterley, Bishop Auckland, Co. Durham DL13 2SX UK Tel: 01388-526803. Compuserve: 100021.3073
I-APL Ltd	Anthony Camacho (for queries, order forms)	11 Auburn Road, Redland, Bristol BS8 6LS UK. Tel: 0117-9760036 email: acamacho@clx.compulink.co.uk Reutemet (Sharp): ACAM
IBM APL Products	Nancy Wheeler	J C Business Services (for pre-paid orders only) 56 The Crescent, Milton, Weston-super-Mare, Avon, BS22 8DU UK Tel: 01934-625181
INFOSTROY	Alexei Miroshnikov	APL Products, IBM Santa Teresa, Dept M48/D12, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 463-APL2 (=2752) Fax: +1 (408) 463-4488 Email: APL2@vnet.ibm.com Cserve: GO IBMAPL2
Insight Systems ApS	Morten Kromberg	3 S. Tulanin Lane, St. Petersburg 191186 Russia. Tel: +7 812 312-2673 Fax: +7 812 311-2184 Email: alm@infostroy.spb.su
Iverson Software Inc.	Eric Iverson	Nordre Strandvej 119C, DK-3150 Hellebæk, Denmark Tel: +45 49 76 20 20 Fax: +45 49 76 20 30 Email: info@insight.dk
J Austria	Joachim Hoffmann	33 Major Street, Toronto, Ontario, Canada M5S 2K9 Tel: +1 (416) 925-6096; Fax: +1 (416) 488-7559 Email: info@jsoftware.com
JAD Software	David Crossley	Münzgrabenstr. 68, A-8010 Graz, Austria. Tel: +43 (0)316 814529 Fax: +43 (0)316 816883 Email: JoHo@ping.at
Phil Last	Phil Last	580 Eyer Drive, #81 Pickering, Ontario, Canada L1W 3B7 Tel: +1 (905) 837-1895 Fax: +1 (905) 831-5172
Lescasse Consulting	Eric Lescasse	148 Crossbrook Street, Cheshunt, Herts, EN8 8JY UK. Tel: 01992 633807
Mackay Kinloch Ltd	Alastair Kinloch	18 rue de la Belle Feuille, 92100 Boulogne, France Tel: +33.1.46.05.10.76 Fax: +33.1.46.04.60.23 Email: eric@lescasse.com
Mercia Software Ltd.	Gareth Brentnall	519 Webster's Land, Edinburgh EH1 2RX, Scotland, UK Tel/Fax/Answerphone: 0131 228 3580 Pager/Voicemail: 01426 98 3858 Email: Alastair_Kinloch@compuserve.com
MicroAPL Ltd.	Richard Nabavi	Holt Court North, Heneage Street West, Aston Science Park, Birmingham B7 4AX UK. Tel: 0121-359 5096. Fax: 0121-359 0375
Ellis Morgan	Ellis Morgan	South Bank Technopark, 90 London Road, LONDON SE1 6LN UK Tel: 0171-922 8886 Fax: 0171-928 1006 Email: MicroAPL@microapl.demon.co.uk
Oasis Systems B.V.	Theo Zwart, Louis Rijkse	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants GU32 3PE UK. Tel: 01730-263843 Email: Ellis@mrtdm.demon.co.uk
Object Oriented Ltd	Walter G. Fil	Lekstraat 4, 3433 ZB Nieuwegein, Holland Tel: +31 30 60 66 336 Fax: +31 30 60 65 844 Email: oasisbv@pl.net or zwart@oasis.nl
Optima Systems Ltd	Paul Grosvenor	Am Grendel 2, CH-6004 Luzern, Switzerland. Tel: 41 41 418 70 70 Fax: 41 41 418 70 77 Email: info@object-oriented.com
Renaissance Data Systems	Ed Shaw	115 Brighton Road, Purley, Surrey CR8 4HE UK Tel: 0181-783 2490 Fax: 0181-783 2491 Email: 100551.1401@compuserve.com
RE Time Tracker Oy	Richard Eller	PO Box 421, Georgetown, CT 06982, USA. Tel: +1 (203) 270-9729
The Rochester Group Inc.	Robert Pullman	Miljonkatu 8 A, 2.krs, PL 363, 00101 Helsinki, Finland. Tel: +358 9-621 3300 Fax: +358 9-621 3378 Email: re@rett.fi
Rome/Italy SIG	Mario Sacco	50 S.Union St., Rochester NY 14607-1828, USA. Tel: not known. Fax: +1 (716) 271-1230
		Casella Postale 14343, 00100-Roma Trullo, Italy Email: marsac@vnet.ibm.com

RusAPL	Boris Makeev	box 971 (for Makeev), Dmitrovskoe Sh., 2, 127434, Moscow, Russia Tel/fax: +7 95 210-7783 Email: makeev@atom.al.x-atom.net
SE APL Users Group	John Marges	413 Comanche Trail, Lawrenceville, GA 30244, USA Tel: +1 (770) 972-3755 Email: seapldoc@aol.com
Shepp & Associates LLC	Andrew Shepp	1312 Washington Avenue, 6th Floor St. Louis MO 63103, USA Tel: +1 (314) 621-3272 Fax: +1 (314) 621-4267 UK Address: Claridge House, 29 Barnes High St, London SW13 9LW Tel: 0181 8768666 Fax: 0181 8758660
Snake Island Research Inc	Bob Bernecky	18 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0854 Fax: +1 (416) 203-6999 Email: bernecky@eeecg.toronto.edu
SOCAL (South California)	Roy Sykes Jr	Sykes Systems Inc, 4649 Willens Ave, Woodland Hills, CA 91364-3812 USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Soliton Associates	Laurie Howard	Soliton Associates Ltd, Groot Blanckenberg 53, 1082 AC Amsterdam, Netherlands Tel: +31 20 646 4475 Fax: +31 20 644 1206 Email: sales@soliton.com
SovAPL	Alexander Skomorokhov	PO Box 5061, Obninsk-5, Kaluga Region, Russia Tel: +7(08439)31463 Email: askom2@kaluga.rosnet.com
Strand Software Inc	Anne Faust	19235 Covington Court, Shorewood MN 55331 USA Tel: +1 (512) 470-7345 Email: amfaust@aol.com
Rex Swain	Rex Swain	8 South Street, Washington, CT 06793 USA. Tel: +1 (860) 858-0131 Fax: +1 (860) 858-9970 Email: rhswain@acm.org
SwedAPL	Christer Ullhielm	Novator Consulting Group AB, Svärdivägen 11C, S-182 33 Danderyd Sweden Tel: +46 8 622 63 50 Fax: +46 8 622 63 51 CService: 100341,404
Swiss APL User Group		Swiss APL User Group, CH-3001, Bern 1, Switzerland Email: sl@ifi.unizh.ch
Sydney APLUG	Bob Bykerk	Bob Bykerk, 100 Lamrock Ave, Bondi, NSW 2026, Australia Tel: +61 2 9385 2520
Sykes Systems Inc	Roy Sykes Jr	4649 Willens Ave., Woodland Hills, CA 91364, USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Toronto SIG	Richard Procter	PO Box 55, Adelaide St. Post Office, Toronto Ontario M5C 2H8, Canada Email: rjp@Interlog.com www.sigapl.mtlake.com/sigapl/welcome.html
Stephen Wynn		8 Clarence Gardens, Brighton, Sussex BN1 2EG Tel: 01273-327238 Email: centre@mlstral.co.uk
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: +1 (860) 872-7806

WORLD WIDE WEB SITES

ORGANISATION	URL
AFAPL	www.ensmp.fr/~scherer/langlet/ (Journal available on line)
APL2000	www.APL2000.com
APL-385	www.demon.co.uk/apl385
The APL Group Inc	www.aplgroup.com
AUSCAN	www.interlog.com/~rjp/auscan
Capital PC User Group	http://cpcug.org
Causeway	www.causeway.co.uk
CODEWORK	www.codework.de
COSY (Bob Armstrong)	www.cosy.com
Dinosoft Oy	http://yritys.kolumbus.fi/dinosoft
Dyadic Systems Ltd	www.dyadic.com
FinnAPL	http://personal.eunet.fi/pp/apl
Godin London Inc	www.godin.com
IBM APL2	www.torolab.ibm.com/apl/apl2.html
Infostroy	www.insight.dk/infostroy
Insight Systems ApS	www.insight.dk
Iverson Software Inc	www.jsoftware.com
Lescasse Consulting	www.lescassee.com
Mackay Kinloch Ltd	ourworld.compuserve.com/homepages/Alastair_Kinloch
MicroAPL Ltd	www.microapl.co.uk
RE Time Tracker Oy	www.rett.fi
Shepp & Associates	www.digitravel.com
SigAPL	www.ecm.org/slgapl
Rex Swain	www.pcn.net.com/~rhwain
The APL Group Inc	www.aplgroup.com
Toronto SIG	www.sigapl.mtnlake.com/slgapl/welcome.html
Jim Weigang	www.chilton.com/~jimw

FTP SITES

ORGANISATION	DOMAIN NAME
IBM APL2	ps.boulder.ibm.com/ps/products/apl2/
Toronto toolkit	see Toronto SIG home page
Waterloo Archive	archive.uwaterloo.ca/ftparch/languages/apl
APL-to-ASCII	archive.uwaterloo.ca/languages/apl/workspaces/aplascil

THE EDUCATION VECTOR

J-ottings 13

by Norman Thomson

Howard Peelle recently gave alternative algorithms for the verbs which appeared in J-ottings 9 to obtain best rational approximations to any given fraction. I propose to show here that there is still lots of mileage in rational numbers! This article, following a section in the encyclopaedic volume *"Concrete Mathematics"* by Graham, Knuth and Patashnik [1], describes how all positive rational fractions can be generated by a scheme called the Stern-Brocot tree, named after the two mathematicians (French and German respectively) who discovered it. Stern-Brocot trees can be described in a remarkably concise way which nevertheless embraces a large number of J features.

Rational fractions are represented by a two-item vector of co-prime integers, representing numerator and denominator in that order, so that, for example, 1.5 is represented by $3 \ 2$. The algorithm starts with the two-item vector of fractions $0 \ 1; 1 \ 0$, which represent zero and infinity. At every step a new fraction is inserted between every pair of fractions currently in the list, the new fraction consisting of the sums of the neighbouring numerators and denominators. Thus after one step, the result sequence is $0 \ 1; 1 \ 1; 1 \ 0$, and after two steps it is $0 \ 1; 1 \ 2; 1 \ 1; 2 \ 1; 1 \ 0$.

This is an algorithm which has a succinct realisation in J. The first subproblem is to compute the inserted vector of fractions at any stage, that is to add the values of all neighbours — the familiar $2+/\$ of APL2 which becomes $2\&(+/\)$ in a tacit J expression. The new layer so generated is

```
lyr=.<"1&(2&(+/\))8:>) NB. generate next layer of Stern-Brocot tree
```

```
lyr v=.0 1;1 1;1 0
```

1	2	2	1
---	---	---	---

The @: is worthy of comment. If @ were used, the verb 2&(+\) would be applied to each of the three rank 0 items of v individually to produce the three-item vector 1 2 3, with each item boxed. The leftmost <"1 is to ensure that the result is a list of fractions and not a matrix.

The new layer is then merged with the old:

malt=.:@,@(1.@#){, NB. merge x. and y., alternate elements from each

v malt lyr v

1	2	1	2	1	1	2	1	1	0
---	---	---	---	---	---	---	---	---	---

A single step of the algorithm is then

sb=.malt lyr

NB. single Stern-Brocot step

m0

0	1	1	0
---	---	---	---

sb m0

0	1	1	1	1	0
---	---	---	---	---	---

At this point the power of power comes into play. A tree n layers deep is generated by sb^: n, for example

sb^: 4 m0

0	1	1	4	1	3	2	5	1	2	3	5	2	3	3	4	1	1	4	3	3	2	5	3	2	1	5	2	3	1	4	1	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

There are never any fractions requiring cancellation, and by generating sufficient layers, any predetermined rational fraction will ultimately be generated.

The above may remind readers of a somewhat better known pattern of the rational fractions called the Farey series. The principle of this is to generate rational fractions as the lower triangular part of the table t%/t=. 1+i. n which, in rational arithmetic, looks like

```

1/1
1/2 2/2
1/3 2/3 3/3
1/4 2/4 3/4 4/4
1/5 2/5 3/5 4/5 5/5
1/6 2/6 3/6 4/6 5/6 6/6

```

....

Again all the rational fractions will eventually appear, but this time cancellation has to be performed, and duplicates eliminated, for example, there is only one distinct item on the diagonal.

Remarkably, Farey series are obtained by two small adjustments to the Stern-Brocot tree, one being to use a new start value which ensures that only one half of the tree, namely the fractional half, is used, and the other a step to eliminate denominators which exceed the chosen size for the Farey square. Cancellation and removal of duplicates has already been achieved at the Stern-Brocot stage. Another fork is called for, this time with copy as the central prong. All the numerators in a Stern-Brocot tree are found by `1s{&.}>`, and the copy vector is found by the hook `>:>`.

```
m1=.0 1;1 1      NB. start vector for fractions half of S-B tree
fy=.(>: >0(1s{&.}>))#] NB. n fy sb^:(n-1) m1 is Farey nos, n=max denom
```

```
5 fy sb^:5 m1
```

0	1	1	6	1	5	1	4	1	3	2	5	1	2	3	5	2	3	3	4	4	5	5	6	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

A Stern-Brocot sequence, thought of as a tree, is a strict binary tree, and so is traversed from its root by alternate left and right branches. The value at any node can be found very simply by associating left and right branching with the two matrices `l` and `r` given below, and applying the ordinary matrix multiplication rule:

```
lp=.*/* **      NB. inner product for matrices
```

```
l=.2 2$1 1 0 1  NB. l and r = left and right search matrices for ..
```

```
r=.2 2$1 0 1 1  NB. .. a S-B tree; use e.g. +/(>(lp&.)>)/l;r;r;l
```

The fraction to be found by following the branches left, right, right, left is:

```
+/(>(lp&.)>)/l;r;r;l
5 7
```

Given a number, rational or not, it is interesting to obtain its two closest bounds within any given Stern-Brocot tree. This is easily achieved by first converting the tree to its numeric values:

```
nvs=.>0(%/&>)      NB. Numerical Values of items in S-Brocot tree
```

```
nvs sb^:3 m0
```

```
0 0.3333333 0.5 0.6666667 1 1.5 2 3 _
```

and then finding the indexes of the lowest fraction in the tree which exceeds it in value, and the greatest fraction which does not.

```
nbrs=.+&_1 0@(+/@(>nvs)){} NB. x. nbrs y. is best approx to x. in tree y.
k=.^1
```

```
k nbrs sb^:3 m0
```

2	1	3	1
---	---	---	---

It is not a requirement that a Stern-Brocot tree start with the "natural" vector pair $0 \ 1; 1 \ 0$ — any pair of nodes which exist in the tree can be taken as argument. This means that successive rational approximations can be made using nodes further and further down the tree. For example, a succession of approximations to π can be obtained by observing that the values $3 \ 1$ and $7 \ 2$ occur in the primary tree $sb^:5 \ m0$.

Execute

```
p=.sb^:5 (3 1;7 2)
```

and observe that the familiar approximation $22 \ 7$ appears in p.

```
(<22 7)e. p
```

```
1
```

Now try repeatedly

```
p=.(0.1) nbrs sb^:5 p
```

and obtain at every step closer and closer rational approximations to π . Alternatively the left and right branch technique described above can be used to write a verb which obtains the first rational approximation to achieve a given degree of accuracy.

```
approx=.3 :0
```

```
:
```

```
p=.y.nbrs sb^:5 m0
```

```
while. x.<=.|y.-nvs p do.
```

```
p=.y. nbrs sb^:5 p end.
```

```
p=: (t=<./t=.|y.-nvs p)#p
```

```
)
```

```
0.00001 approx 0.1
```

```
355 113
```

Here in conclusion is the complete set of the nouns and verbs used:

malt=.(/:@,s(i.@#))({	NB. merge x. and y., alternate elements from each
lyr=.(<"18(2s(+/\)@:>)	NB. generate next layer of Stern-Brocot tree
m0=.0 1;1 0	NB. start vector for full S-Brocot tree
sb=.malt lyr	NB. sb^:n m0 is Stern-Brocot tree n deep
m1=.0 1;1 1	NB. start vector for fractions half of S-B tree
fy=.(>: >@(1s{s.>))#]	NB. n fy sb^:(n-1) m1 is Farey nos, n=max denom
nvs=.>@(%/s>)	NB. Numerical Values of items in S-Brocot tree
nbrs=.+s_1 0@(+/@(>nvs))[]	NB. x. nbrs y.=best approxns to x. in tree y.
l=.2 2s1 1 0 1	NB. l and r =left and right search matrices for..
r=.2 2s1 0 1 1	NB. .. a S-B tree; use e.g. +/>(ip&.>)/l;r;l
lp=.+/. *	NB. Matrix Inner Product

Quite a small amount of J to achieve all possible rational numbers!

Reference:

- [1] *Concrete Mathematics - A Foundation for Computer Science*: R.L. Graham, D.E. Knuth, O. Patashnik. Addison-Wesley 1989.

Technical Note

From: Norman Thomson

April 1997

Chris Burke has asked me to point out that my attribution to him of the J Phrase book is misleading (see Vector 13 No.3 p.117). The principal author of the J Phrase Book is Roger Hui; Ken Iverson and Eugene McDonnell were also major contributors, and other contributions were made by Chris Burke and Donald McIntyre. Also Roger is revising the Phrase Book to correct known errors, and to accommodate language changes in J3.03.

Chris would also like me to point out that *à propos* the article on windows programming in J-ottings 12, the windows driver was completely revised in J3, and is now much simpler. In particular wd 'wait' is unnecessary, and not recommended.

DNA Fingerprinting in APL

by Ian Clark

Symbolic Modelling

I'm sure I'm not the only one who subscribes to lots of worthy periodicals which pile up unread. But two which get scanned from cover to cover (besides VECTOR of course) are *New Scientist* and *School Science Review*. What I like about the latter are the short practical articles which bring mysterious fashionable subjects to life with classroom demonstrations a four-year old could understand. Like Groucho Marx I miss having a four-year-old on standby, so *School Science Review* is the next best thing.

A recent issue contained a marvellous exposition of DNA Fingerprinting by Elizabeth Burns [1]. This sparked an idea of my own. Burns clearly explains the 'cookery' of the whole business. With its intricate procedures it's like the early days of photography, but the simple principle shines through the author's account.

But how do you explain this principle to a class if you've not got the apparatus for agarose gel electrophoresis, the expensive enzymes, the radioactive tracers and the X-ray film? You could use charts and diagrams, but these really serve to mystify the whole thing. By that I don't mean they make it hard for the class to understand, but they move it out of the real world into the fairy world of coloured shapes and arrows.

Or you could use a computer running APL. For some people that's just another fairy world, but others appreciate having an 'abstraction' that's concrete enough to play with. There's not one but three or four independent bodies of literature dealing with this sort of thing, but I'm deliberately not using the terminology of any of them. Back in the 70s there were people who called it Simulation and Gaming, but that's not serious enough for today. What should we call it? The mathematics of genetic engineering? Mathematical modelling? Too frightening. It's not really mathematics, more like numerical engineering. The numbers aren't really important either. So it's symbolic engineering. Perhaps there's nothing wrong with Gaming after all. So here's my DNA fingerprinting game.

Junk DNA

Everyone knows that DNA is the machine-code for a wet computer. The Human Genome is written in DNA. It codes for all our proteins, notably enzymes, little molecular nuts, bolts and spanners which build us up from basic nutrients and keep us ticking over.

But during your lifetime and mine, perhaps only 10% of it ever gets used to create proteins, or *expressed*. What does the remaining 90% do? Nothing (not, at least, during an individual's lifetime), except to take part in chromosome replication and slow down cell-division so that we don't grow up too fast. But replication isn't a perfect process and since it doesn't do anything it soon gets in a mess. Just like some of my APL workspaces! In fact unlike the 10% of functional 'genes', or *exons* (which have to be replicated dead right, or you're dead in the water before you start) the 90% of 'junk DNA', or *introns*, are highly variable and particular to you, the individual, like your fingerprints. That's why DNA fingerprinting works. Only identical twins have the same DNA fingerprint. Non-identical twins show up quite different patterns.

We now know almost everything about the DNA 'wet computer', but only at the machine code level. It's as if aliens get hold of the complete works of Shakespeare. They quickly discover the punctuation and the letters of the alphabet, and recognise all the permissible combinations of characters into words and sentences. They classify the Bard's complete vocabulary of 10,000 or so words, and discern the correct usage of verbs, nouns and prepositions. But the remorseless tragedy of Romeo and Juliet, or why Hamlet does the weird things he does, completely escapes them. 'What does it all *mean*?' they'll ask. But until they can observe real people going about their lives and connect it in detail with their syntactic knowledge of Shakespeare's works they'll be none the wiser.

Sample data

Consider the following character matrix, taken from the workspace *DNAFING* whose functions are listed in the appendix:

- [1] *Warning to Children*
[2] *by Robert Graves*
[3]
[4] *Children, if you dare to think*
[5] *Of the greatness, rareness, muchness,*
[6] *Fewness of this precious only*
[7] *Endless world in which you say*
[8] *You live, you think of things like this:*
[9] *Blocks of slate enclosing dappled*
[10] *Red and green, enclosing tawny*
[11] *Yellow nets, enclosing white*
[12] *And black acres of dominoes,*
[13] *Where a neat brown paper parcel*
[14] *Tempts you to untie the string.*
[15] *In the parcel a small island,*
[16] *On the island a large tree,*
[17] *On the tree a husky fruit.*
[18] *Strip the husk and pare the rind off:*
[19] *In the kernel you will see*
[20] *Blocks of slate enclosed by dappled*
[21] *Red and green, enclosed by tawny*
[22] *Yellow nets, enclosed by white*
[23] *And black acres of dominoes,*
[24] *Where the same brown paper parcel -*
[25] *Children, leave the string alone!*
[26] *For who dares undo the parcel*
[27] *Finds himself at once inside it,*
[28] *On the island, in the fruit,*
[29] *Blocks of slate about his head,*
[30] *Finds himself enclosed by dappled*
[31] *Green and red, enclosed by yellow*
[32] *Tawny nets, enclosed by black*
[33] *And white acres of dominoes,*
[34] *With the same brown paper parcel*
[35] *Still untied upon his knee.*
[36] *And if he then should dare to think*
[37] *Of the fewness, muchness, rareness,*
[38] *Greatness of this endless only*
[39] *Precious world in which he says*
[40] *He lives - he then unties the string.*

I reproduce Robert Graves' poem [2] in full because it sums up the whole DNA business for me, on a variety of levels. The level which matters here is that we're going to use it in a 'symbolic model' of DNA fingerprinting.

Three blocks of text, lines [9] to [14], [20] to [25], [29] to [35], are repetitions, right? Wrong. There are significant differences. Let's construct char vectors *s1* *s2* and *s3* out of them like this:

```

s1
blocks of slate enclosing dappled/red and green, enclosing tawny/yellow nets
, enclosing white/and black acres of dominoes,/where a neat brown pape
r parcel/tempts you to untie the string./
s2
blocks of slate enclosed by dappled/red and green, enclosed by tawny/yellow
nets, enclosed by white/and black acres of dominoes,/where the same br
own paper parcel -/children, leave the string alone!/
s3
blocks of slate about his head,/finds himself enclosed by dappled/green and
red, enclosed by yellow/tawny nets, enclosed by black/and white acres
of dominoes,/with the same brown paper parcel/still untied upon his kn
ee./

```

Let's imagine they are DNA samples taken from 3 suspects. Now what about the scene-of-crime sample, *sx*? Run the function: *secret* (see Appendix) to select *sx* from one of *s1 s2 s3* at random. Now the deed is done! (Don't peep).

Burns [1] describes the chemistry of digesting a blood clot and precipitating the DNA. A series of *restriction endonucleases* (or *restriction enzymes*) are then added. Each restriction enzyme is able to find its own special short sequence of DNA *codons* and attach itself to the DNA strand at that position, whereupon it actually snips the strand there. A *codon* is a 'letter' in the DNA 'alphabet'. There are precisely 64 possible distinct codons. Remarkably this is the same order of magnitude as most human language alphabets. Using poetic verses for strands, and letters for codons, we can illustrate this process by applying the function: *restrict* to *s1* ...

```

s1 restrict 'in'
blocks of slate enclosin*g dappled/red and green, enclosin*g tawny/yellow ne
ts, enclosin*g white/and black acres of domin*oes,/where a neat brown
paper parcel/tempts you to untie the strin*g./

```

The arg: '*in*' models a particular restriction enzyme which searches out all occurrences of the substring: '*in*' and snips the strand immediately after it. Only we don't snip *z* physically. Instead we insert an asterisk at the snipping site. The function: *do* applies a series of such restrictions to its argument *z* (representing the DNA strand) so that more and more asterisks appear:

```

z+z restrict'in'
z+z restrict'the'
z+z restrict'os'
z+z restrict'of'
z+z restrict'you'
z

```

blocks of slate enclos*in*g dappled/red and green, enclos*in*g tawny/yellow
nets, enclos*in*g white/and black acres of* domin*oes,/where a neat b
rown paper parcel/tempts you* to untie the* strin*g./*

Function: *do* now models the chopping of the strand by counting the number of characters between successive asterisks...

```

i+(z='*')/ipz apositions of '*'
i+i-0,-1+i alengths of chopped strings
i
10 14 3 32 3 28 3 27 7 48 14 7

```

Each number in *i* represents a chopped strand of a particular length. Thus there are strands of 3 codons, 7 codons, 10, 14, etc.

Now electrophoresis simply applies an electrostatic field which drags the DNA strands along a ribbon of gel from the cathode (-) a little pool of DNA at one end, towards the anode (+). After a set time interval each strand has travelled a distance which depends on its length. Simple as that.

However, because DNA itself is not coloured, and there's so little of it in the gel anyway, a complex process using X-ray film and a radioactive *probe* which sticks preferentially to DNA is now used to 'develop' the chromatogram, as it were, and 'print' it onto a piece of nylon fabric. This process, called *Southern blotting*, shows up each clump of chopped strands of a given length as a dark band. To minimise calibration problems, samples to be compared with each other are processed in parallel on the same gel plate.

The function: *do* models the ribbon of gel (the chromatogram) as a string (*z* again) of *w* periods ' '. Then it sets *z[i]* to a fat blobby character to represent the dark bands caused by pieces of length *i[1]* *i[2]* etc.

```

z+w*'. ' a the fresh chromatogram strip
z[i]+'█' a add the dark bands
z
..█..█..█.....█..█.....█.....

```

We have grossly simplified things. You have to imagine the pool of DNA to be at the right hand end of *z*, not the left, because shorter strands diffuse further through the gel in a given time. The physical mechanism is complex. DNA

strands are negatively charged, but shorter strands happen to have a greater charge/mass ratio, which is what basically determines the distance they travel. But at the level of our model, this doesn't matter.

Here is a sample forensic analysis, sx being the scene-of-crime sample and $s1$ $s2$ $s3$ taken from suspects. We first 'do' sx by itself, then in comparison with each suspect sample in turn...

```
do sx
.....□.□.□.□□.....□□...□.....□.....
do s1 o do sx
..□.□.□.□.□.....□□...□.....□.....
.....□.□.□.□□.....□□...□.....□.....
do s2 o do sx
.....□.□.□.□□.....□□...□.....□.....
.....□.□.□.□□.....□□...□.....□.....
do s3 o do sx
.....□.□.□.□.□.....□.□□...□.....
.....□.□.□.□□.....□□...□.....□.....
```

from which we see that the best match with the unknown sample sx is $s2$.

Interestingly the scene-of-crime sample need only be a partial strand of the culprit's DNA. Try chopping a length off the front and back of sx :

```
sx+10+~15+sx
do s1 o do sx
..□.□.□.□.□.....□□...□.....□.....
.....□.□.□.□.....□□...□.....□.....
do s2 o do sx
.....□.□.□.□□.....□□...□.....□.....
.....□.□.□.□.□.....□□...□.....□.....
do s3 o do sx
.....□.□.□.□.□.....□.□□...□.....
.....□.□.□.□.....□□...□.....□.....
```

Sample $s2$ still gives the best match with sx , with 6 of the 7 sx bands lining up, as opposed to 2 coinciding bands with $s1$ and 4 with $s3$. In a real example, there would of course be many more bands and a few more coincidences. The court evidence (if it was going to be disputed) would be presented in much the same way as with a traditional scene-of-crime fingerprint.

Burns concludes her article by revealing that DNA fingerprinting is not only a forensic tool in criminal investigation, but is also useful in settling paternity disputes and tracing patterns of migration and evolutionary trees in biology, due to the DNA becoming more similar the closer two individuals are related. Or even

two species. Few animals carry so much junk DNA in their chromosomes as we do, so the DNA from different but related species is not that dissimilar. Studies like this are revising considerably our view of the living world based on 19th century taxonomy.

However a computer programmer may see other possibilities in the *do* procedure. It avoids the labour of comparing two sequences of symbols (bytes) for (long) common substrings by systematic search. Yes, it still does string-search, but only using 2- to 4-byte strings, so the whole process needn't take much longer than calculating a check-total. How about trying it with the object code of different release levels of a given program? How about using it on a memory dump to detect the presence of a given code-section by its characteristic bands, maybe a computer virus?

As a class exercise, try generating a random string of characters and apply *do* to see its pattern of bands, or 'fingerprint'. Experiment with different 'restriction enzymes'. Build a 'family tree' by swapping a pair of adjacent characters chosen at random to form a 'child' from a 'parent' string. Now mix the samples up and see if the class can reconstitute the family tree, or at least the degree of kinship, from the *do*-fingerprints.

References

- [1] Burns, E., DNA fingerprinting: a detective story. *School Science Review*, 77 (279), Dec. 1995, 29-35. ISSN 0036-68911
- [2] Graves, R., Warning to Children. *The Penguin Book of Contemporary Verse*, 1962, 128. Penguin Books Ltd.

Appendix

These fns are written for Dyalog APL, but will translate without trouble into an ISO standard APL (e.g. I-APL), except for the 'string-search' function, *ss*. Dyalog APL has an auxiliary processor XUTILS for useful service functions, *ss* being one of them. If you don't want to use it, a replacement *ss* is offered below. I-APL users will have to write their own which does not use Find (€). It's easy if you limit yourself to 2-char args of: *restrict*. Contact the author in case of difficulty, from whom workspace DNAFING is available free of charge for Dyalog APL.

```

v secret
[1]  assigns sx at random to one of: s1 s2 s3
[2]  sx←s's',#?3
[3]  aa...optional partial sample...
[4]  aa sx+10~15+sx
v

v z+z restrict str
[1]  arestriction enzyme, splits after seq: str
[2]  z←ss z str(str,'*')
v

v z+do z,i;w
[1]  analyse sample: z
[2]  w←PW alength of chromatogram
[3]  aApply the following restriction enzymes
[4]  a(These put '*' after their target sequence)
[5]  z+z restrict'in'
[6]  z+z restrict'the'
[7]  z+z restrict'os'
[8]  z+z restrict'of'
[9]  z+z restrict'you'
[10] aNow model the chopping of the string at each '*'....
[11] i←(z='*')/1p2 apositions of '*'
[12] i←i-0,~1+i alengths of chopped strings
[13] i←w/i alimit i to max length of strip
[14] z←wρ'.' athe fresh chromatogram strip
[15] z[i]+'□' aadd the dark bands
v

v z←ss u;x;y;v
[1]  astring rearch/replace (c/f Dyadic XUTILS auxproc)
[2]  z x y←3+u astring,target,replacement
[3]  +0;~1←v+xgz aquit if target is absent
[4]  v←~1+v11 adisplacement to target
[5]  z←(v+z),y,ss((v+ρx)+z)x y apatch in the replacement
v

v go
[1]  secret
[2]  'scene of crime...'
[3]  do sx
[4]  'suspect s1 vs sx...'
[5]  do s1 ◊ do sx
[6]  'suspect s2 vs sx...'
[7]  do s2 ◊ do sx
[8]  'suspect s3 vs sx...'
[9]  do s3 ◊ do sx
v

```

Do APL2 and J Come up to Scratch?

by Norman Thomson

There are now two broadly comparable array languages which are widely available to ad hoc problem-solvers.

Chris Burke, on p.119 of part (3) of his series explaining the J way of thinking [1], states that "...in APL it would be unusual to define new operators to solve problems". If this is really the case, I find it regrettable. In the pages immediately preceding his article [2], I demonstrated how operator programming in APL provides elegant means of programming general routines for numeric integration — more elegant in my view than their analogues in J — and further, a substantial part of 'APL2 in Depth' [3] (for which modesty forbids me to make an explicit recommendation) is devoted to illustrations of how operator programming in quite ordinary applications can help to exploit APL2 to good advantage.

The present article builds up to a further demonstration of how operator programming in APL matches the use of adverbs in J, and makes explicit comparisons between the two languages in the context of their relative suitability for solving a pragmatic problem of the sort in which they have, historically speaking, excelled other high level languages. The intention is not to extol one over the other, but rather to articulate features arising in ad hoc programming which incline the problem solver towards one or other, or in the case of J, towards one or other of the two programming styles which it supports, namely tacit versus explicit programming.

The problem chosen concerns a motor manufacturer who has decided to commit part of the advertising budget for a new model to the distribution of a scratch cards containing names of drivers occupying various places in a race. The aim is that the average sum per card given away by way of prize money should be £5. Four schemes have been proposed, as follows:

- A. Each card has four positions; if the customer reveals the winning driver's name, he wins £20.
- B. As for A, except that if he reveals the winning driver he wins £15, and if he reveals the runner-up driver he wins £5.
- C. Each card has two rows of two positions. The customer reveals one name from each row; if the first row choice is ahead of the second, the customer wins £10.

- D. Each card has three rows of two sections. If the first row selection is ahead of both of those in the other two, the customer wins £15.

In each case, 14,400 cards are to be printed, and these will in equal proportions of possible cards, there being $4!=24$ possibilities for A, B and C, and $6!=720$ for D. The programming objective is to simulate the distribution of cards under each scheme, and find which are most/least likely to overrun budget.

The first stage is to simulate customer behaviour, and the second to evaluate means and standard deviations as comparative measures of the results.

To begin with scheme A, the function *PLAYA R* simulates the scratching of R cards and calculation of the cost in prize money distributed.

```
[0]  Z←PLAYA R
[1]  Z←+/20×1=?Rp4
```

As stated above, J offers the choice of writing in either of two styles, *tacit*, which is strongly associated with function composition, and *explicit*, in which explicit references are made to the left and right arguments *x.* and *y.*, thereby broadly mirroring APL. In J, verbs model functions in the mathematical sense of transformations of data, and a train of J verbs models a single function which is a composition of the functions corresponding to its component verbs. The explicit approach in J applied to *PLAYA* results in an almost literal transliteration of the APL2 function:

```
playa=.3 : '+/20*0=?y.#4'
```

which is strongly reminiscent of what was known at one time in APL circles as "direct definition".

It is instructive to write a tacit form of *playa* :

```
+/@(20&*(0=?@ (4&(#~))))
```

and then to ask why this has so much less appeal than the explicit form. First the algorithm is bestrewn with constants, 20, 0, 4, and secondly things happen in sequence, as indicated by the two @ conjunctions, that is *n* random integers between 0 and 3 are generated, THEN each is compared with 0, THEN multiplication by 20 takes place, THEN everything is summed. This leads to the following two general observations:

1. Constants militate against tacit programming.

2. Actions in sequence militate against tacit programming.

Now consider scheme B, and compare an APL2 version and a J version (header lines in the former will be omitted, because these are identical apart from the function name):

```
[1] Z++/ε15 5×1 2="c?Rp4
```

```
playb=.3 : '+/,15 5×0 1=?y.#4'
```

In the APL2 form explicit enclosure arises because two objects, one in APL2 and one in J, which are of identical appearance in display, for example:

```
1 3 7
3 6 9
```

may be far from identical in structure. The former is an array of scalars, the latter a stack of two rank 1 objects. This leads to the observation, which was also made in [1]:

3. Explicit enclosure in APL2 may correspond to using objects of non-scalar rank in J.

Now compare J and APL2 alternatives for scheme C.

```
[1] Z++/10×0>-/"(Rp2)?"4
```

```
playc=.3 : '+/10*(>/"1)2?y.#4'
```

Scheme D in APL is

```
[1] Z++/15×FN"3?"Rp6
FN : (↑R)=⌊/R
```

The function *FN* suggests strongly the fork construction in J. There is a core function "equals", to which preliminary transformations have to be applied, namely "first" (↑) to form the left side of the comparison, and "take minimum value" (⌊) to form the right. The composite verb is {.=<./, which need not be explicitly named — bracketing the characters {.=<./ is sufficient both to define it, and embed it in an explicit or tacit program. This unnamed verb must be applied at rank 1 level to each of the "three-out-of-six" selections without repetition, followed by the usual computation of prize money:

```
playd=.3 : '+/15*(({.=<./)"1)3?y.#6'
```

The fork is an instance of more general J function compositions, defined by e.g. atop, with, and under, which leads to the generalisation:

4. J function compositions do not need to be explicitly named.

Mean and standard deviation are described as

```
MEAN : (+/R)÷ρR
SD    : (+/(R-MEAN R)*2)*0.5
```

in APL2, and as

```
sd=.mean&.*:0-mean=.+/%#
```

in J, mean is defined in the style of APL in-line assumption, which leads to a further generalisation:

5. Function (that is verb) definitions can be made in-line in J, which is not possible in APL2.

The formula

```
mean`sd/.>playa &>100#144
```

can be repeated in turn for each of the four "play" verbs, which suggests defining an adverb:

```
msd=.1 : 'mean`sd/.>x.&>100#144'
```

The use of the gerund along with the conjunction "oblique" (/.) allows two functions to be composed in the sense that the results of the composition is the join of both applied separately. In APL2 this requires two calls followed by an explicit join. This leads to a further generalisation:

6. Gerunds can join verbs at the function level where an explicit join at the result level is required in APL2.

Another way of expressing this is to say that J provides the facility to model function arrays. As an aside, the verb `playa msd` is an instance of a niladic J verb, since the argument is effectively hidden within the adverb.

The entire experiment is now described as:

```
>playa msd;playb msd;playc msd;playd msd
```

In APL2, the analogue of an adverb is an operator:

```
[0] Z+ (P STATS)R
[1] Z+ (MEAN Z),SD Z+P~R
```

leading to the matching experiment as:

```
(PLAYA STATS T)(PLAYB STATS T)(PLAYC STATS T)(PLAYD STATS T+100p144)
```

The display of results has been suppressed, so as not to spoil the excitement in advance for any readers who want to try out the experiment for themselves!

The corresponding generalisation as suggested at the start is:

7. Adverbs in J often correspond to operators in APL2.

The simulations described above were run on J for Windows under Windows 95, and on two available APL2 systems, namely APL2 for Windows 95 and APL2/PC. Execution times of course depend on the size of the data, but are consistently in the ratios 3:2:1. Presumably there is a message that if speed is important, avoid the paraphernalia of Windows!

References

- [1] APL and J (3): *Operators – Reduce, Scan, Outer Product*, Chris J. Burke, Vector 13.3 January 1997, pp.118-126
- [2] *J and APL Operators – Reduce*, Norman Thomson, Vector 13.3 January 1997, pp.112-118
- [3] *APL2 in Depth*, Norman D. Thomson and Raymond P. Polivka, Springer-Verlag, 1995

Typesetter's Note

In Dyalog APL 7.3 you can get very close indeed to the J syntax with:

```
MEAN+{(+/w)÷p w}
SD+{(+/ (w-MEAN w)*2)*0.5}
STATS+{(MEAN res),SD res+aa" w}    A Adrian Smith 29/6/97
```

The UK National Lottery

by Alan Wilson

Background

The UK National Lottery draw holds the attention of most ticket holders for the few moments the actual draw takes. For APL programmers this fascination can last a little longer. The draw picks six numbers plus a bonus number from all numbers in the range 1 to 49 inclusive and allocates prizes. The number of winning tickets assuming that each of the possible combinations is bought *once* is shown below.

Tickets Matching	Winning Tickets	Prize Allocated
All six numbers	1	First prize
Any five numbers plus the bonus number	6	Second Prize
Any five numbers	258	Third Prize
Any four numbers	13545	Fourth Prize
Any three numbers	246820	£10 prize per ticket. *

* There is a provision for reducing this prize but in the 115 draws to 25 January 1997 it has not been and is unlikely to be in future.

Each ticket costs £1.00 and has six numbers. Should there be no tickets claiming the first prize (a roll-over draw), the prize is accumulated with the following draw's prize money.

The number of winning tickets, assuming only unique combinations are sold, is arrived at as follows:

```

φ~4+(6!49) * ODDS6
▽ R*ODDS6
[1]  A Probability of picking n matching numbers in 6!49
[2]  R*(×\1,φ!6)*(φ×\1,φ37+16)*((0,16)!6)÷*/43+16
▽

```

The number for five numbers plus the bonus number, i.e. 6 reflects the fact that the one winning line of 6 can have each of the 6 numbers replaced by the bonus number.

In practice any combination can be sold any number of times, including not being sold at all.

Some Lottery Statistics To 25/01/1997

Number of roll-over draws	19
Largest first prize per ticket	£22,590,829
Smallest first prize per ticket	£122,510

Further statistics are available on the internet at <http://lottery.merseyworld.com>

Lottery Arithmetic

The probabilities of choosing the n numbers drawn are:

1	2	3	4	5	6
$7 * 1 + (\ast \setminus 1, \phi \setminus 16) \times (\phi \ast \setminus 1, \phi 37 + i \setminus 6) \times ((0, 16) \setminus 16) \div \ast / 43 + i \setminus 6 \circ \wedge \square IO \text{ is } 1$					
0.4130195	0.1323780	0.0176504	0.0009686	0.0000184	0.0000001

The odds on winning a prize are:

		3	4	5	6
$\lfloor 0.5 \div 1 + (\ast \backslash 1, \phi \vdash 6) \times (\phi \ast \backslash 1, \phi 37 \vdash 16) \times ((0, \vdash 6) \vdash 6) \div \ast / 43 \vdash 16 \circ n \square 10 \text{ is } 1$					
		57	1032	54201	13983816

This confirms the quoted odds: 57:1 for the smallest prize, 14m:1 for the first prize.

Which Combination?

If all combinations are sorted in ascending order,

1 2 3 4 5 6 is combination number 1

44 45 46 47 48 49 is combination number 13,983,816

What about 7 9 11 13 15 17?

The function *COMBNUMBER* can compute the cardinal order of any combination. Its left hand argument (*LHA*) is the specific combination and the right hand argument (*RHA*) is the maximum possible value, for example 49. This function will work with any combination in the form $n!m$, for example 6 from 49, 4 from 60 etc.

```
7 9 11 13 15 17 COMBNUMBER 49
7998474
```

```

v Z+L COMBNUMBER R;a;N;b;l;TOT
[1]  a Return combination corresponding to a cardinal number (range 1,6!49)
[2]  a+pL o N+0 o b+R o l+L
[3]  TOT+(0,+ \phi(a-1)!(a-2)+1+(b-a))[l[1]]
[4]  L0:a+a-1 o N+N+1
[5]  TOT+TOT+(0,+ \phi(a-1)!(a-2)+1+(b-a)-l[N])[l[N+1]-l[N]]
[6]  +(a>1)/L0
[7]  Z+TOT+1
v
```

Is there a link between combinations, their cardinal number and the number of winners? Here is a sample to investigate:

Combinations						Number Of Winners	Date
2	12	19	28	38	48	57	16/03/1996
7	17	23	32	38	42	133	14/01/1995
5	23	25	30	33	37	0	20/01/1996
21	29	31	32	34	48	0	13/01/1996

The reverse problem: given $6!49$, that is, 13,983,816 combinations, what is the 10,000,000th?

The function *NUMBERCOMB* will find the answer. The *LHA* is the length of the combination (6), the *RHA* is the maximum possible value (49) and the cardinal number of a combination(10,000,000).

```

6 NUMBERCOMB 49,10000000
9 19 32 34 35 45

▽ Z←L NUMBERCOMB R;a;b;NUM;STORE1;COMBS;STORE2
[1]  a Return the cardinal number of a combination
[2]  a←L ◊ b←R[1] ◊ NUM←R[2]
[3]  STORE1←+∖φ(a-1)!(a-2)+11+(b-a)
[4]  COMBS←0,1++/NUM>STORE1
[5]  LO:STORE1←+∖φ(a-1)!(a-2)+11+(b-a)-1+1+COMBS
[6]  a←a-1
[7]  STORE2←+∖φ(a-1)!(a-2)+11+(b-a)-1+COMBS
[8]  NUM←NUM-(0,STORE1)[(-1+COMBS)-1+1+COMBS]
[9]  COMBS←COMBS,1+(-1+COMBS)+/NUM>STORE2
[10]  +(L>pCOMBS)/LO
[11]  Z←1+COMBS
▽

```

All $6!49$ Combinations

Generating all $6!49$ combinations can be problematic for two reasons: it can take too long and, more than likely, it will create *WS FULL* error. If the sum of the first combination (1 2 3 4 5 6) is 21 and that of the last (44 45 46 47 48 49) is 279, how many combinations add up to the numbers between 21 and 279 inclusive?

The brilliant *COMB* function (Gilman & Rose p159) fails to compute $6 \text{ COMB } 49$, generating *WS FULL*.

```

▽ Z←L COMB R
[1]  a Return combinations of L numbers from all numbers to R
[2]  a See Gilman & Rose page 159
[3]  +(L=1,R)/LO,L1
[4]  Z←1+(0,(L-1)COMB R-1),[1]L COMB R-1 ◊ →0
[5]  LO:Z←(1R)◊.×11 ◊ →0
[6]  L1:Z←(11)◊.×1R
▽

```

An alternative strategy is called for, one which deals with the $6!49$ combinations in manageable chunks. This can be done as follows:

1. Create the combinations 3 COMB 49.
2. For every n between 3 and 46 determine
 - a) every combination given by 3 COMB 49 ending with n (call this the first half, *FH*)
 - b) every combination given by 3 COMB 49 whose first entry is $n+1$ or higher (call this the second half *SH*)
3. Combine all of a) with all of b). This is done by the function *EVERY*.

```

V R←L EVERY R
[1]  A Replicate every row of L with all rows of R, both rank
    2 or less
[2]  R←(⌊2+1 1,ρR)ρR
[3]  L←(⌊2+1 1,ρL)ρL
[4]  R←((1+ρR)⌈L),(((1+ρL)×1+ρR),0⌊ρR)ρR
V

```

This third process may well overflow a typical workspace.

However, the two sub combinations, *FH* and *SH* can be used to obtain information about the full set of combinations 6 COMB 49 without using *EVERY*. By using $\circ.=$ and $+.×$, the function *TOTALS* will produce the sum of all 6!49 combinations which range between 21 (+/1 2 3 4 5 6) and 279 (+/44 45 46 47 48 49).

```

V Z←TOTALS;N;COMB349;FH;n1;t1;SH;N1;T1;TT;NN
[1]  A Return frequency of sum of each row of 6!49
[2]  N←3 ∘ Z+279ρ0 ∘ COMB349+3 COMB 49
[3]  L0:FH+COMB349[WHERE N=COMB349[;3];]
[4]  n1++/[1](+/FH)∘.=144 ∘ t1+WHERE n1 ∘ n1+n1-0
[5]  SH+COMB349[WHERE COMB349[;1]>N;]
[6]  N1++/[1](+/SH)∘.=144 ∘ T1+WHERE N1 ∘ N1+N1-0
[7]  TT+,(t1∘.+T1)∘.=1(⌈/T1)+⌈/t1 ∘ NN+,n1∘.×N1
[8]  Z←Z+279+NN+.×TT
[9]  →(46≥N+N+1)/L0
V

```

The function *TOTALS* takes 25 seconds on a Pentium 133mhz Pc. The function *WHERE* makes *TOTALS* more readable.

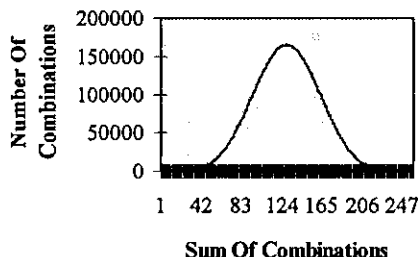
```

V R←WHERE R
[1]  A Return indices of R where elements are non zero
[2]  R←,R
[3]  R←(R≠0)/⌊ρR
V

```

The number of combinations for each sum 21 to 279 would appear to be normally distributed.

Distribution of Combinations



A Guaranteed Lottery Win?

How many combinations are needed to guarantee at least three numbers in any draw?

My correspondence with the magazine *The Actuary* prompted some discussion on this point. The minimum number of tickets to guarantee 3 numbers (£10) is not more than 174. The method for generating these numbers is as follows.

"Group A consists of 22 numbers consisting of 18 in sub-group D and 4 in sub-group E.

Group B consists of 22 numbers.

Group C consists of the remaining 5 numbers.

The lines are set out in 4 sections:

Section 1 consists of 77 lines using only the numbers in Group A.

Section 2 consists of 77 lines using only the numbers in Group B.

Section 3 consists of 18 lines with each containing all 5 numbers from Group C plus 1 of the numbers from sub-group D.

Section 4 consists of 2 lines with each containing the 4 numbers from sub-group E plus 2 of the numbers from Group C.

If any of groups A,B or C contains 3 or more selected numbers then a winning line will arise in section 1,2 or 3. If all 3 groups contain 2 of the selected numbers then section 3 will give a winning line if any of the 18 numbers from sub-group D were selected.

The only other possibility is that the 4 numbers in sub-group E include 2 numbers. These 4 numbers are used in both lines in section 4. Since the 2 lines in section 4 between them use 4 of the numbers from group C, we must have used at least 1 of the selected numbers from group C in section 4 and so would have a winning line."

The Actuary pages 33/34, January/February 1996.

The function *ALLΔTHREES* produces the required 77 lines. It is not robust. The second group of 77 lines just need 22 to be added to each entry in the first group. This can be used to verify that 3 numbers can be matched from any draw.

```

v R←ALLΔTHREES;n;MAT3Δ22;LAST;NEXT;FH;SH
[1]
[2] R←1 6p16 ∘ n+1 ∘ MAT3Δ22+3 COMB 22
[3] L0: LAST+~1 6+R
[4] NEXT←WHERE,(LAST[;13] ∈ MAT3Δ22 )[;1]
[5] L1:NEXT←NEXT+1 ∘ →(3 ∈ (,MAT3Δ22[NEXT;]) +.ε 6R)+L1
[6] L2:FH←,MAT3Δ22[NEXT;]
[7] L3:SH←,MAT3Δ22[NEXT+NEXT+1;]
[8] →(1∈FH∈SH)/L3
[9] →((FH,SH) TEST R)/L3
[10] R←R,[1]FH,SH
[11] →(77>1+ρR)/L0
v

```

Unfortunately, £174 to win £10 means I won't be giving up my day time job just yet.

Finally, a one-liner — how does an APLer check his winnings.

WL shape 6, the winning numbers

CARD shape n 6, the weekly selection

CARD+ .ε*WL*

Good Luck!

OpenGL Graphics and J

by Chris Burke

OpenGL is an industry-standard 3D graphics and modelling library originally developed by Silicon Graphics for its high-end graphics workstations, and used for applications such as CAD, medical imaging, and film special effects.

The OpenGL libraries have been ported to Windows 95/NT and are now included with the Windows distribution. With a fast PC and graphics adapter, you can have exactly the same facilities that were once available only on expensive workstations.

J Release 3.04 provides a complete interface to the OpenGL API allowing us to create high quality 3D graphics that were simply not possible with the standard Windows GDI. To see examples of this, download J3.04 for Win95/NT from www.jsoftware.com and run menu item Extras/Demos.../opengl.

Why Bother?

J and APL are great languages for specifying graphics output, and OpenGL provides us with a high quality 3D graphics facility for the first time on PCs.

An immediate use for OpenGL will be for the Plot system, which currently uses the Windows GDI. Plot has not yet been fully converted, but we intend to have this ready in time for APL97.

Another use is for the visualization of mathematical curves and surfaces, for which several examples are given in the J3.04 OpenGL demo. One of these examples can be seen in our ad in this Vector, showing a surface with a wood-grain texture.

Why OpenGL?

OpenGL is widely used and supported, and available on a variety of platforms. It is one of only two such languages that can be called directly by Windows programs. The other is Microsoft's DirectX (a.k.a. Direct3D), which has been devised specifically for game developers, but this is only available on PCs. Microsoft has its feet firmly planted in both camps, much to the annoyance of OpenGL zealots.

For us, OpenGL seems clearly the better choice - for example, we could port our interface to UNIX - but of course there is nothing to prevent us supporting any other graphics API in future.

OpenGL System

The system is provided by various DLLs (e.g. OPENGL32.DLL, GLU32.DLL) that are included in Windows NT 3.5, and in recent Windows 95 distributions. For those with earlier versions of Windows 95, the DLLs are freely available from www.microsoft.com.

OpenGL has a library of core functions for 3D graphics operations. These basic functions are used to manage object shape description, matrix transformation, lighting, coloring, texture, clipping, bitmaps, fog, and antialiasing. The names for these core functions have a `gl` prefix.

Another library (GLU) has utilities to manage texture support, coordinate transformation, polygon tessellation, rendering spheres, cylinders and disks, NURBS curves and surfaces, and error handling. These functions have a `glu` prefix.

J Interface

The J interface consists of a set of foreign conjunctions that allow direct calls to the underlying API.

The names used match the OpenGL names, except that many of the OpenGL commands have several variants differing in the number and type of their parameters, which are typically replaced by a single J function. For example, there are 30-odd OpenGL functions of the form `glColor3f` (specify colour, using 3 floating-point parameters), `glColor4ui` (use 4 unsigned integers), etc. - and one corresponding J function `glColor`, that takes care of all parameter and storage type conversions. Also, some OpenGL commands have a parameter that is a pointer to data, in which case the corresponding J parameter is given as a boxed data item.

The close correspondence between the underlying OpenGL command and the equivalent J function means that OpenGL examples, which are invariably provided in C, can be keyed directly into J. In fact the implementation was largely tested by keying in examples from the *OpenGL Programming Guide*.

All the OpenGL verbs (in fact, all windows verbs) in J have rank 1. For example, this means you can give a matrix argument to any verb, and have it evaluate each row in turn. This considerably reduces the amount of looping needed for drawing graphics.

As well as the standard `gl` and `glu` functions, there are an additional set of utilities prefixed `glu`, which provide facilities needed by the J programmer, but which are not part of standard OpenGL. For example, the function `gluARC` creates a render context for drawing into an isigraph window. These utilities, and the underlying J system, take care of most of the tedious details of creating and writing to a graphics window that have to be provided by the C programmer.

As well as the direct API facilities, the J system includes supporting utility scripts, tutorials and demos.

Efficiency

An interesting aspect of OpenGL is that using J is almost as efficient as using C. This is because all the compute-intensive work such as lighting and hidden-surface elimination is done by the API. Moreover, OpenGL can store commands in an internal form, called Display Lists, which once created, can be reused to draw a graphic. I have compared exactly the same graphics created in J and in C, and found no discernible difference in the speed of the initial display or any subsequent redrawing.

Nevertheless, the speed of OpenGL on today's hardware can be a little disappointing. Films like *The Lost World* give the impression that detailed graphics can be created and animated at high speed, but in practice a few seconds of film make have taken several hours to produce, even on the fastest machines.

Suppose, for example, that you draw a cube of a single colour, and switch on lighting so that its faces are properly visible (without lighting, you can only see the outline of the cube, and not distinguish each face). If the cube is displayed in a window occupying a quarter of the screen, it will appear to rotate smoothly when pressing and holding down a key that gives it a 5-degree turn. Now, maximize the window to full screen, and the cube rotation is noticeably slower and jerky. To get a smooth rotation you have to reduce the degrees turned in each key press, and therefore slow down the display even more. The bottleneck here seems to be the graphics card. The good news is that 3D graphics hardware is improving rapidly, largely in response to the availability of 3D graphics APIs for the PC.

Display Lists

Typically, you draw a graphic by creating several Display Lists. For example, the first basic OpenGL demo creates a graphic showing a dodecahedron in a box, with the program:

```
demo=: 3 : 0
  stdlist''
  drawviewbox makenewlist RED
  drawdodecahedron makenewlist ''
)
```

(Here, the box is coloured red, while the dodecahedron is colored at random, since no colours were given.)

This code creates six Display Lists, the first four created by `stdlist`, which:

- initialize the viewing area
- specify the viewpoint (where the observer is)
- specify the translation amounts (movements away from the origin)
- specify rotation angles (rotations around local axes)
- draw the box
- draw the dodecahedron

You then display the graphic by running the OpenGL command `glCallLists`.

Now suppose you want to rotate the graphic. All you have to do is to redefine the Display List that specifies the rotation angles and call `glCallLists` again. This is a trivial operation for J, and is completely independent of the complexity of the graphic.

The J utilities include a paint event handler that invokes `glCallLists`. This means that the graphic is drawn automatically when the window is first painted or needs repainting.

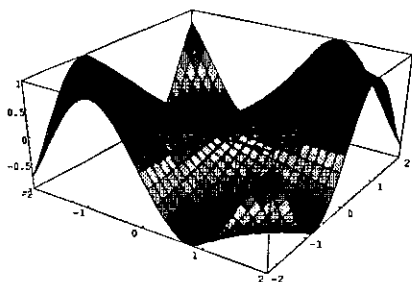
Double Buffering

OpenGL supports *double buffering* which allows you to draw a graphic to one buffer, while displaying the other. Then a swap command displays the new graphic instantly. This means that complex graphics are displayed at one go, and animations appear much smoother. The paint event handler therefore swaps the buffers after calling the Display Lists.

Plot and OpenGL

OpenGL allows us to solve display problems that became apparent when creating 3D plots in our current Plot system, which uses the Windows GDI.

Actually, it is amazing what you can do with the GDI, given its primitive drawing functions. Practically all the plots in the Plot demo use only straight lines and polygons (which in Plot are always 4-sided). You can draw 3D surfaces by tessellating them into polygons, colouring by height, and then drawing the polygons in order of their distance from the observer, to allow for hidden surface elimination. Here is an example from the Plot demo (the polygons are brightly coloured by height):



Graphics of this type are typical of PC graphics systems. However, there are a number of problems with it.

In practice, surfaces are not covered with a wire mesh, but if you try to remove it (Plot command `mesh 0`), the surface looks blotchy.

Also, surfaces are not in general coloured by height, in fact are usually of a single colour. However, if you

draw the plot using a single colour (e.g. Plot command `color green`), you lose all sense of shape.

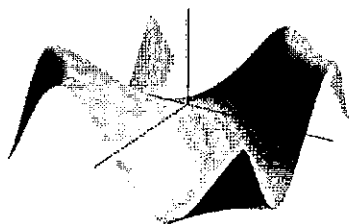
Notice the axes are drawn in a box around the graphic. This is done for good reasons. Quite apart from making it easy to show the axis labels, if you tried to draw axes through the origin, it would be almost impossible to get hidden surface elimination correct. You would either see too much of the axes, or too little.

While the simple hidden surface elimination algorithm used in Plot works well for a smooth surface such as the one shown, it can go wrong and show sides that should be hidden. To do it properly, you need to examine each pixel in the display. The graphic makes no recognition of lighting. It would be very hard to set up a light source and shade the graphic accordingly.

If you wanted to rotate the graphic, then on each movement the graphic would have to be drawn in place, and the movements would be very jerky.

With OpenGL, all these problems disappear. Here is an OpenGL version, using a single colour, with lighting, and with the axes drawn through the origin.

NURBS



The OpenGL graphic was drawn by providing a mesh of control points, then using 2D-spline interpolation to create the smooth surface. The interpolation routines are called NURBS (Non-Uniform Rational B-Splines) and are provided as part of the GLU utilities. You can adjust the interpolation by specifying the order of the spline, and the knot values that determine the influence each control point has on the surface. In practice, the default values work pretty well.

Interestingly, NURBS typically requires much fewer control points than Plot. In the examples shown, both graphics were based on a 31×31 matrix of control points. Plot typically needs a matrix of at least this size in order that the surface appear smooth. If instead, a 16×16 matrix were used, the surface would appear noticeably chunkier, and at 6×6 , the surface loses its characteristic shape. Using NURBS, there is essentially no difference when reducing to 16×16 , while the graphic looks good even at 6×6 , though there is some crinkling of the surface where the curvature is greatest.

OpenGL for APL

While we have implemented an OpenGL interface in J, it could just as well be done in APLW or APL+WIN. An enterprising programmer could probably create the interface from APL using DLL calls, though I suspect a great deal of work would be involved.

References

The official reference is the *OpenGL Reference Manual (Second Edition)*, and the official learning guide is the *OpenGL Programming Guide (Second Edition)*; both published by Addison-Wesley. The second editions of both were released this year. The Programming Guide is highly recommended, and largely dispenses with the need for the Reference Manual. Also, the contents of the reference manual are available online, with a great deal of other OpenGL material from the SGI web page www.sgi.com/Technology/OpenGL, and the OpenGL web page www.opengl.org. There is an Internet newsgroup: comp.graphics.api.opengl.

Emulation of IBM APL2 Auxiliary Processor AP207 in Dyalog APL

*by Alexander Skomorokhov and Alexander Kornilovsky
(askom@apl.obninsk.ru, askom2@kaluga.rosmail.com)*

Background

What did we miss most of all on starting to use Dyalog APL after years of APL2? Not AP124 of course.

The clear Dyalog approach to GUI based on the object-property-event triangle helps one to forget AP124 easily. The real loss was Universal Graphic Auxiliary Processor AP207. Its easy and powerful syntax seems to us more convenient for creating scientific or business charts than thinking of a line as an object. If you need something in APL you look around or do the thing you need yourself. That is why we implemented an AP207 emulator for Dyalog APL. The goals of development were:

- Easy migration of APL2 graphic applications to the Windows environment;

- Giving APL2 programmers the ability to use AP207 syntax in Dyalog APL;

- Developing a powerful and easy-to-use set of raw graphic functions for using with Dyalog APL.

Begin from the End

Let us have a look at the whole thing to start with. Table 1 shows two APL sessions which illustrate working with graphic auxiliary processor AP207, well known to APL2 users on any platform. The left side of the table represents APL2 for OS/2 session. You may get almost the same results running AP207 in the beta version of APL2 for Windows, but it is not fully implemented yet. The right hand side is a Dyalog APL session. We run Dyalog 8.0 under Windows NT (of course you will get exactly the same running it under Windows 95, or running Dyalog APL 7.2 for Windows 3.xx).

The sessions start AP207, open the graphic window, draw simple graphic objects like lines, markers and ellipses, define different fonts and write texts. The sessions look just the same except for the strange pairs "`□SVO,△SVO`" and "`+,is`". The results are shown in Figures 1-2 as copies of graphic windows made in OS/2 and

Windows respectively. You may distinguish them by window control buttons in the left upper corner and menu bars. How does it look in raw GUI? Here is part of the session up to drawing the ellipse:

```
'G' □WC 'Form' 'AP207' ('Posn' 0 0)('Size' 480 640)('Coord' 'Pixel')
    ('Bcol' (255 255 255))
'G' □WS ('Coord' 'User')('Xrange' 0 100)('Yrange' 100 0)
'G.P' □WC 'Poly' ('Points' ((0 50)(0 50)))
'G.A' □WC 'Ellipse' ('Points' 10 60)('Size' 40 40)('Start' 0)
    ('End' 360)('Fstyle' 3)
```

In general the difference from AP207 syntax is not significant. Why does AP207 seem better in our opinion?

It gets rid of many object names. Doing scientific or business charts we usually do not want to give a name to each line or sector.

AP207 syntax looks more compact and more natural for plotting charts. And, minor detail, but X should be the first, when we specify point coordinates (remember your college).

IBM APL2	Dyalog APL
<pre>207 □SVO 'G' 1 □SVO 'G' 2 G+ 'OPEN' 0 G+ 'WINDOW' (0 0 639 479 0 0 100 100) G+ 'DRAW' (50 50) G+ 'PATTERN' 3 G+ 'BEGAREA' '' G+ 'ARC' (80 30 20 20 0 360) G+ 'ENDAREA' '' G+ 'MOVE' (50 50) G+ 'WRITE' 'I like AP207' G+ 'FONTDEF' ('ROMAN' 'ROMSIM.AVF') G+ 'FONT' ('ROMAN' 2) G+ 'MOVE' (50 60) G+ 'WRITE' 'I like AP207' G+ 'FONT' ('ROMAN' 3) G+ 'MOVE' (50 70) G+ 'WRITE' 'I like AP207' G+ 'FONT' ('ROMAN' 2 90) G+ 'MOVE' (10 20) G+ 'WRITE' 'I like it even rotated' G+ 'MARKER' 3 G+ 'MARKER' (3 2710 80 30 80 15 90) X+80 80+[2]730 2p20 G+ 'DRAW' X</pre>	<pre>207 △SVO 'G' 1 △SVO 'G' 2 G is 'OPEN' 0 G is 'WINDOW' (0 0 300 350 0 0 100 100) G is 'DRAW' (50 50) G is 'PATTERN' 3 G is 'BEGAREA' '' G is 'ARC' (80 30 20 20 0 360) G is 'ENDAREA' '' G is 'MOVE' (50 50) G is 'WRITE' 'I like AP207' G is 'FONTDEF' ('FR' 'TIMES NEW ROMAN') G is 'FONT' ('FR' 16) G is 'MOVE' (50 60) G is 'WRITE' 'I like AP207' G is 'FONT' ('FR' 36) G is 'MOVE' (50 70) G is 'WRITE' 'I like AP207' G is 'FONT' ('FR' 26 90) G is 'MOVE' (10 20) G is 'WRITE' 'I like it even rotated' G is 'MARKER' 3 G is 'MARKER' (3 2p10 80 30 80 15 90) X+((pX)p80 80)+X+730 2p20 G is 'DRAW' X</pre>

Table 1. Two sample APL sessions

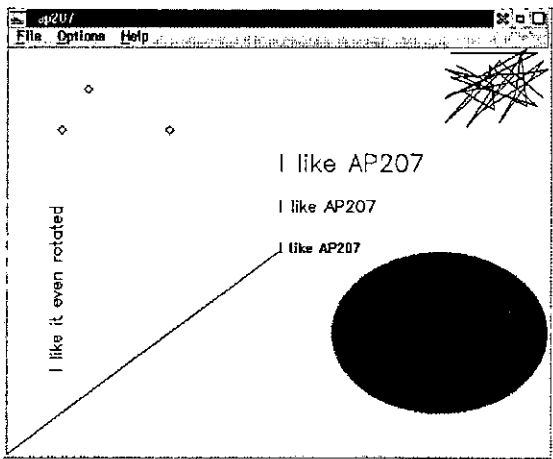


Figure 1. APL2 for OS/2 graphic window

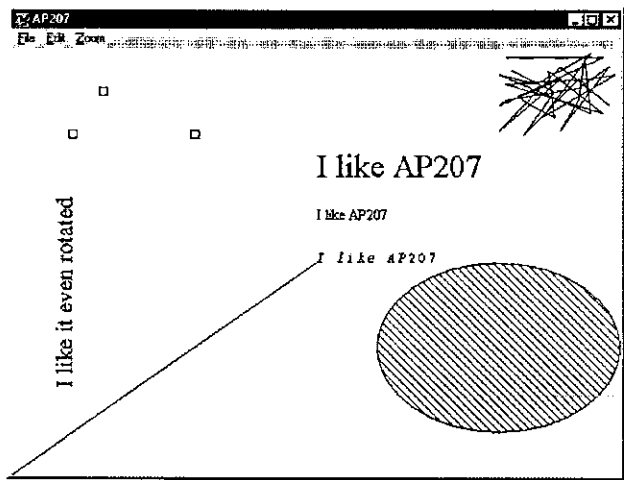


Figure 2. Dyalog APL graphic window

How it Works

The AP207 emulator is implemented as a single namespace and two interface functions. Each AP207 command is implemented as a separate Dyalog APL function (sometimes with subfunctions) placed in namespace AP207. Utility

function `ΔSV0` creates the “shared” variable to pass AP207 calls to namespace functions and utility function “`is`” translates AP207 calls to call the appropriate Dyalog APL function and provides a given argument for it. More details follow.

Giving you the AP207 feeling

To start “real” AP207 you use system function `□SV0` (Shared Variable Offer). The purpose is to establish an interface (shared) variable to communicate to an external program (AP207.COM for APL2/PC, for instance). We follow this approach to give a feeling of working in “real” AP207, keep the syntax and simplify migration. But in our case utility function `ΔSV0` establishes an interface variable to communicate to functions stored in the AP207 namespace. The function is as follows:

```
[0]  Z←(L)ΔSV0 SHV;□ML
[1]  ⍝ Replacement for □SV0 in AP207 emulator
[2]  □ML←3
[3]  Z←1 ⍝ Initial degree of coupling
[4]  ⍝((0=□NC'L')∧2=□NC SHV)/'Z+2 ♂ →0' ⍝ Final degree of coupling
[5]  →(0=□NC'L')/0 ⍝ Exit if was check of degree of coupling
[6]  □CS'AP207' ⍝ Change to AP207 namespace
[7]  NAME←SHV ⍝ Remember name of "shared" variable
[8]  ΔPNAME←+'. '□WG'PrintList' ⍝ Default printer
[9]  □CS'#' ⍝ Change to root namespace
[10] Z←⍝SHV,'+0' ⍝ Create "shared" variable
```

It just assigns the “shared” variable name to global variable `NAME` in namespace `AP207`, gets the name of the default printer, and assigns an appropriate return code to the shared variable.

To follow AP207 syntax and to communicate to functions in namespace `AP207` we had to replace normal APL assignment by utility function “`is`”. This function checks if the AP207 call is valid, calls the appropriate function from namespace `AP207` with given argument and assigns a return code to the shared variable:

```
[0]  I is r;t;res;k;□ML
[1]  ⍝ Replacement for assign (left arrow) in AP207 emulator
[2]  □ML←3
[3]  ⍝(2|pr)/'t+33 ♂ →EXIT' ⍝ AP207 length error
[4]  ⍝(∼v/2 3ε=r)/'res+32 ♂ →EXIT' ⍝ AP207 domain error
[5]  □CS'#.',t+'AP207' ⍝ Change to AP207 namespace
[6]  I←NAME ♂ res←'' ⍝ Get name of "shared" variable
[7]  LOOP:k←2+r ⍝ Get command name and arguments
[8]  ⍝(∼cv/ΔLISTεk[1])/'res+31 ♂ →EXIT' ⍝ AP207 invalid command
[9]  ⍝ Call APL function named as command (k[1]) with args k[2]
[10] res,←⍝ε't ',k[1],'+k[2]' ⍝ and get AP207 return code
```

```

[11]  + (0*pr+2*r)/LOOP      a Continue if multiple-call sequence
[12]  EXIT:[]CS'#'           a Change to root namespace
[13]  1, '+res'              a Return result to "shared" variable

```

The core is line [10], where the AP207 function is executed.

Do real work

The real work of drawing is done by functions stored in namespace AP207. Each command function checks if the argument is valid and produces the needed graphic result using raw GUI. If the argument is invalid the function returns a numeric error code the same way as AP207 does. Here we show as examples a couple of command functions from namespace AP207. Some of them are trivial, as function MOVE below, which changes or reports the current cursor position:

```

[0]  Z+MOVE R
[1]  a AP207 'MOVE' command to ask or set cursor position
[2]  a R - empty vector (ask), or 2 items numeric vector (set)
[3]  + (~ΔFLAG)/0*Z+51 a Device was not opened
[4]  + (0=ρR)/REPORT a Report current cursor position
[5]  + (1≠ρR)/0*Z+34 a Shared variable rank error
[6]  + (1≠R)/0*Z+32 a Shared variable domain error
[7]  + (2≠ρR)/0*Z+33 a Shared variable length error
[8]  ΔCURR+R a Set new cursor position
[9]  REPORT:Z+0, cΔCURR a Return code

```

Global variable ΔCURR is used for cursor position reference. Another function QWRITE is a query of text size and position, useful before actually writing on the screen. This function uses the Dyalog APL system function []WS to get the GUI object properties.

```

[0]  Z+QWRITE R
[1]  a AP207 'WRITE' command - query position and size of string
[2]  a returns x,y coordinates of the lower-left, upper-left and
[3]  a lower-right corners of the string R
[4]  + (~ΔFLAG)/0*Z+51 a Device was not opened
[5]  + (1≠R)/0*Z+32 a Shared variable domain error
[6]  + (1≠ρR)/0*Z+34 a Shared variable rank error
[7]  a Set coordinates for parent object
[8]  ΔDESTINATION []WS'COORD' 'PIXEL'
[9]  a Get text size in pixels
[10] L+ΔDESTINATION []WG='TEXTSIZE'(R)ΔFNAME
[11] a Restore coordinates for parent object
[12] ΔDESTINATION []WS'COORD' 'PROP'
[13] a Calculate result
[14] Z+0, cε(ΔCURR), (ΔCURR+1(1>L)*ΔY), ΔCURR+(ΔX ΔY)*ΔL

```

Global variable `DESTINATION` keeps the name of the parent object for graphics. Other functions may be larger. For instance functions `DRAW` and `MARKER` have about 60 lines, but in general it is quite simple stuff.

Sample PLOT Functions

Let us illustrate using AP207 syntax in programming chart functions in Dyalog APL. We created a set of basic charts. Here we show the function `PLOT` to illustrate X, Y dependencies. `Lines`, which draws axes, is omitted to simplify illustration.

```
[0] {L}PLOT XY;X;Y;C
[1]  a Simple function for various XY plots (axis stuff is omitted)
[2]  z(1:=XY)/'X+',c;pXY o Y+,cXY'      a Y Vs 1pY
[3]  z(2:=XY)/'X+('1+pXY)p<+XY o Y+1+XY' a (Y1 Vs X) and (Y2 Vs X) and...
[4]  z(3:=X z(3:=XY)/'X+'XY o Y+2>'XY'   a (Y1 Vs X1) and (Y2 Vs X2) and...
[5]  z(0=NC'L')/'L+(pY)p1'      a Default are lines
[6]  + (2=NC'AP207')/SKIP      a Skip if already shared
[7]  AP207+207 ASVO'AP207' a Start AP207
[8]  AP207 is'OPEN' 0      a Open graphic window
[9]  SKIP:AP207 is'CLEAR' 0 a Clear previous draw
[10] a Define window and viewport
[11] AP207 is'WINDOW'(0 0 100 100,{'/'e"X Y),{'/'e"X Y)
[12] C+'BLACK' 'BLACK' 'BLUE' 'RED' 'GREEN' 'BROWN'
[13] LOOP:AP207 is'COLOR'(+C+1pC) a Color for first XY pair
[14] z(1+=L)/' AP207 is'DRAW'({>+"X Y)' a Line if L=1
[15] z(0+=L)/' AP207 is'MARKER'({>+"X Y)' a Markers if L=0
[16] (L X Y)+1+"L X Y      a Drop first XY pair
[17] + (0<pY)/LOOP
```

The function looks quite simple (and it is!). But it supports powerful argument types. It may be used to draw one or more dependent variables *vs* the same independent variable, or a set of pairs of dependent variables *vs* its independent variable. Each pair may be drawn as points or lines. We used this argument convention for years and found that it serves everyday needs very well.

The details of possible arguments in call `L PLOT R` are presented in the table below:

<code>L=1</code>	Draw lines			
<code>L=0</code>	Draw markers			
<code>L</code> – logical vector	Item <code>L[I]</code> defines line (<code>L[I]=1</code>) or marker (<code>L[I]=0</code>) style to draw <code>Y[I]</code>			
	<code>L1</code>	<code>L1</code>	<code>L1 L2...LN</code>	<code>L1 L2...LN</code>
<code>R</code>	<code>X</code>	<code>X Y</code>	<code>X Y1 Y2...YN</code>	<code>(X1 Y1)(X2 Y2)...(XN YN)</code>
<code>=R</code>	1	2	2	3

Below we illustrate using the function `PLOT` for regression models analysis.

Regression models

Let us create sample variables in accordance with the model $y = a_0 + a_1x + a_2x^2$:

```
X←1:6
Y←1 -.2 .3+.×qX°.×0 1 2
```

And add noise (Normal distribution with mean 0 and variance 1) to the dependent variable.

```
Y1←Y+(pY) RAND 'N' 0 1
```

The goal is to estimate (using the Least Squares Method) coefficients of models:

$$\begin{aligned}
 y &= a_0 \\
 y &= a_0 + a_1x \\
 y &= a_0 + a_1x + a_2x^2 \\
 y &= a_0 + a_1x + a_2x^2 + a_3x^3
 \end{aligned}$$

and draw sets of real and predicted data. Estimation was made using points 1–4 (learning subset) to keep points 5,6 (testing subset) for models verification.

Here is the Least Square Estimation for all models at once:

```
A←(c4+Y1)@((c4+X)@. *"-1+1"-14.
```

We calculate predicted values for 60 points to smooth polynomial drawing.

```
Yp←A+.x"@((c.1×160)@. *"-1+1"-14
```

Now assign axis labels:

```
XLABEL←'Independent variable X'
YLABEL←'Dependent variable Y'
```

And call the function to draw the original 6 points of data as markers and 4 regression model predictions (with 60 points) as lines:

```
0 1 1 1 1 PLOT (cX Y1),(c.c.1×160),"c"Yp
```

The resulting plot is shown in Figure 3.

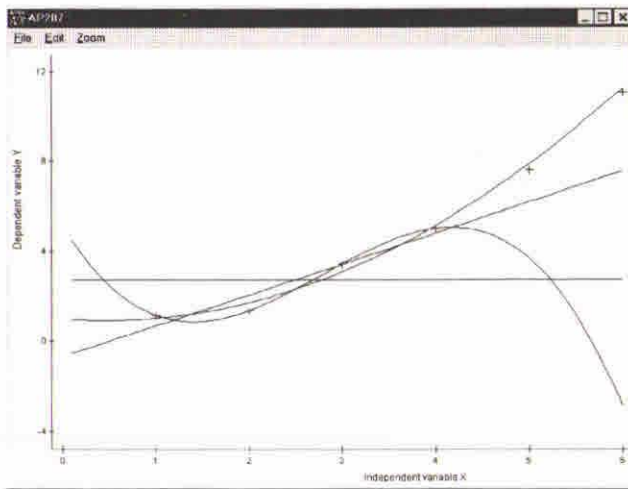


Figure 3. Regression models

Time series

In addition to the function *PLOT* we wrote function *PLOTN* to present *N* dependent variables *vs* the same independent variable; each is a separate window and viewport. This function is especially useful for joint analysis of time series. We illustrate using this function with data of power and control rod positions in a nuclear reactor during special measurements:

```

XLABEL←'Time'
YLABEL←'Control rod 1' 'Control rod 2' 'Power'
PLOTN TIME AP1 AP2 W

```

The result is shown in Figure 4.

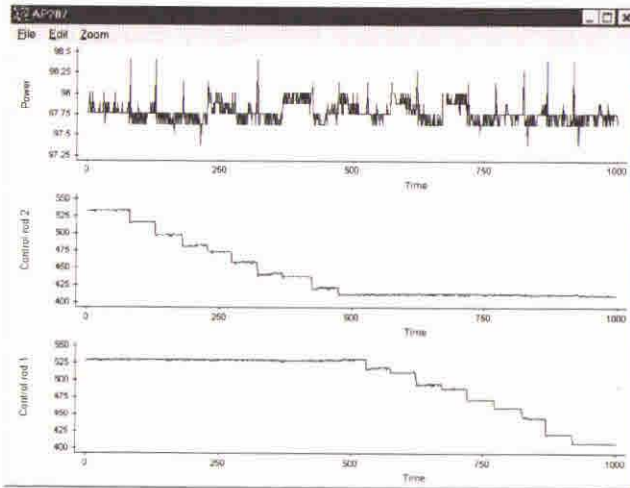


Figure 4. Time series plot

Another Look at the Stuff

Functions `ΔSV0` and `"is"` `AP207` calls via "shared" variables are good if you want to migrate APL2 graphic functions directly. But if you write applications in Dyalog APL you probably do not want to play "auxiliary processor" and "shared variable" games. Look at namespace `AP207` as a source of useful and universal graphic utilities. Call them directly as `AP207.FUNCTION`, but keep `AP207` conventions for arguments. Just bring the clear and powerful syntax to drawing charts in Dyalog APL.

Here is an illustration. Let us start from a piece of "normal" GUI to create a main form and 4 containers (Static objects) for charts:

```

'G' □WC 'Form' 'Multiple Plots'(0 0)(80 80)
'G.P1' □WC 'Static'(0 0)(50 50)
'G.P2' □WC 'Static'(0 50)(50 50)
'G.P3' □WC 'Static'(50 0)(50 50)
'G.P4' □WC 'Static'(50 50)(50 50)

```

Now back to namespace *AP207*. The first part is a simple chart:

```
AP207.USE 'G.P1'
CHART 110
```

Function *USE* is the *AP207* command to select an output device and *CHART* is a sample function, written in *AP207* as function *PLOT* above. An example pie chart follows:

```
AP207.USE 'G.P2'
PIE 10 20 30 40
```

Select the third static object and draw a histogram of random digits with Normal distribution and Gaussian curve:

```
AP207.USE 'G.P3'
X←1000 RAND 'N' 0 1
21 HIST X
```

Now let us show a piece of raw *AP207*:

```
AP207.USE 'G.P4'
AP207.WINDOW (0 0 100 100 0 -1 300 1)
X←1300
Y←(φ(÷900000)×X*2)×10.3×X
```

We defined a viewport suitable for a sine-curve modulated by a parabola function. Then draw this data:

```
AP207.DRAW (X,[1.5]Y)
AP207.DRAW (X,[1.5]φ(÷900000)×X*2)
AP207.DRAW (X,[1.5]-1×φ(÷900000)×X*2)
```

Now define the font and write text with angle 45°:

```
AP207.FONTDEF ('FAR' 'Times New Roman')
AP207.FONT('FAR' 18 45)
AP207.COLOR 'BLACK'
AP207.MOVE (200 .5)
AP207.WRITE 'Sample plot'
```

The final result is shown in Figure 5.

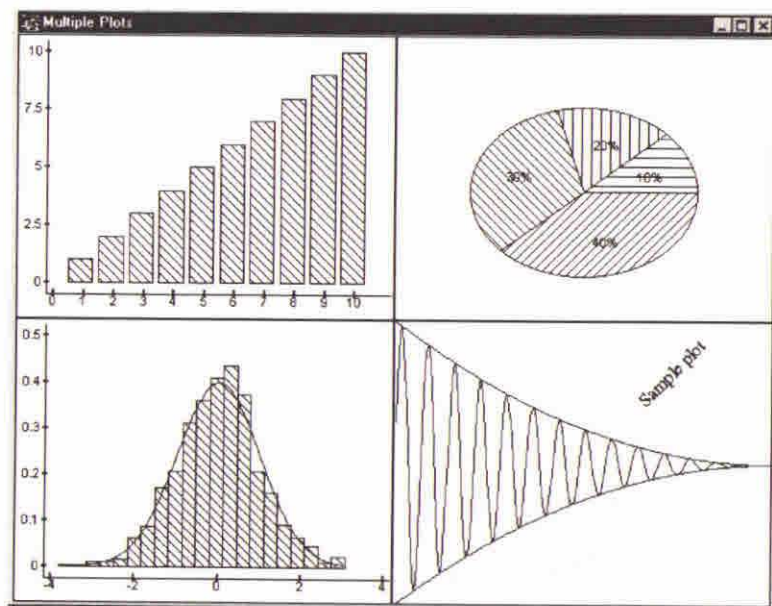


Figure 5. Sample multiple plots

Easy Migration

How to migrate AP207 code from APL2 to Dyalog APL with the AP207 emulator?
You should:

1. Find in any function string "207 □SVO" and replace it by string "207 ΔSVO".
2. Extract name of variable in string "207 □SVO SHV"
3. Find in all functions all strings "SHV+" and replace them by "SHV is "

Function *MIGRATE* does that for you.

Function *MIGRATE*

```

      VMIGRATE[0]V
[0]  MIGRATE;F;C;R;D;SHV
[1]  A Migrate APL2 AP207 code to Dyalog APL
[2]  A Find function with AP207 offer
[3]  F←(1ε"(c'207 □SVO')ε"□CR"R)/R+((c[2]□NL 3)~'' ')-c'MIGRATE'
[4]  →(0=ρεF)/0 A There is no share with AP207
[5]  A Replace {207 □SVO 'SHV'} by {207 ΔSVO 'SHV'}
[6]  C←('207 □SVO')εF+□VR+F
[7]  C←C11
[8]  SHV+1→(-'''=D)←D+(C+8)+F A Get name of shared variable
[9]  F[C+4]←'Δ'
[10] A Replace {□SVO 'SHV'} by {ΔSVO 'SHV'} if exists
[11] C←('□SVO ''',SHV,'')εF
[12] Δ(1εC)/'F[C11]←'Δ''
[13] □FX F
[14] A Find assignments to shared variable
[15] F←(1ε"(cSHV,'+')ε"□CR"R)≠R
[16] L1:R+□VR+F A Replace {SHV+} by {SHV is}
[17] (((-ρSHV)φ(SHV,'+')εR)/R)+c' is '
[18] □FXεR
[19] →(0<ρF+1+F)/L1

```

Bring in the content of a migrated APL2 workspace using the Dyalog APL utility workspace APL2IN. Then copy and run *MIGRATE*. Then copy the namespace *AP207* and that works.

GRAPHPAK

What is GRAPHPAK? It is a well known powerful graphics package. It is 156 functions and 66 variables. It is 153 pages of manual. It is a good example to test how migration works. Figures 6 and 7 are two examples taken from GRAPHPAK DEMO. Many people first saw these figures in mainframe times. Now you may see them in Dyalog APL for Windows.

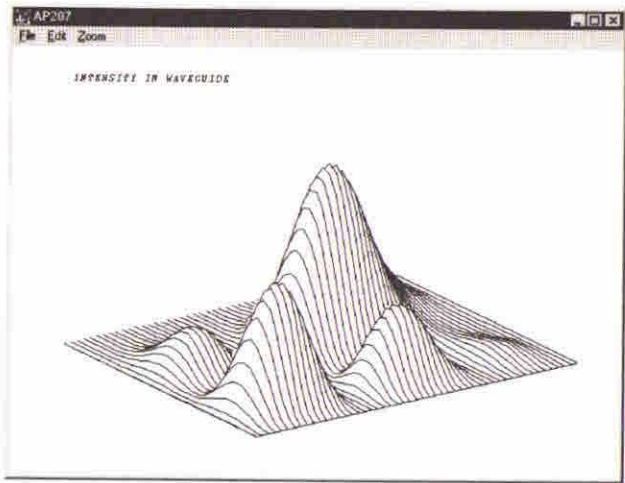


Figure 6. GRAPHPAK DEMO Waveguide

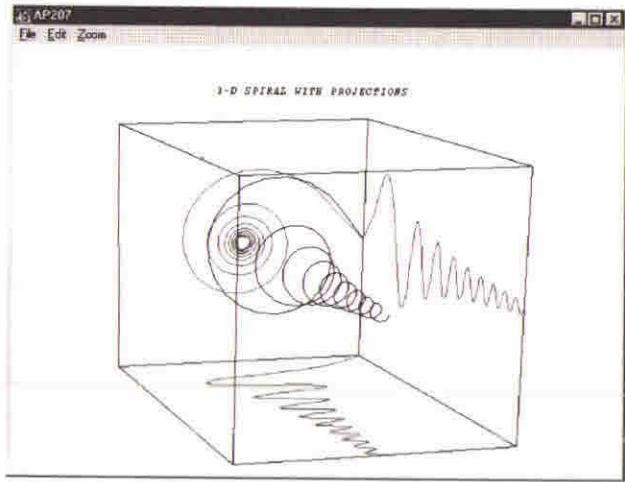


Figure 7. GRAPHPAK DEMO Spiral

AP207 Tutorial

For those who never used AP207 before we wrote a tutorial workspace. The organization idea is taken from the GUI Tutorial workspace in Dyalog APL. You may read an explanation, execute sample code by pressing a button and see the

graphics in a separate window. The starting window of the tutorial is shown in Figure 8.

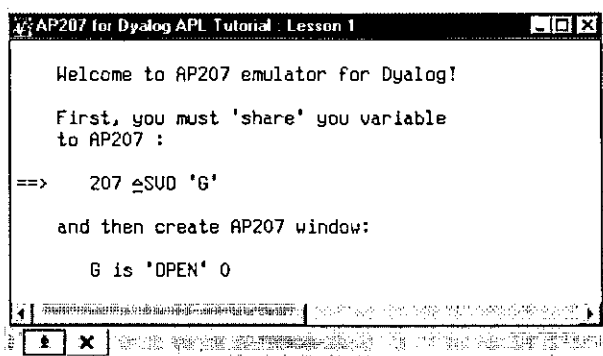


Figure 8. AP207 Tutorial window

Conclusion

We didn't measure the performance in any way, but we feel ourselves quite comfortable using the AP207 emulator. No slowness in comparison to "real" AP207 under APL2 for DOS or OS/2 is felt.

Every Dyalog APL user knows the RAIN product of Causeway Graphical Systems. To say in one word it is fine. Why use the AP207 emulator? There are two reasons. First, it gives you lower level tools and thus the freedom to create any special plot output you like. Second is the direct action of AP207 commands. We mean that RAIN functions create PostScript code. It's very good for high quality printing output. But not so good for on-line charts, for instance.

In some ways the AP207 emulator is "a better than real prototype". We mean the use of Windows facilities. It was easy to implement zooming, copy the content of the graphic window to clipboard, save it as bitmap file, select a printer and print. Pay attention to the menu bar of windows shown in the figures.

References

- [1] Rex Swain, *Migration From APL2 to APL/W*, Vector, Vol. 12 No.4 p.64.
- [2] AP207 - *The Universal Graphics Auxiliary Processor*, in "APL2 for OS/2: User's Guide", Version 1.0, IBM Publication SH21-1091-00, 1994.

The IML Matrix Language: A Belated Introduction

by Donald B. Pittenger (dbpitt@msn.com)

This is an introduction to yet another array-processing language. Why even bother? Because it is one that might be acceptable to Information Systems bureaucrats, and this could affect your work situation, perhaps your career.

Let me explain by using my own experience as an example. My own business is creating and selling international demographic and household income projections, mainly to car makers. The main systems are written in APL, though my new county/province system is in J. In order to have more predictable income to pay for my children's extravagant lifestyles at university, I also create demographic systems for a government agency: this is where the trouble starts.

Information Systems (IS) bureaucrats are fairly tolerant of researchers in my group 'doing their own thing' with respect to personal productivity software. However, software relating to agency missions is expected to be written in languages that can, in principle, be supported by IS staff. This is reasonable. If I did all my work in APL and got hit by the proverbial truck, no one could carry on what I had been doing. (Actually, if I were in charge of my unit, I would start doing everything in J — mostly because it costs less than the major APLs — and then cross-train a couple of people to get the required redundancy.) At any rate, I began by writing 'prototype' systems in APL and using the results for some agency products, but eventually the law was laid down and I had to commit myself to rewriting the systems in something acceptable.

I'll skip over a nasty experience with Visual Basic. Eventually, it was decided that I should try writing in the Interactive Matrix Language (IML) component of the SAS database/statistics system; SAS is widely used, and about five people in the research group use it heavily. There is even a State SAS users' group that attracts 30 or so people to its monthly meetings. Clearly, SAS is not a trivial factor in the data analysis world.

I have just completed a system written in IML. My bosses should be happy because I am using SAS. I am not totally unhappy, because I didn't have to do it in Visual Basic. But the ironic thing is that, even though I 'did it in SAS', I'm the only one in the research section who understands IML. If that truck hits me, the agency will still be up the creek!

The point of this introduction is that there are acceptable array-language fallback options for APL programmers. As will be explained below, IML is a mere shadow of APL. But it can be learned quickly, and perhaps you can buy some time to convince management that J, say, is just like IML but a lot more powerful, etc., etc.

IML Overview

IML dates back to the late 1970s, not long after SAS Institute set up shop. In 1979, I briefly used an earlier version (known as Matrix Language, I believe) to write a simple population projection system. That was before I had heard of APL. In fact, I tried APL because it didn't need all those loop statements — just like Matrix Language! Actually, SAS documentation admits that IML is actually based on APL. This means that it is not hard for an APL programmer to do useful work in IML within a few days; the obstacle is that a certain amount of basic SAS needs to be learned first.

SAS itself allows users to access data from a variety of formats from a variety of computer platforms. My SAS-fan co-workers, who have to dig into pretty large databases, think this is great because the same code they write on their PC can be used to process data on a VAX or an IBM mainframe. Furthermore, SAS Institute takes great pains to keep core language features consistent and compatible over time. Non-core features such as windowing environments are kept up to speed with host systems; for example, the Windows 95 version has the Microsoft approval seal, just like Dyadic's APL.

IML is sort of an oddball in the SAS scheme. It can work with SAS data sets (the 'data set' — a grouping of fields and records — is the core of SAS), yet it can also deal with outside files directly and do other tasks that can be handled in SAS. In other words, it is semi-independent: a sort of APL, I suppose.

I don't pretend to be an IML expert. I've only worked with it a month or so and I had no experienced user or SAS Institute tech rep to consult. Therefore, some of the methods I used are likely to be sub-optimal or even frowned upon. My only other excuse is that I was under deadline pressure and had to get something working. Bottom line: take what I say with a little salt, though the gist of it is probably sound.

The first thing you have to do is get data into IML. This can be done at the keyboard, but if you want to do back-of-the-envelope calculating, fire up APL or J instead. For more serious work, you can either read files directly using IML or else read the files with SAS and then pass the data to IML. I used the latter

approach. Why? According to my IML manual (published in 1990), IML can read comma-delimited, but not tab-delimited files. SAS for sure can read either, and over the past few years I've been standardizing on tab-delimited files for communication with the non-APL/J world. (Specifically, I use the *.txt format that can be created by Excel. I often key in data in Excel and save the results both in spreadsheet and text format — the latter can easily be used by J with its `clipunfmt` function.)

The way my system works — it sounds a little screwy, but nothing else I tried seemed to do — is to have a SAS 'Main' program `$include` (this is using the SAS Macro Language — yet another tool) both SAS data read-in programs and IML scripts. The `$include` causes the called program to execute. The data programs cause files to be opened and SAS data sets to be created. The other programs are a little odd: they invoke IML, define an IML module (in IML jargon, a program is called a module) and then store that module in a temporary file before exiting IML. When all this has been done, the main program calls IML, loads all the modules from the temporary files, and then runs the main IML module which, in turn, can run other modules.

To move data from SAS data sets to IML takes only a few lines of code, and it takes even fewer to transform IML data objects into SAS data sets. You can get fancier, plucking or placing subsets on either side of the wall, but I haven't needed to do that yet. In any case, we are now in IML and we have our data to chew on. What is IML really like?

IML Up Close

The M in IML pretty much tells you the whole story: it relentlessly deals with matrices. As nearly as I can tell, a one-number IML variable is a one-row, one-column matrix. The corollary is that you are stuck in two dimensions. What might be a three- or four-dimension array in APL has to be flattened into two dimensions, and this means that the programmer has to do a lot of careful indexing work. Clearly, you are in good shape to the extent that your data objects have fixed coordinates — you aren't doing a lot of reshaping, even though reshaping is permitted.

Here is an example from my demographic system. In APL and J, I use three- or four-dimensional arrays deal with population data. The dimensions are sex, age, time, and (sometimes) areas: the system being discussed uses the first three. Therefore, the design decision is whether to create a single in-memory matrix for data caching or make two matrices. The two-matrix option would have an age-time matrix for males and another for females. The one-matrix alternative has

males by age as rows, followed by females by age as more rows, with time as columns. Because I tend to dislike variable clutter, I opted for the one-matrix alternative, even though it slightly complicates data indexing. I call the matrix `poply`.

Now let's say I'm in the midst of the process of moving a population from one year to the next. I want to extract the start-year population and do something with it, so I write

```
popt1=shape(poply[,yrcent],2,86);
```

SAS is case-insensitive, and names cannot be more than eight characters long. `shape` is an IML function that does about what the APL `rho` and the J `$` do. Note that all the action is specified within the parentheses. The first-position item indicates the data being used. Here, it is the column of `poply` called `yrcent`, which is a year counter; square brackets enclose matrix position information which are in row, column order with a comma separating the specifications. The next two items — the numbers 2 and 86 — are the row and column dimensions of the new matrix: in this case, two sexes and 86 ages (0, 1, 2, ..., 84, 85+). The final semicolon is IML's end-of-statement marker. So IML creates a 2-row, 86-column matrix and places the 172 numbers from a column of `poply` into it.

Now I need to age the population. This entails adding the two highest age groups and moving all the rest one notch older. My IML code does it this way:

```
agedpop=shape(0,2,86);
agedpop[,2:85]=popt1[,1:84];
agedpop[,86]=(popt1[+,{85 86}])';
```

Since I am creating a new variable, the first item in the `shape` expression is 0; I'm filling the matrix with zeros for starters. Then I drop in the data from the first 84 columns of `popt1`, but beginning at the second column of `agedpop`. As you might have guessed, the colon serves as an indexer — the number to the left is the start position, the number to the right is the end position. The final line adds the last two columns in `popt1` and places the result in the last column of `agedpop`. Note the difference in indexing: columns 85 and 86 are designated by curly brackets. The plus sign to the left of the comma indicates addition within rows. Also note that all this is surrounded by parentheses. This was done because I needed to rotate the result (now one row by two columns) into two rows by one column — the rotation operator is the front-quote mark to the right, just before the semicolon.

I ought to mention that IML does not use the right-to-left precedence scheme used in APL and J; it basically follows conventional precedence practice. This caused me some trouble because IML is just enough like APL to let me drop my guard. Worse, the calculations executed, but some results were odd. These had to be detected and the cause corrected.

Another irksome IML shortcoming is its string handling. Here is code I used to take a column of years, convert the numbers to characters, and place the letter 'Y' in front of each in order to create a set of labels. I hope I simply overlooked some documentation on the easy way to do this; at any rate, this illustrates what you might have to deal with:

```
use Script var{endyear};
read all var _num_ into years;
mxyrs=nrow(years);
tmp1=cshape('Y',mxyrs,1,1);
tmp2=char(years,4);
tmp2=cshape(tmp2,mxyrs,4,1);
tmp3=tmp1||tmp2;
yearcols=rowcat(tmp3);
```

The first two lines bring a column of data (from the forecast 'script' file) into IML from the SAS data set `Script`. Next, I use the IML function `nrow` to get the number of rows in the fetched column. `tmp1` is a 1-column matrix of 'Y's; note that, since I am using character data, I must use function `cshape` rather than `shape` to create the matrix. The following line uses `char` to transform the years into characters with a maximum of four. Now things get 'interesting'. I `cshape` the `tmp2` matrix into a matrix of `mxyrs` rows and four columns of one character width! (IML, unlike APL, does not seem to like treating character data as a big slab; you deal with columns that contain a specified number of characters each.) Next, I use the horizontal concatenation operator, `||`, to mate the 'Y's matrix to the years matrix; now our matrix has five columns. The final step collapses the five columns into a single column of five-character data using the function `rowcat`. I really hope all of the above is a huge mistake — on my part, not SAS's.

I think this gives you the flavour of IML. You manipulate matrices using either operators or keyword functions as appropriate. Compared to APL and J, IML seems crippled, but it is still a lot nicer than, say, Visual Basic for number handling. I will soon be writing a J version of the system to validate results and will be able to make comments on execution speed once both are running.

HTML Basics for APLers – Tables

by Adrian Smith (Causeway@compuserve.com)

Background to Part II

The code that forms much of this second article was begun on the plane on the way over to the Frankfurt GSE meeting in spring 1996. It was shown to a few more people at Orlando, and hastily tidied up, completed (and partially tested) as it became clear that several people really needed something like this. The workspace (for APL+Win) and equivalent namespace (for Dyalog 7.2/8) are available 'as is' on the Causeway web site (as www.causeway.co.uk/html.zip), and may be freely downloaded, improved and distributed. All the illustrative code is from the APL+Win version.

It assumes that you have already read (or have easy access to) the first article on this subject [1] which covers the basic structuring rules of HTML and runs through to the point where you can format a simple page of text with some subheadings and rules. If you want to revise the basics from another source, use [2] as your initial reference point, and if you want to extend what I have done to include some of the more modern tags (which will not work on old browsers) I suggest reading [3].

This article is almost entirely about tables, and all the examples should work in Netscape (from 1.22 upwards) and Explorer. As before, the message is to keep it simple, and to test your output on a wide variety of browsers (including some very old ones) before you go public. For an excellent example of a web site constructed almost entirely along these lines, point your browser (as they say) at www.devplan.com/qkw/renvill which currently has some 20 pages of mostly tabular data created directly from APL*PLUS component files.

Worked Example: Formatting a Simple Report

Let's assume you have an APL system as the access point for a large component file of timeseries data. Rather than typing directly into an APL session, or filling in an AP124-based form, your users now submit a query to you from their web browser over the corporate network. This specifies a product, a range of years, and requests tabulation by region. You want to return a stream of text which Netscape will form as a subheading, the product description as a wrapped

paragraph, and a table of sales data with two columns (units and value) for each year. Something like this in fact ...

Annual Statistics for Widget #005

File Edit View Go Favorites Help

Product Description

Widget #005

Some complete rhubarb about this wonderful product. Some complete rhubarb about this wonderful product. Some complete rhubarb about this wonderful product. Some complete rhubarb about this wonderful product. Some complete rhubarb about this wonderful product.

Year	Europe		USA		Rest of the World	
	Units	Value	Units	Value	Units	Value
1987	914	828	971	814	798	840
1988	449	643	694	816	469	194
1989	402	931	468	379	518	228
1990	900	986	137	459	690	280
1991	410	740	32	304	193	814
1992	275	718	409	786	723	578
1993	633	799	1	628	347	472

Notes

More useless information to read here. More useless information to read here. More useless

Note the product id in the window title, and repeated below the subheading. The table has various column alignments, and shows that it is possible to make tree-structured headings by spanning cells across more than one row or column. Apart from this, the presentation is entirely decided by the browser, which in this case is Netscape 1.22 - as good a generic model as you can find.

Here is the APL code which generated it:

```

v r+vector;mat;sink
[1]  a HTML example for Vector 14.1
[2]  mat+(1986+17),?7 6p1000
[3]  a Page title and product info ...

```

```

[4]   'Annual Statistics for Widget #005'htmUse''
[5]   'Subhead'htmPlace 'Product Description' 'Widget #005'
[6]   htmFlow €5p<'Some complete rhubarb about this wonderful product. '
[7]   ª Tables are the real challenge ...
[8]   sink+htmlTitles 'Year' 'Europe' 'Units' 'Value' 'USA' 'Units' 'Value'
      'Rest of the World' 'Units' 'Value'
[9]   sink+htmlTDepth 0,9p 0 1 1
[10]  sink+htmlTAlign (c'Cent'),9p'Cent' 'right' 'right'
[11]  sink+htmlAlign 1 6/'centre' 'right'
[12]
[13]  ª Use pixel widths for the columns
[14]  sink+htmlCellWidths 72,9p96
[15]  htmlList mat
[16]
[17]  ª Now a rule off and that's all folks
[18]  'Subhead'htmPlace'Notes'
[19]  htmFlow €9p<'More useless information to read here. '
[20]  htmRule 2
[21]  htmPlaceª Widgets International Inc' 'April 1996'
[22]
[23]  PG+htmlClose
[24]  r+''vector.htm'' htmPut PG ª to see it in Netscape'
[25]

```

7

Making Tables in HTML

Tables are quite a late runner in the HTML world, so Netscape and Microsoft have been falling over each other to add bells and whistles to the format. The result is that you have a surprisingly large amount of fine control over the output, even down to giving pixel widths for columns:

```

sink+htmlTitles 'Year' 'Europe' 'Units' 'Value' 'USA' 'Units' 'Value' 'Rest
of the World' 'Units' 'Value'
sink+htmlTDepth 0,9p 0 1 1
sink+htmlTAlign (c'Cent'),9p'Cent' 'right' 'right'
sink+htmlAlign 1 6/'centre' 'right'
ª Use pixel widths for the columns
sink+htmlCellWidths 72,9p96
htmlList mat

```

The column-title strategy is a straight lift from *NewLeaf* [4], and uses the same approach as Dyalog APL in setting up hierarchical grid titles. It may help to turn the table on its side and view the titles 'File Manager' style:

```

Year
Europe
|
+-----+Units
      +Value
USA
|
+-----+Units      etc.

```

If you look at the *htmltInit* function, you will see that this sets a collection of table properties to sensible defaults. Each of the remaining table 'methods' called here makes some simple checks on its argument, and resets the property, returning the previous value:

```

v htmltInit
[1] a Set all values to saved defaults for HTML table
[2] htmlt$alignment+,0
[3] htmlt$alignment+,1
[4] htmlt$gridwalls+0
[5] htmlt$gridlines+1
[6] htmlt$coltitles+''
v
v r+htmltTitles vtv
[1] a Set column headings for table
[2] a Argument may give just a vtv of title text or
[3] a a nested vector of (multi-line title)( etc)
[4] a typically ('Name' 'Age' ('Current' 'Salary'))
[5] a Resets title-depth to 'flat'
[6] r+htmlt$coltitles
[7] +(' 'A.,,vtv)+Null
[8] +(0epvtv)+Encl ◊ +(1<|=vtv)+Encl ◊ vtv+c,vtv
[9] Encl:htmlt$coltitles+vtv ◊ htmlt$depth+(pvtv)p0 ◊ htmlt$align+(pvtv)p1 ◊ +0
[10] a Default case - set alignment to 'Centre' for all columns
[11] Null:htmlt$coltitles+'' ◊ htmlt$depth+0 ◊ htmlt$align+0
v
v r+htmltDepth nv
[1] a Set title depth as tree level (from 0)
[2] a Ensure this matches the number of title text elements!
[3] r+htmlt$depth ◊ nv+(phtmlt$coltitles)pnv
[4] htmlt$depth+nv
v
v r+htmltAlign str
[1] a Set/query title alignment in table (vector result)
[2] r+('Left' 'Centre' 'Right')[1+htmlt$align] ◊ +(0epstr)+0
[3] +(1<|=str)+Encl ◊ str+c,str
[4] a Ensure the alignments match the headings
[5] Encl:str+,0 1 2 0 1 2 0[(='LCRlcr');]"+str
[6] htmlt$align+(phtmlt$coltitles)pstr
v

```

```

v r+htmlAlign str
[1]  a Set/query cell alignment in table (vector result)
[2]  a Decimal alignment not yet available in html
[3]  r+('Left' 'Centre' 'Right' 'Decimal')[1+htmlAlign]
[4]  +(0epstr)+0
[5]
[6]  +(1<)*str+LC+1 o str+c, str
[7]  htmlAlign+,(0 1 2 3 0 1 2 3 0)[(='LCRDlcrd')i]+str]
v

v r+htmlCellWidths cells
[1]  a Define cells within table. Arg gives reqd widths in points.
[2]  a User can give 1 for any cell width which divides spare
[3]  a frame width equally after taking off the others.
[4]  r+htmlGridwalls o htmlGridwalls+,cells
v

```

Again, some *NewLeaf* properties are left in, for example decimal alignment, in the hope that the HTML 3.0 proposals may eventually catch up with reality and get implemented. Now that we have defined all the required properties, it is time to explore the main workhorse which formats our data and adds all those table tags for us ...

```

mat+(1986+17),77 6p1000
htmlList mat
html_PG
<HTML><HEAD>
<Title>Annual Statistics for Widget #005</title>
</HEAD>
<BODY text="#004040" bgcolor="#FFFFFF">
<h2>Product Description<br>Widget #005</h2>

<TABLE border cellpadding=2>
<TH rowspan=2 align=center width=72>Year</TH>
<TH colspan=2 align=center>Europe</TH>
<TH colspan=2 align=center>USA</TH>
<TH colspan=2 align=center>Rest of the World</TH>
<TR>
<TH align=right width=96>Units</TH>
<TH align=right width=96>Value</TH>
<TH align=right width=96>Units</TH>
<TH align=right width=96>Value</TH>
<TH align=right width=96>Units</TH>
<TH align=right width=96>Value</TH>
<TR>
<TD align=center valign=middl>1987</TD>
<TD align=right valign=middl>19</TD>
.... and so on

```

... as you can see, this suddenly got a great deal harder. My feeling on basic HTML is that Notepad is one of the best editing tools you can find, and if you get serious then Kenn Nesbitt's *WebEdit* is fine. Both these are simple text editors -

you just type in the tags as you go along. However as soon as you hit tables, you need help, and what better tool to keep track of all that tedious tagging than APL!

Here is the code which did it ...

```

v htmList mat
[1]  A Dummy as Netscape wraps regardless
[2]  htmSpread mat
v

v htmSpread mat;blk;walls;aln;dpl;cw;cc;cw;lcr;txt;htm;valn;tmb
[1]  A HTML table format of nested matrix. Simple vectors just get enclosed
[2]  →((1<|mat)×2÷ppmat)+Shaped ⋄ mat+c,mat
[3]  Shaped:mat+(~2÷1 1,pmat)pmat
[4]  htm_grid←htmGridwalls
[5]  A Worry about frame width later (probably VGA say 450)
[6]  cw+(~1÷pmat)+htm_grid
[7]  A Repeat alignments for each column (no decimal support here)
[8]  aln+(~1÷pmat)phmtaalignment ⋄ aln÷2⌊|aln
[9]  valn+(~1÷pmat)phmtavalignment
[10] htm+ '<P><TABLE>'
[11] :if 0≠htmGridlines ⋄ htm←htm,' border cellpadding=4' ⋄ :end
[12] :if 0≠htmGridwalls ⋄ htm←htm,' width=100%' ⋄ :end
[13] htm_cat htm,'>'
[14] A Run headings for chosen columns
[15] htm_titlegrid cw
[16]
[17] A ===== ROWS =====
[18] NextRow:→(0÷pmat)+Exit ⋄ blk←mat[1;]
[19] A ===== COLUMNS =====
[20] cc÷1 ⋄ →(cc÷~1÷pmat)+EndCols
[21] NextCol:lcr÷(1+cc÷aln)÷' ' 'center' 'right'
[22] tmb÷(1+cc÷valn)÷'top' ' 'bottom'
[23] :If 1<|cc÷blk
[24]   txt+~4÷+,/(cc÷blk),'<'<BR>'
[25]   :Else
[26]     txt+~,cc÷blk
[27]   :End
[28] A Leave out attributes where defaults taken
[29] txt+ '<TD',((0÷plcr)/' align=',lcr),
      ((0÷ptmb)/' valign=',tmb),'>',txt,'</TD>'
[30] htm_cat txt
[31] cc÷cc+1 ⋄ →(cc÷~1÷pmat)+NextCol
[32] EndCols:htm_cat '<TR>'
[33]
[34] mat÷1 0÷mat ⋄ →NextRow
[35]
[36] A Closing paragraph tag required for Explorer, harmless in Netscape!
[37] Exit:htm_cat '</TABLE></P>' ' '
v

```

Note that we can take out 'align' where it is 'left' and 'valign' where it is 'middle' – every byte counts when you are on the end of a slow modem line. Hopefully the code is simple enough to read with no further explanation, so onward to the titles:

```

v htm_titlegrid cw;grid;td;grid;ch;txt;cell;lab;lw;rs;cs;td;ln;ta;htm;seq;tr;fmt
[1]  * HTML Table headers if provided
[2]  * Will take a tree specification to make multi-layered headings
[3]  * (0<phmt&acoltitles)+0
[4]
[5]  * Assume everything is at level zero if atdepth not given
[6]  txt+htmtacoltitles * td+htmtatdepth * ta+htmtatalign * (0v.*td)+Deep
[7]  * Reshape titles to match cells in this grid (simple case)
[8]  txt+((pcw)ptxt * td-(ptxt)p0 * ta-(ptxt)pta * Parse
[9]
[10] * Count leaf nodes, and trim/extend to match <cw>
[11] Deep:ln-1+(0,td)>td,0 * +((pcw)=+/ln)+Parse
[12] txt+txt,99p<+ptxt * td+td,99p-1+td * ta+ta,99p0 * ln+1+(0,td)>td,0
[13] ln-ln^(+/ln)s<pcw * txt+ln/txtd * td+ln/td * ta+ln/ta
[14]
[15] Parse:grid+htm_parsesetree td
[16] * Sort out -ve cell widths (must be % values)
[17] :if 0v.>cw
[18]   ((cw>0)/cw)+1 * cw+|cw
[19]   cw+(1v"cw*100:/cw),"%"
[20] :else
[21]   cw+1"cw
[22] :end
[23]
[24] * Sort cells into Netscape-preferred order (row-col)
[25] seq+&grid[;2]
[26] * Detect row ends
[27] tr+1+(grid[seq;2],99)>0,grid[seq;2]
[28]
[29] * Now fit the captions into the correct cells (alignment req'd)
[30] :For cell :in seq
[31]   htm+<TH>
[32]   rs+</grid[cell;4 2] * :if rs>1 * htm+htm,' rowspan=',(vrs) * :end
[33]   cs+</grid[cell;3 1] * :if cs>1 * htm+htm,' colspan=',(vcs) * :end
[34]   htm+htm,' align=',(1+cell>ta)>'left' 'center' 'right'
[35] * Leaf cells can set the column width up
[36] :if cell>ln
[37]   htm+htm,' width=',cell>ln\cw
[38] :end
[39] :If 1<|<cell>txt
[40]   fmt+~"4+/,/(cell>txt),"c'<BR>'
[41] :Else
[42]   fmt+~vcell>txt
[43] :End
[44] htm_cat htm,'>',fmt,'</TH>'
[45] * If cell spans to last column we need a new title row!
[46] :if tr[seq;cell]
[47]   htm_cat '<TR>'
[48] :end
[49] :End
v

```

All I can say about the logic of this is that it seems to work! I have tried it on some very complex heading structures and Netscape always makes sense of the output.

The only little detail which still needs explaining is the conversion from the hierarchy of headings to the 'rowspan' and 'colspan' specification required by Netscape (try doing this by hand some day!). This was one of those apparently recursive problems (like the Hanoi Tower puzzle) where you are sure that there has to be a 'straight-through' boolean solution if only you can see it.

When I find myself faced by such a problem, I do one of two things: I take the dog for a very long walk; or I ask Duncan. Unfortunately we do not currently have a dog, and taking the wife for a long walk is nowhere near as effective, so I asked Duncan. He went very quiet for about 90 minutes and came up with this:

```

v bxs+htm_parsetree tree;depth;leaf;top;left;bottom;lm;slm:right;[]0
[1]  a Take a depth vector and return xxy matrix to draw it as headings
[2]  a Test with: htm_parsetree 0 1 2 3 2 3 3 4 2 2 3 1 2 3 2 0 1 2 2 1 2 3
[3]  a NB This function contains no user-servicable components
[4]  tree+tree,[]0+0
[5]  depth+1+[]/tree
[6]  leaf+tree+1+tree
[7]  top+tree
[8]  left+~1++~1+leaf
[9]  bottom+1+top
[10] (leaf/bottom)+depth
[11] lm+(-tree)*o(ptree)/(depth,1)*depth+1
[12] slm+v\lm
[13] right+1+(,slm)/,(pslm)*o(,slm)\(1+slm)/,(pslm)*left
[14] bxs+[]0 ~1+>left top right bottom

```

```

htm_parsetree 0 0 1 1 0 1 1

```

0 0 1 2	0	1	2	3	4	5
1 0 3 1	0	+	+	+	+	+
1 1 2 2						
2 1 3 2	1	+	+	+	+	+
3 0 5 1						
3 1 4 2	2	+	+	+	+	+
4 1 5 2						

...I added the comments. The result gives the (x,y) co-ordinates (on a zero-origin chessboard) of the corners of each heading cell. Now we can find the rowspan and colspan trivially, simply by subtracting adjacent elements (lines[32]-[33] of *htm_titlegrid*).

Remaining Odds and Ends

Here are all the other functions which I haven't found time to illustrate:

```

v alt htmBitmap pic
[1]  a Insert picture (.GIF) in line
[2]  :if 0=[nc 'alt' o alt+pic o :end
[3]  pic+pic,(~'. '€ pic)/'.GIF'
[4]  htm_cat ''
v

v alt htmBitmapLeft pic
[1]  a Insert picture (.GIF) at left margin
[2]  :if 0=[nc 'alt' o alt+pic o :end
[3]  pic+pic,(~'. '€ pic)/'.GIF'
[4]  htm_cat ''
v

v htmBump pts
[1]  a Add vertical space. Do what we can here!
[2]  htm_cat (4*[pts÷12])p'<br>'
v

v sty htmFlow txt
[1]  a Simple text flow, taking account of style
[2]  htmUseDflt o htmStyle'Body'
[3]  →(0=[nc'sty'])+Dflt
[4]  htmStyle sty
[5]  a Ensure correct enclosure
[6]  Dflt:→(2=|txt)+Encl o txt+c,txt
[7]  Encl:txt+(,/'<','htmltag','>'),'txt','/(</'>),'(¢htmltag),'>'
[8]  htm_cat txt
v

v htmRule wt
[1]  a Rule across (for parameters see Netscape extensions)
[2]  a The default is a 2-pixel 'shaded' rule.
[3]  htm_cat '<HR noshade size=' ,wt, '>'
v

```

That is as far as I have got to date. Numbered and bulleted lists would seem to be an obvious next step, then we should look at simple input forms. You can include input elements (such as text fields and drop lists) within tables, so it is interesting to contemplate an API24 look-alike which might add a few years to the life-expectancy of a number of old mainframe systems, which would suddenly come out smelling strongly of Netscape.

References

- [1] *HTML Basics for APLers*, Adrian Smith, Vector 13.4 page 84
- [2] *The HTML Sourcebook*, Ian S. Graham, John Wiley 1995
- [3] *Official HTML Publishing for Netscape*, Harris & Kidder, Netscape Press 1996
- [4] *NewLeaf User's Manual*, Causeway Graphical Systems Ltd, 1996.



Renaissance Data Systems
P. O. Box 421 - V
Georgetown, CT 06829
+1 (203) 270 9729

**Books on APL and J and other curiosities
of merit!**

Renaissance Data Systems announces a change in its mailing address. Please note that the telephone number remains the same.

If you would like a copy of our latest **catalog**, please send us a self-addressed legal sized envelope with one first class stamp (if in the U.S.).

Included are such titles as: **APL is EASY**, **APL - An Interactive Approach**, **APL2 at a Glance**, **APL - the Language and its Actuarial Applications**, **APL as a Tool of Thought** proceedings, **I-APL** publications and software, **Boolean Functions and Techniques**, **The FinnAPL Idiom List**, **The Toronto APL Toolkit**, **Mathematical Experiments on the Computer**, **Probability in APL**, **APL - Stat: Do it yourself guide to computational statistics**, **A Source Book in APL** - Approximately 80 titles in all!

We also carry J publications, including **Programming in J**, **An Implementation in J (structure and source code)**, **Arithmetic**, and **Calculus**.

Shareware and commercial APL interpreters and J2 interpreters are available as well.

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Namespaces in Causeway	Duncan Pearson	94
Object Oriented Programming with APL+Win	Eric Lescasse	106
APL and J (5): Operators — Inner Product, Each, Commute, Compose	Chris Burke	129
At Play with J: Oh, No, Not Eigenvalues Again!	Gene McDonnell	135
The API Problem as a Particular Recurrence Problem	Andrei Buzin	140

Namespaces in Causeway

by Duncan Pearson (causeway@compuserve.com)

Causeway is an application development environment for Dyalog APL that has been described in other articles in Vector and demonstrated at various conferences. The latest version which is in beta test is wholly integrated with the Dyalog APL namespace extensions, taking advantage of them in its own internal organisation and being sensibly tolerant of namespace changes within application code developed in Causeway. This article introduces the concepts upon which Causeway is based and describes how it is used in application development. It then describes the rôle of namespaces in the internal organisation of Causeway and the way in which the use of different namespaces in application code affects the behaviour of Causeway.

Overview of Causeway

Causeway is a tool for generating Windows interfaces for APL applications. The basic unit of interface is a *form*, represented by a window on the screen. It contains *controls* with which the user may interact. Controls can do two main things, display application data such as the content of edit fields, and allow the user to trigger actions in the application such as on the pressing of buttons. Of the controls which display data some will allow the user to change the data.

Data

Each control may have more than one aspect of its visual appearance. For a list box one aspect might be the names in the list, another might be the selection, those items that have been highlighted. These aspects are called *properties*. The principal concept behind Causeway is that a property of a control may be linked to data in the application, typically a variable. When the user changes the property of the control then the variable is updated. When the application changes the variable the control is updated to reflect the new data. Both aspects of this interchange occur automatically, managed by Causeway.

The data in the application need not be just a variable. For properties of a control that the user cannot change, the data can be defined by any APL expression that returns an array of the correct type for the property. If the user can change the property then that change must be passed to the data. The way that Causeway does this is to surround the expression supplying the data with parentheses and

use the resulting phrase to the left of an assignment. Thus any variable, list of variables, indexed variable or selective specification of a variable can be used.

Each expression providing data for a property will depend on a number of variables. The developer needs to specify these variables as they cannot be derived reliably from the expression, for instance when the indices of an indexed variable are provided by a function that calls another function etc.

Events

In general the developer can define for a control a set of APL expressions to execute on each event that can occur to a control. A typical event for a button might be being pressed, for a form being resized, for an edit field a key press.

If the APL expression changes application data then it is a good idea for the developer to tell Causeway that this has happened. This is the only way that Causeway can know that the data has changed. It needs to know so that all controls with properties dependent upon those variables can be updated. When specifying the expression to run on an event there is space for putting just this information.

Controls as Classes

Each control on a form is an example or "instance" of a particular "class" of control. It has appearance, behaviour and properties that are defined by the class of which it is an instance. Dyalog APL itself has control classes such as EDIT and BUTTON. In Causeway the need to link controls to data and to extend or modify the functionality of the Dyalog classes has encouraged us to define our own classes. Each control on a Causeway form is an instance of one of these Causeway classes. In the version of Causeway distributed with Dyalog APL the definitions of the classes together with all the functions managing the behaviour were held in a single global variable called `Δclasstab`. In the new version each class is defined by the functions and variables in a namespace of the name of the class.

Form Definitions

The definition of a form and its controls in Causeway is held in a five-column APL matrix. The first row defines the form and each subsequent row defines one control. The first element of each row is always the control's class. The next three are caption, position and size and the final element is itself a two-column matrix of property-value pairs.

The form is created by running a Causeway function, passing in the definition variable. There are two functions that do this. *Call* creates the form, waits while the user interacts with it and returns when the user closes it. *Make* creates the form and returns immediately leaving the form on the screen for the user to interact with at his leisure.

In the previous version of Causeway the functionality of Make and Call were provided by Gui_make and Gui_call. The Gui_ prefix is no longer necessary as the functions are in a separate namespace and the initial letter is capitalised to indicate that they are functions. Similarly Gui_reject is now Reject and Gui_return is Return. The conversion routine for form definitions makes these and other replacements in property and action expressions. A routine is provided to make the replacements in application functions.

Namespace Dependence

The evaluation of application expressions and resolution of application variable names is central to the work that Causeway does. Traditionally the functions *Gui_make* and *Gui_call* lived in #, the root namespace, and thus all names were resolved relative to # and all expressions were evaluated there also.

If the Causeway functions were placed in a different namespace, e.g. #.Gui, then names and expressions would be resolved and evaluated in that namespace. If this approach was adopted then all variable names, in dependence lists for properties or in lists of changed variables for actions, would need to be fully qualified (beginning with #.). Furthermore all names in all expressions, whether they be for properties or actions, would need also to be fully qualified.

It seemed to us when designing the new version that the most desirable namespace in which to resolve names and evaluate expressions would be the namespace from which *Make* or *Call* was called. Thus *Make* (which is a sub-function of *Call*) picks the first namespace in *NSI* which differs from its own and marks that as being the namespace for that instance of the form. All names are taken as relative to that namespace and all expressions are evaluated there.

The way in which this is achieved is explained below in **Name Resolution** and in **Expression Evaluation**.

Expression Evaluation

In order to execute the developer's code in the namespace from which the form was called, Causeway collects all the expressions, whether they be property expressions or event actions, and fixes them in a function the first line of which changes into the relevant namespace. The following line branches to the line to be executed. If there is more than one expression to be evaluated it does them in the same call, visiting each required line in turn. The list of lines to visit is held in a local variable in the calling function, for reasons that will become apparent. After each line has been executed the list is beheaded and branched to. In the simple case 0 is appended to the list before the first branch, so that the final branch exits the function. This is complicated if the form has local variables, about which more later, but the simple case is illustrated below. The function executes the expressions for form 1 of which there are two, on lines 15 and 19 below.

```

V Exec1;clicks
[1]  #.CPro._dummy clicks+1, #.CPro.loc_val[[]IO+0]
[2]  []CS #.CPro._lcl>#.CPro.lcl_spc
[3]  :If 1<p#.CPro._fns
[4]  1[]IO>#.CPro._fns+1+#.CPro._fns
[5]  :EndIf
[6]  ->#.CPro._next+#.CPro._next, 0p[]LC+1
[7]  #.CPro.loc_val[[]IO+0]+1+1 clicks
[8]  :Return
[9]  A Set #.CPro._obj to the control name and refresh controls
[10] #.CPro._ref+0 2p0 #.CPro._obj+([]IO+1)>#.CPro.ctl_name
[11] ->#.CPro._next+1+#.CPro._next
[12] :If 0<1+p#.CPro._ref #.CPro._ref #.Class.FM.Refresh #.CPro._obj #.CPro._obj :EndIf
[13] ->#.CPro._next+1+#.CPro._next
[14] A Execute action expressions
[15] :If ~#.CPro._rej # clicks+clicks+1 #.CPro.Touch 1 #.CPro._obj :EndIf
[16] ->#.CPro._next+1+#.CPro._next
[17] A Evaluate property expressions
[18] #.CPro._rej+0
[19] :If ~#.CPro._rej #.CPro._obj :AndIf
[20] #.CPro._forcev(([]IO+0)>#.CPro.prp_val)1#.CPro._val+(vclicks), ' clicks so far' #.CPro._val
[21] ( []IO+0)>#.CPro.prp_val)+#.CPro._val #.CPro._ref, '+'Data'#.CPro._val #.CPro._val :EndIf
[22] ->#.CPro._next+1+#.CPro._next
[23] A Read properties and execute property assignment expressions

```

The first five lines and also the seventh implement variables local to forms as described below. Lines 6, 11, 13, 16 and 20 implement the branching mechanism and the remaining lines do various bits of housekeeping. The many references to `#.CPro.` in the body of the function are to avoid name pollution in the execution namespace.

Name Pollution

One of the problems with all application development tools is name pollution, a problem that namespaces are supposed to solve. You wish to import some functions of your own but the names clash with some functions in your development tool. The method of placing all the form's application code in special functions that execute in the application's space removes all the Causeway management code and working variables from the application namespace. However when a Dyalog function changes space all the local names are taken into the new space. Local names include not only local variables but also labels, arguments and the result. Therefore the execution function must neither have locals, arguments nor return a result. Thus the list of lines to execute must be global to the function and in the Causeway namespace. In the assembly of the function the ∇ character is used in place of a direct path to the Causeway namespace. In the fixing of the function all ∇ s that occur in the executable code are replaced by the absolute path of the Causeway namespace. The function above started life looking marginally more readable as the vector of text vectors:

```
Exec1;clicks
 $\nabla$ _dummy clicks +1, $\nabla$ loc_val[[]io+0]
  []cs  $\nabla$ _cli= $\nabla$ cli_spc
  :if 1< $\nabla$ _fns
    *[]io= $\nabla$ _fns+1+ $\nabla$ _fns
  :endif
  + $\nabla$ _next+ $\nabla$ _next,9 $\nabla$ _lc+1
 $\nabla$ loc_val[[]io+0]+1+1 clicks
: return
  A Set  $\alpha$  to the control name and refresh controls
 $\nabla$ _ref+0 2p0  $\alpha$ +([]io+1)> $\nabla$ cti_name
+ $\nabla$ _next+1+ $\nabla$ _next
:if 0<1+ $\nabla$ _ref  $\alpha$   $\nabla$ _ref #.Class.FM.Refresh  $\alpha$   $\alpha$  :endif
+ $\nabla$ _next+1+ $\nabla$ _next
  A Execute action expressions
:if ~ $\nabla$ _rej  $\alpha$  clicks+clicks+1  $\alpha$   $\nabla$ Touch 1  $\alpha$  :endif
+ $\nabla$ _next+1+ $\nabla$ _next
  A Evaluate property expressions
 $\nabla$ _ref+0
:if ~ $\nabla$ _rej  $\alpha$  :andif  $\nabla$ _forcev(([]io+0)> $\nabla$ prp_val) $\nabla$ _val+(clicks),' clicks so far'  $\alpha$  (([]io+0)> $\nabla$ prp_val)+ $\nabla$ _val  $\nabla$ _ref,+'Data'  $\nabla$ _val  $\alpha$  :endif
+ $\nabla$ _next+1+ $\nabla$ _next
  A Read properties and execute property assignment expressions
```

This substitution has the side effect that any ∇ in the application expressions will automatically refer to Causeway. Thus in a form definition a reference to $\#$.CPro.Make could be replaced by ∇ Make which would be converted on form creation to $\#$.CPro.Make. If at a later stage in the development of the application the Causeway namespace was moved or renamed (e.g. to

`#.Causeway`) then code in form definitions need not be changed as the `vs` would then automatically be converted to `#.Causeway..`

Local Variables

A Causeway form can contain local variables. Each instance of the form will have its own data associated with the local names so that if a form definition is used more than once to create separate forms, each can have its own copy of certain variables invisible to the other instances. The names of the variables are given as the "Locals" property of the form and any references to those names in any data expressions or event actions of the form or its controls will resolve to data specific to the form. The values of the variables are actually held in a table in the Causeway namespace and on execution of the application code they are assigned to local names in the execution function. At the end of the execution then the values of all local variables must be placed back in the appropriate places in the value table. This is done on the seventh line of the function, after which the function ends. Thus on the sixth line `7` is added to the end of the list of lines to execute. Effectively the function is a header followed by a set of subroutines and `v_next` is the list of subroutines to execute.

If a Causeway form is created from an action on another form then that form is its parent. If the parent form has any locals then these are visible to all actions on the child form, except of course those that the child form has shadowed with locals of its own. This is very similar to the way in which a called function inherits the locals of its calling function. Causeway needs to match this behaviour. The easiest way to do this is to have a function in the stack for each parent form, each function localising, setting and finally saving the locals for that form. This is what lines 3 to 5 of `Exec1` are doing.

For each form we hold its context which is the list of those forms in its list of parents that contain local variables. When we need to execute an action on a form we execute in turn each line of the function for each form in the context. They each set up their local variables, execute the next function in the list and then save the values of their local variables. Because we are implementing form locals with functions then everything that is true of function locals, with respect to changes of space, is also true of form locals. Thus, if in an action on a form I change space, then my form locals are still available in the new space.

The function that executes the context functions for a form is *ExecLines* which takes a set of lines to execute and a form index.

```

v _lcl ExecLines _next;stack;_fns;_dummy
[1]  A Execute the lines <_next> in the locale <_lcl>
[2]  A <_dummy> is used in the assignment of locals in the ExecNN functions
[3]  A <_stack> is the list of environment functions currently on the stack
[4]
[5]  A Save the value of <_stack>
[6]    stack+_stack
[7]
[8]  A Create the environment function list
[9]    _fns+(<[CS''],'.Fns.Exec'),*'"(_lcl=lcl_ctx)v_lcl
[10]
[11] A Remove any already running
[12]   _fns+<_stack
[13]
[14] A Add the number of new functions to <_stack>
[15]   _stack,+_fns
[16]
[17] A Execute the new functions
[18]   ++_fns
[19]
[20] A Restore <_stack> to its previous value
[21]   _stack+stack
v

```

Name Resolution

When a name occurs in a list of dependence for a property or of variables changed by an action it must be matched against the dependence lists for each property, not only on that form but on any form. The name in the list is both relative to the namespace in which the form runs and also the localisation affecting the form. In order to avoid confusion all dependence between variables and properties is held in Causeway as a correspondence between indices into a properties table and indices into a name list. Each fully qualified global name appears in this list once and each local variable appears once per instance fully qualified by the namespace of the form of which it is the local.

When a name is given as being relative to a form it goes through a two-stage process. First it is qualified by the function *Qualify*.

```

v r+space Qualify names;dots;spaces
[1]  A Fully qualify <names> given relative to the namespace <space>
[2]    :If ','names
[3]      dots+1*','*names
[4]      spaces+(-1+dots)*names
[5]      names+(-dots)*names
[6]      spaces+space*([CS''],'.spaces*"'[CS'']

```

```

[7]      r+spaces,"'.'",names
[8]      :Else
[9]      r+(<space>'[]cs''),"'.'",names
[10]     :EndIf

```

v

If the name contains no dots then the space name is prefixed to it. If it has a dot in it then it has a path part and a name part. These are separated and the phrase `[]CS ''` is run in the space named in the path part. `[]CS ''` will always return the shortest full path to the space in which it is run. So even if the same variable is referred to using two different paths they will resolve to the same entry.

Next the qualified name is compared to local names for the form and then to known global names. Any new global names are added to the list of known global names.

```

v vids+locale Resolve names;unames;locals;lvids;linx;global;ginx;new
[1]  A Resolve <names> relative <locale> into name indices
[2]
[3]  A Ensure names are fully qualified
[4]  names+2+"(locale>lcl_spc)Qualify names
[5]
[6]  A Work on the unique set
[7]  unames+unames
[8]
[9]  A Set up the result
[10] vids+(pnames)p^1
[11]
[12] A Retrieve the set of name indices for the
[13] A locals of the context of <locale>
[14] locals+elcl_locs[locale>lcl_ctr]
[15] lvids+loc_vid[locals]
[16]
[17] A Look for <names> in the list of local names
[18] linx+vid_name[lvids]\unames
[19]
[20] A If not local, a name is global
[21] global+linx+plocals
[22]
[23] A Place indices for local names into the result
[24] ((-global)/vids)+lvids[(-global)/linx]
[25]
[26] A Look for the remaining names in the known global names
[27] ginx+vid_name[glob_vid]\global/unames
[28]
[29] A Any not found are new
[30] new+ginx+pglob_vid
[31]
[32] A Place indices of found names in the result
[33] ((-new)/global/vids)+glob_vid[(-new)/ginx]
[34]
[35] A If there are any new global names then add them to

```

```

[36]  A the name list and the global list, placing the new
[37]  A indices in the result.
[38]  :If v/new
[39]      (new/global/vids)+glb_vid,+AddVids new/global/unames
[40]  :EndIf
[41]  vids+vids[unames\names]

```

▽

The resulting index distinctly identifies either a global name in a namespace or a local variable on an instance of a form.

Control Library

One of the main criticisms of the previous version from the point of view of advanced users was that it was difficult to add a new control class or to change the behaviour of an existing one. The class definitions were held in a deeply nested matrix in the global environment. The functions that implemented the behaviour of a control were held within this matrix as `□ORs` and it needed a set of special maintenance tools to edit these functions. The structure of the matrix was not readily accessible and this reflected the fact that nearly all class development was done by either Adrian or myself, both of whom have a deep understanding of the internal workings of Causeway.

Another disadvantage of the single control definition matrix was that if a new version of a control class was sent out inclusion of it would override any changes to the class that had been made by the developer. These disadvantages are catalogued in admirable detail in Jon Sandles' article (in Vector 12.3 p.51) and it is mostly due to his experience that we have avoided many of the pitfalls in the new version.

In the new Causeway a control class is implemented by a namespace of the same name which contains a specification, which also documents the class, some required variables and functions and any other functions or variables used by the class. The required variables and functions and the reasons for the requirement are given later. The individual control class namespaces are held in a parent namespace that can contain common code for dealing with e.g. bitmaps and icons or `□NA` calls.

In my test version my parent namespace is called `#.Class` and the control classes are `#.Class.AC`, `#.Class.TM` etc. However the parent namespace could be called anything and indeed there could be more than one class parent namespace. The variable `Δclasspath` in the Causeway namespace is a list of namespaces in which can be found control classes. The initialisation of Causeway traverses this path and collects a list of all the classes found therein. If a class of

the same name appears in two places on the path then Causeway will use the first and ignore all subsequent occurrences. Thus the class developer can have two or more versions of the same class and determine which is used by the order of the search path.

The main advantage of the namespace approach is in the accessibility of the class to the developer. The Dyalog namespace explorer can be used to browse the class tree and to show and edit the components of a particular class. The session space can also be set to the class. If while an application is running an error occurs in a class function then the session will suspend in the class and the error can be fixed. The application can be restarted and the bug will be fixed not only for the instance in which the error had occurred but also in any other active instances of the same class. This makes for much more dynamic application development, something to which APL has made us accustomed.

Class requirements

A control class namespace must contain a number of variables and functions without which Causeway will fail. These are for the management of instances of the class. Causeway must manage the instances but from the simplest (the label) to the most complex (the text grid) it knows nothing of the internal workings. It needs to be able to create and destroy instances, to pass data for the control to display, to know when the control has changed and to read new data from the control. To do each of these tasks there must be a function in the class with a set name that the Causeway engine can call to perform these actions.

When a new form is run from a definition variable each row defines a control (instance of a class). For each row Causeway runs the *Create* function in the class passing it the remainder of the row which contains the name of the parent, the position and size and caption and a table of other properties that are needed for the creation (e.g. colour, hint, caption position). The *Create* function will create the instance, normally but not always as a child of the parent, and return the actual name of the new instance. In all subsequent dealings with the class regarding the control, that name will be given for identification of the specific instance. As the data which the control is displaying changes, Causeway will run the *Refresh* function in the class, passing the name of the instance to be refreshed and the new data. The task of presenting the data to the user in the instance is given to the class.

The class *specification*, a variable in the class namespace, contains a list of the events that will be triggered if the application user changed any aspect of the data of the instance (property). If one of these events occurs then Causeway is alerted

and it calls the *Read* function in the class with the name of the property that has changed and the message reporting the event, which contains the name of the instance. The *Read* function returns an APL array corresponding to the changed property, which Causeway assigns to the expression associated with the property in the form definition. Thus the class knows how to represent data in and retrieve data from the control, but not the application expression giving that data nor the namespace in which it is run. Conversely the Causeway engine knows how and where to get and put the application data, but not how it is represented in the control.

Finally when the user closes the form Causeway must call the *Destroy* function in the class of each control on the form. It cannot simply expunge the control because associated with the instance, and created by the *Create* and *Refresh* functions, there could be other resources that might then remain.

As each of the above functions is called, the current namespace changes to the class namespace for the duration of the call. This is normal (and unavoidable) in calling functions in Dyalog namespaces. Thus any other utility functions and variables needed by the main functions are there ready to be used. Classes that use $\square NA$ functions need only associate them once and they will then be available for use by all instances. Bitmaps, such as the VCR button faces and the tick box for the Yes/No control, can be shared between instances, thus saving space and reducing instance creation time. As mentioned above, utility functions can be shared between classes by including them in the parent namespace. The class functions need only refer to them e.g. as $\#\#.Label$ or $\#\#.Uppercase$.

Instances as Namespaces

The fact that all Dyalog GUI objects are themselves namespaces is not used by Causeway. It is possible for class functions to use the instance as a namespace but the decision is that of the implementor of the class.

An alternative approach is to copy the class functions into the instance namespace on creation of the instance, or to create the instance from the $\square OR$ of a previously created "proto-instance". This has the advantage that any data relevant to the instance is immediately available to the class functions. I have not taken this approach for a number of reasons.

Firstly, as mentioned above, if each instance uses the class functions directly then fixing a bug in a class function will fix it for all instances of the class. If the functions are copied into the instance or appear there from a $\square OR$ then, although

all the instances of the function begin identical, they are in fact separate and a change in the source will not affect the instances.

Also mentioned above is the sharing of data or resources between instances of a class. Although this is possible with functions existing and executing in the instance, by recording in the instance the location of the class definition and keeping the shared data there, it is easier using functions in the class namespace which have direct access to the data.

The third objection is that of space efficiency. Due to the mutable nature of Dyalog namespaces each one must carry its own symbol table, and all functions defined in the namespace must have their internal symbols matched against the symbols of the namespace. Thus when a function is copied from one namespace to another, although Dyalog APL is able to share the memory for the actual function itself, it must allocate memory for the correspondence of the function's symbols to those of the namespace. Furthermore any of the function's local symbols that are not already in the symbol table of the namespace must be added to it. This is entirely due to the dynamic nature of the namespace. In C++ a class definition is static for the running of the application, as it must be for a compiled language. Thus an instance of a class only takes enough memory for the data of its members. In a language as flexible as APL some added overhead is unavoidable.

Conclusion

It is clear that for the writer of an application development tool, namespaces are a two-edged sword. On the one hand they allow great simplification of the class management code and the structure of the class definitions. On the other hand to give this same flexibility to the user more work needs to be done before executing the application code.

There is no doubting however the benefits to the application developer. He can organise his application in the way that suits him. He can start a Causeway form from anywhere in the namespace tree and it will behave sensibly. He can change namespace willy nilly in the actions of a form: any new forms will run from the new namespace, but will still see local variables from their parent form.

Object Oriented Programming with APL+Win (Part I)

by Eric Lescasse (eric@lescasse.com)

*I want to thank Chris Lee from Softmed who inspired me
with his UDC (User Defined Classes) concept.*

Introduction

This long article, to be published in Vector in 2 parts, is extracted from the **Monthly APL+Win Training Program** available on the Internet from the **Lescasse Consulting** Web site at www.lescasse.com. It represents one of 3 chapters included in the APL+Win Training for May 97.

The article will show how a few simple APL utilities can be created to allow real Object Oriented Programming in APL+Win. It will both demonstrate how these utilities can be progressively built and introduce various Object Oriented concepts like *Encapsulation*, *Inheritance*, *Polymorphism*, *Methods Overloading*, etc. It will also show how you can create new classes of objects which inherit from the standard APL+Win Windows Interface objects or can inherit from each other, allowing you to build a complete hierarchy of new objects.

Object Oriented Programming with APL can be quite pure and much simpler than you may think. Read on.

The Source Idea

The original idea is as follows: APL+Win offers us a limited set of built-in objects which we programmers can easily manipulate with one system function: `⌊WI`. With `⌊WI`, we can query or set object properties, event handlers and invoke object methods.

If we create our own new objects for use in APL+Win, we will not be able to use the `⌊WI` system function to manipulate them since they will not be known to the APL+Win system itself. However, we can create a utility which would behave exactly like `⌊WI` and would allow us to work with our own objects "as if" they were built into the APL+Win system. A good name for this utility would be `Qwi`.

This being so, using *Qwi* we can resolutely adopt a pure Object Oriented style of programming and benefit from encapsulation, inheritance, polymorphism and methods overloading!

But, first let's think a little more about what exactly is an object.

Objects

An object is something which exists and has a particular set of properties and behaviour. An APL+Win object is something which exists and has a particular set of properties and methods. A car for example is a real-life object: it has a colour, a weight, a size, a maximum speed and average mileage per gallon which all are **properties**. It also knows how to start its engine when you turn the ignition key.

An APL+Win button is an APL+Win object: it has a colour, a size, a caption which all are **properties**; it also knows how to hide itself when you invoke its **Hide** method. A car has a **Start** method: somebody has built some mechanism within the car so that it understands the Start method (=turning the ignition key on) and reacts to it by starting the engine.

The APL+Win button has a **Hide** method: somebody has built some computer code within the button so that it understands the **Hide** method and reacts to it by hiding itself. The important thing to understand is the following:

- as a car user, you are not concerned about how mechanics work inside the car to start the engine when you turn the ignition key on: all you need to know is that it works the way you expect and also that it always reacts in the exact same way in the future
- as an APL+Win application user, you are not concerned about the C code which hides the button when you invoke its **Hide** method: all you need to know is that it works the way you expect and that it will always do so.

In other words, objects are "black boxes" with a well defined consistent built-in behaviour. Once the object has been made and is tested, you will never need to look inside it again: it will always behave according to the laws built into it. From the outside, all you can do, at any time, is:

- change some of its properties (you can repaint the car another colour)
- invoke any of its methods (turn the ignition key for example)

The key point is that a well-defined and well-built object will faithfully serve us forever (or almost: as you know it's not really the case as far as cars are concerned, unfortunately!).

On top of all this, we will programming in such a way that an APL+Win object will *really be an APL+Win function* (when you think about it, any software object always ultimately ends up being computer code anyway!). Functions are one of the easiest things to manipulate in APL and you may be surprised to see how easy it is to use Object Oriented Programming in APL+Win.

The Rewards of Creating New APL+Win Objects

In this chapter we will be extending the power of APL+Win by creating several new APL+Win objects — and we will be able to use and reuse these objects forever! Through these few examples, you will quickly see how some very simple new objects can save us a lot of time and effort, as well as make our programs much more secure and our applications much more consistent.

The rewards can be tremendous! But of course we have to pay a price for it (nothing is ever free!). It will cost us quite a lot of thinking and design to build our objects, perfect as we want them. They have to be totally secure so we will need to test them thoroughly. It will also cost us a small performance degradation. This is because we will not be able to avoid adding an APL code layer above `QWI` in order to work with our own object properties and methods. But we will try to keep this code layer as minimal as possible.

The First Building Block: the `Qwi` Function

The key to our new Object Oriented approach to APL+Win programming is the `Qwi` utility. We want it to accept any of the following possible syntax:

```
Qwself+'object_name' Qwi 'New' 'class_name'
Qwself+'object_name' Qwi 'New' 'class_name' ('prop1' value1) ... ('propN'
valueN)
res+ ['object_name'] Qwi 'prop'
      ['object_name'] Qwi 'prop' value
      ['object_name'] Qwi ('prop1' value1)...('propN' valueN)
res+ ['object_name'] Qwi 'Get_Method' [arguments]
      ['object_name'] Qwi 'Set_Method' [arguments]
```

where:

<code>object_name</code>	is a user defined object name
<code>class_name</code>	is the new object class name
<code>prop, prop1, ... , propN</code>	are object property names
<code>value, value1, ... , valueN</code>	are property values (character or numeric)
<code>Get_Method</code>	is a method which returns some values
<code>Set_Method</code>	is a method which sets some values
<code>arguments</code>	is a single argument or a nested vector of arguments
<code>[]</code>	denote optional elements

As you can see the *Qwi* syntax is very similar to the *QWI* syntax. You choose the class name of the new class you want to create: new class names can be identical to existing APL+Win built-in class names. *Qwi* will distinguish between the two:

```
'ff' Qwi 'New' 'Form'
```

But, since this new *Form* class is yours, you can make it work as you want. Let's take an example and we will explain the *Qwi* utility later in this chapter.

The Second Building Block: the Class Function

What will our new class of objects be represented by? As already mentioned, it will indeed and not surprisingly be represented by an APL function. This APL function name will also be the class name of our new class of objects. That's why I am calling it the **class function**.

Here is a very simple function which can serve as a model for building new objects for APL+Win. I have voluntarily simplified the first version to make things clearer. We will complement it later on.

```

v A QwiNewClass B;C;D;R;class;io
[1]  av A QwiNewClass B -- Template class function
[2]  av A ↔ object name
[3]  av B ↔ c'property'
[4]  av or 'property' value
[5]  av or c'Method'
[6]  av or 'Method' argument1 ... argumentN
[7]  av Requires: (F) QwiRegister
[8]  av Copyright (c) 1997 Eric Lescasse [8mar97]
[9]
[10]  io+1                                a environment
[11]  C+B                                a property or method
[12]  D+1+B                                a arguments or value
[13]
[14]  :select C
[15]
[16]  :case 'New'                            a constructor
[17]      a Add code here to create an instance of the class
[18]      a Example:
[19]      a R←A Qwi 'New' 'Form' 'Hide'    a create a Form object
[20]      a '#QwiAwres' R                  a result returned in Awres property
[21]      QwiRegister A                    a register new class name
[22]
[23]  :case 'Close'                            a close object
[24]
[25]  :case 'Destroy'                          a destructor
[26]
[27]  :end
v

```

The function comments describe the function arguments.

On line 11 the property or method name is extracted from the right argument.

On line 12 the property value or the method argument(s) if any, are retrieved from the right argument into *D*.

Then a simple `:select :case :case :end` control structure allows us to branch to various parts of the program depending on the property or method name used.

Note that all new objects will have a *New* method which we call the **constructor** method and that they will often have a *Close* method and a *Destroy* method which we call the **destructor** method: that's why we have already hard coded these special cases into our template new object function.

Note also that you should never remove the *class* local variable from a **class function**, even if it is not used in the function code. This local variable will help the *QwiClasses* utility find all user defined objects in the workspace: we will see that later in this chapter.

Finally, there is one subroutine called on line 21: *QwiRegister*, whose role is to register the new class of objects. We will comment on it later, but let's simply know for now that registering the new class consists of saving the object class name in the object *Δclass* user-defined data property.

When defining a new object we will:

- edit the *QwiNewClass* function
- change its name to some meaningful object class name
- write custom code for the *New* and possibly for the *Close* and *Destroy* methods
- add other properties or methods

Now that you have been quickly introduced to the basic blocks, let's take an example.

A New Class of APL+Win Forms

When you make some trials in your APL session, building forms with `QWI`, you might be tired of having to first delete an existing form before you can recreate it.

Example (assuming you already have an `ff` form object):

```
'ff' Qwi 'New' 'Form'
QWI OBJECT ERROR: Object already exists
'ff' Qwi 'New' 'Form'
^

'ff' Qwi 'Delete'
'ff' Qwi 'New' 'Form'

ff
```

So, let's create a new APL+Win Form object, which will know how to delete itself before it is recreated.

Here is a first attempt at this new `Form1` class:

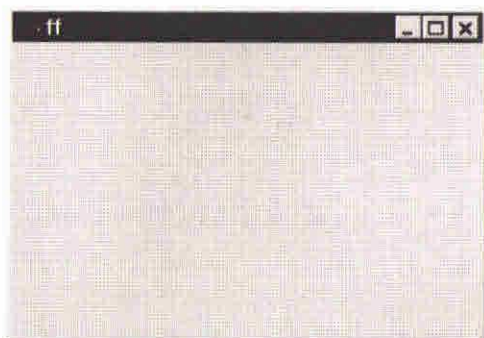
```
∇ A Form1 B;C;R;class
[1]  ∇ A Form1 B -- User Defined Form1 Class
[2]  ∇ A ↔ object name
[3]  ∇ B ↔ 'property'
[4]  ∇ or 'property' value
[5]  ∇ or 'Method'
[6]  ∇ or 'Method' argument1 ... argumentN
[7]  ∇ Requires: (F) QwiRegister
[8]  ∇ Copyright (c) 1997 Eric Lescasse [8mar97]
[9]
[10] C←B                                a property or method
[11]
[12] :select C
[13]
[14] :case 'New'                          a constructor
[15]     A Qwi 'Delete'                  a delete existing form
[16]     R←A Qwi 'New' 'Form'            a create form
[17]     '#'Qwi'Awres' R                 a result returned in Awres property
[18]     QwiRegister A                   a register class name
[19]
[20] :end
∇
```

The above `Form1` function represents our new `Form1` class of objects. These objects distinguish themselves from standard APL+Win forms by being able to first destroy themselves before being recreated. The `Form1` function is a modified copy of the `QwiNewClass` template function where we have removed all

unnecessary code. Our first example, the *Form1* class is a very simple new class. But it can already save us a lot of keystrokes every day.

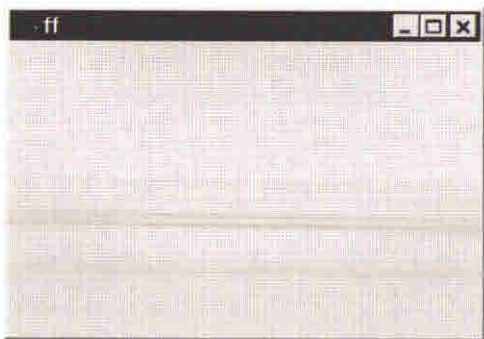
Let's use this new class:

```
ff      'ff' Qwi 'New' 'Form1'
```



Well! Let's try to recreate the form:

```
ff      'ff' Qwi 'New' 'Form1'
```



As you see this works well, without producing the traditional `QWI OBJECT ERROR: Object already exists` error message.

If you look at the *Form1* function you see that our new *Form1* class is the same as a standard APL+Win form class (`R←A Qwi 'New' 'Form'` on line 16) except

that its *New* method has been overloaded with an additional instruction (*A* `Qwi 'Delete'` on line 15).

OK, I am sure you already have some ideas on how to improve our new user defined *Form1* class.

Improving Our New *Form1* Class

My own preference with forms, is always to work with pixel coordinates ('*scale*' 5). So why not build that property in, by default. I would also like to have a nicer default caption, of *APL+Win form*. And be sure always to use the same default font of '*MS Sans Serif*' in size .8. Let's make these changes to our *Form1* class and rename it *Form2*. Here is what we get:

```

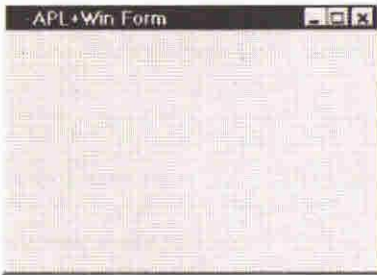
v A Form2 B;C;D;R;class;Qio
[1]  A Form2 B -- User Defined Form2 Class
[2]  A A ↔ object name
[3]  B ↔ 'property'
[4]  A or 'property' value
[5]  A or 'Method'
[6]  A or 'Method' argument1 ... argumentN
[7]  A Requires: (F) QwiRegister
[8]  A Copyright (c) 1997 Eric Lescasse [8mar97]
[9]
[10] Qio+1                                A environment
[11] C+A+B                                A property name or method name
[12] D+1+B                                A property value or method arguments
[13]
[14] :select C
[15]
[16] :case 'New'                            A constructor
[17]   A Qwi 'Delete'                        A delete existing form
[18]   R+A Qwi 'New' 'Form'                  A create form
[19]   '#Qwi 'Awres' R                        A result returned in Awres property
[20]   A Qwi 'caption' 'APL+Win Form'         A default caption
[21]   A Qwi 'font' 'MS Sans Serif' .8       A default font
[22]   A Qwi 'scale' 5                        A pixel coordinates
[23]   QwiRegister A                        A register class name
[24]
[25] :end
v

```

Lines 10 and 12 are not necessary for now, but will be later in this paragraph.

Let's try this new class:

```
ff' Qwi 'New' 'Form2'
```



Of course, since our new form class is built over the standard APL+Win form class, we can use the standard `Qwi` system function to query or set form properties afterwards:

```
ff' Qwi 'scale'
5 0 0 353 504

ff' Qwi 'font'
MS Sans Serif 0.8 0 ansi
```

However, if we wanted to use our `Qwi` function to retrieve standard form properties, we can see that no values are returned:

```
ff' Qwi 'font'
```

or more exactly the `Qwi` function returns an empty matrix.

```
p'ff' Qwi 'font'
0 0
```

This is because our `Form2` class function does not attempt to process right arguments other than those starting `'New'`.

We can easily fix this problem by adding an `:else` clause to the `:select` control structure of our `Form2` function and renaming it `Form3`, as follows:

```
v A Form3 B;C;D;R;class;0
[1] av A Form3 B -- User Defined Form3 Class
[2] av A ↔ object name
[3] av B ↔ 'property'
[4] av or 'property' value
[5] av or 'Method'
[6] av or 'Method' argument1 ... argumentN
[7] av Requires: (F) QwiRegister
[8] av Copyright (c) 1997 Eric Lescasse [8mar97]
[9]
[10] 0←1                                a environment
```

```

[11] C←+B                                ^ property name or method name
[12] D←+1+B                             ^ property value or method arguments
[13]
[14] :select C
[15]
[16] :case 'New'                           ^ constructor
[17]   A [Qwi>Delete'                     ^ delete existing form
[18]   R←A [Qwi 'New' 'Form'               ^ create form
[19]   '#'[Qwi 'Awres' R                   ^ result returned in Awres property
[20]   A [Qwi 'caption' 'APL+Win Form'     ^ default caption
[21]   A [Qwi 'font' 'MS Sans Serif' .8    ^ default font
[22]   A [Qwi 'scale' 5                     ^ pixel coordinates
[23]   QwiRegister A                       ^ register class name
[24]
[25] :else                                  ^ process all other props/methods
[26]   :if (C)←A [Qwi'properties'           ^ if a standard property ...
[27]   :orif (C)←A [Qwi'methods'           ^ or a standard method
[28]     :if 0←pD                           ^ when no value or arguments passed
[29]       '#'[Qwi'Awres'(A [Qwi C)         ^ return current value
[30]     :else                               ^ else...
[31]       A [Qwi (C),D                     ^ apply value or arguments
[32]     :end
[33]   :end
[34]
[35] :end

```

Now, we can also use *Qwi* to get or set standard APL+Win form properties as well as invoke standard APL+Win form methods:

```

'ff' Qwi 'New' 'Form3'
ff

'ff' Qwi 'font'
MS Sans Serif 0.8 0 ansi

'ff' Qwi 'font' 'Arial' 1

'ff' [Qwi 'font'
Arial 1 0 ansi

'ff' Qwi 'font'
Arial 1 0 ansi

('ff' [Qwi 'font')='ff' Qwi 'font'
1

```

Therefore, please add the same *:else* clause to our *QwiNewClass* template function so that it now reads as follows:

```

V A QwiNewClass B;C;D;R;class;[]io
[1]  A V A QwiNewClass B -- Template class function
[2]  A V A ↔ object name
[3]  A V B ↔ c'property'
[4]  A V or 'property' value
[5]  A V or c'Method'
[6]  A V or 'Method' argument1 ... argumentN
[7]  A V Requires: (F) QwiRegister '
[8]  A V Copyright (c) 1997 Eric Lescasse [8mar97]
[9]
[10] []io+1                                A environment
[11] C++B                                  A property name or method name
[12] D++B                                  A property value or method arguments
[13]
[14] :select C
[15]
[16] :case 'New'                            A constructor
[17]     A Add code here to create an instance of the class
[18]     A Example:
[19]     A R=A []wi 'New' 'Form' 'Hide'      A create a Form object
[20]     #'[]wi'Awres' R                      A result returned in Awres property
[21]     QwiRegister A                       A register new class name
[22]
[23] :case 'Close'                          A close object
[24]
[25] :case 'Destroy'                        A destructor
[26]
[27] :else                                  A process all other properties/methods
[28]     :if (cC)≠A []wi'properties'          A if a standard property ...
[29]     :orif (cC)≠A []wi'methods'          A or a standard method
[30]     :if 0≠pD                             A when no value or arguments passed
[31]         #'[]wi'Awres'(A []wi C)         A return current value
[32]     :else                                 A else
[33]         A []wi (cC),D                   A apply value or arguments
[34]     :end
[35] :end
[36]
[37] :end
V

```

Adding a Property to a User Defined Class

Let's add a new property to our *Form3* class and make it the *Form4* class. The new property will be called *ontop*. Just as for any standard APL+Win object property, you must use only lower case characters for property names.

The *ontop* property will be either 0 or 1. If it is set to 1, then the created instance of the *Form4* class will stay on top of all other forms.

We have already seen how to make a form “on top” of all other forms (see code starting at line 63 in the *SystemMenu* function described in Chapter 8 pp. 65-66).

Here is how we can implement the *ontop* property:

```
[25] :case 'ontop'                                # ontop property
[26]   :if 0 espD                                # if not property value
[27]     '#' [w1 'Δwres' (A [w1 'Δontop'])        # query property
[28]   :else                                       # else
[29]     S+(1+i=+D)>'hwnd_notopmost' 'hwnd_topmost' # windows constant to use
[30]     S+S 0 0 0 0 'swp_nomove swp_nosize'        # other arguments
[31]     S+[wcall'SetWindowPos' (A [w1 'hwnd'),S    # call SetWindowPos
[32]     A [w1 'Δontop' (1=+D)                    # register property value
[33]   :end
```

Let's comment the above lines of code. First, whenever you implement a property, you need to process 2 kinds of cases:

- the property is queried
- the property is set

That's why we have an *:if ... :else ... :end* clause above.

The key point to understand and the cleverness here is to use user-defined data properties to save our property values into the object itself. So we are using a *Δontop* user defined data property to store our boolean *ontop* property value.

When we query the *ontop* property, the code is as simple as retrieving the current value of the *Δontop* user-defined data property (line 27) and to return it as a result.

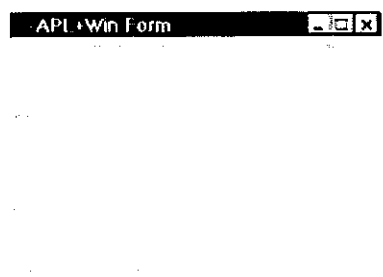
When we set the *ontop* property (lines 29 to 32), we need to perform 2 different actions:

- do the required job (lines 29 to 31)
- save the new value of the *ontop* property in the *Δontop* user-defined data property (line 32)

As you see it is pretty simple to implement a new property in a user-defined object. You just have to add a *:case 'property'* clause to your class function with the right code to query and set the property.

Let's test our new class:

```
'ff' Qwi 'New' 'Form4' ('ontop' 1)
ff
```



Now click into the APL session, outside the APL+Win form: as you see the APL+Win Form stays on top!

We can query the **ontop** property of the form:

```
'ff' Qwi 'ontop'
1
```

Of course, if we tried to do it using `QWI` we would get an error since the APL+Win system does not know about the **ontop** property for an APL+Win Form object:

```
'ff' Qwi 'ontop'
QWI ACTION ERROR: Action "ontop" not found
'ff' Qwi 'ontop'
^
```

Change the APL+Win form **ontop** property to 0:

```
'ff' Qwi 'ontop' 0
```

Now click into the APL session outside the APL+Win Form: this time the APL+Win form is sent to the back.

Let's now see how we can implement a new method for our form.

Adding a Method to a User Defined Class

Let's create a new *Form5* class starting with the *Form4* class, by adding a new method to the class. We would like to be able to easily move our form out of our way so that it is moved to the top right of the screen (or to the bottom right or bottom left or top left of the screen). Likewise it would be good if this new method knew how to centre the form on the screen.

Whenever you add a new property or method to an object, you have to think carefully about the design so that the method will be simple for the developer to use. All methods should start with an uppercase letter, as do all standard APL+Win methods. Let's call our new method *Move*. This method will accept 2 arguments. Each of them can be a character vector or an integer.

The 1st argument describes the vertical position of the form: it can be 'top' 'bottom' or 'center' or an integer denoting the percent of the vertical size of the screen where the form will be located

The 2nd argument describes the horizontal position of the form: it can be 'left' 'right' or 'center' or an integer denoting the percent of the horizontal size of the screen where the form will be located

So we would like to use such expressions as:

```
'ff' Qwi 'Move' 'top' 'right'
'ff' Qwi 'Move' 'bottom' 'left'
'ff' Qwi 'Move' 0 0
'ff' Qwi 'Move' 'center' 10
'ff' Qwi 'Move' 50 50
```

We will also design the *Move* method so that it returns the old form position in pixels. Here are the lines of code you should add to function *Form4* (which we rename at the same time as *Form5*) in order to implement the *Move* method:

```
[25] :case 'Move'
[26]   :if 2≠pD 0 :return 0 :end           * exit if not 2 arguments
[27]   (E F)→D                           * get method arguments
[28]   G←(1+82=⌊dr E)⇒E 0                 * G is 0 if 1st arg char
[29]   H←(1+82=⌊dr F)⇒F 0                 * H is 0 if 2nd arg char
[30]   (I J)←'#'⌊w1'size'                  * get screen dimensions
[31]   (K L M N)←A ⌊w1'where'              * get form position & size
[32]   '#'⌊w1'Awres'(K L×2p'#'⌊w1'units') * return old form position
[33]   K←(.01×0 50 100 G×I-M)⌈'top' 'center' 'bottom'⌈E * new vert form pos
[34]   L←(.01×0 50 100 H×J-N)⌈'left' 'center' 'right'⌈F * new horz form pos
[35]   A ⌊w1'where' K L M N                * move form
```

This method is pretty simple to understand.

First, if the **Move** method did not receive exactly 2 arguments, we do nothing and exit (line 26).

If 2 arguments were passed to **Move**, as expected, we retrieve them in variables *E* and *F* (line 27).

On lines 28 and 29, we transform *E* and *F* to 0, only if they are character vectors: if *E* and/or *F* were numeric, we do not change their values. We store the new values in variables *G* and *H*.

Then we need to retrieve the screen dimension in row/col coordinates (line 30).

Then we retrieve the current form position in variables *K* and *L* and the current form size in variables *M* and *N* on line 31.

On line 32 we save the current form position in the *Δwres* property (the result of our **Move** method).

The new form vertical position is easily calculated on line 33 and the new horizontal form position is easily calculated on line 34, which each handle all cases of arguments in just one simple instruction. A much longer alternative way of writing line 33 would for example be:

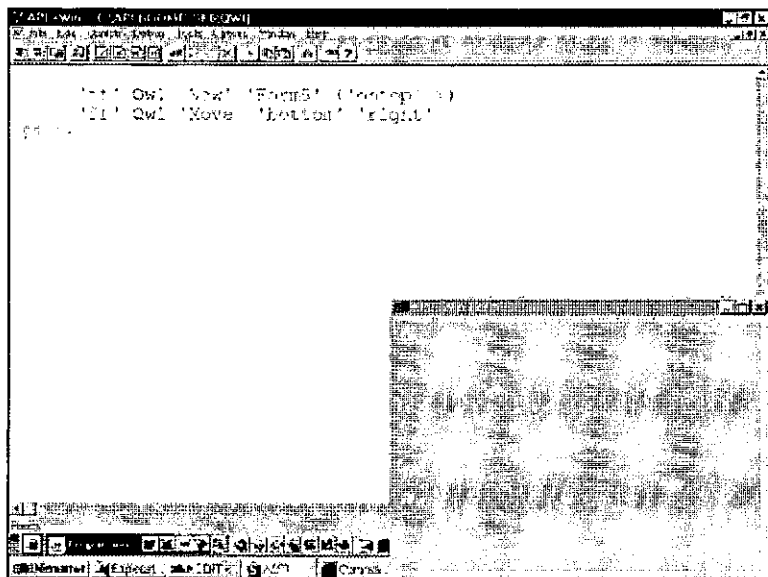
```
:select E
:case 'top'
    K+0
:case 'center'
    K+.5*I-M
:case 'bottom'
    K+I-M
:else
    K+.01*E...
:end
```

Finally the form is moved by changing its *where* property with the $\square$ WI system function, on line 35.

Let's try our new class:

```
ff      'ff' Qwi 'New' 'Forms' ('ontop' 1)
ff
      'ff' Qwi 'Move' 'bottom' 'right'
12 32
```

resulting in the following screen where I have increased the APL font size so that you can read it better .



Note that we could have specified everything in just one expression:

```
10' Qwi 'New' 'Form5' ('ontop' 1) ('Move' 'bottom' 'right')
192 256
```

but please note that in such a case, *Qwi* always returns the result of the rightmost method in the phrase.

So, after addition of the *Move* method, the *Form5* class function is as follows:

```

v A Form5 B;C;D;E;F;G;H;I;J;K;L;M;N;R;S;class;[]io
[1]  av A Form5 B -- User Defined Form5 Class
[2]  av A ↔ object name
[3]  av B ↔ 'property'
[4]  av or 'property' value
[5]  av or 'Method'
[6]  av or 'Method' argument1 ... argumentN
[7]  av Requires: (F) QwiRegister
[8]  av Copyright (c) 1997 Eric Lescasse [Bmar97]
[9]
[10] []io+1                                a environment
[11] C+B                                  a property name or method name
```

```

[12] D+1+B                                     * property value or method arguments
[13]
[14] :select C
[15]
[16] :case 'New'                                 * constructor
[17]   A [w1]'Delete'                           * delete existing form
[18]   R+A [w1] 'New' 'Form'                     * create form
[19]   #'[w1] 'Awres' R                         * result returned in Awres property
[20]   A [w1] 'caption' 'APL+Win Form'           * default caption
[21]   A [w1] 'font' 'MS Sans Serif' .8         * default font
[22]   A [w1] 'scale' 5                          * pixel coordinates
[23]   Qw1Register A                             * register class name
[24]
[25] :case 'Move'
[26]   :if 2≠pD :return 0 :end                  * exit if not 2 arguments
[27]   (E F)+D                                  * get method arguments
[28]   G←(1+82=[dr E])>E 0                      * G is 0 if 1st arg char
[29]   H←(1+82=[dr F])>F 0                      * H is 0 if 2nd arg char
[30]   (I J)+#[w1]'size'                        * get screen dimensions
[31]   (K L M N)+A [w1]'where'                  * get form position & size
[32]   #'[w1]'Awres'(K L×2p#[w1]'units')        * return old form position
[33]   K←(.01×0 50 100 G×I-M)[ 'top' 'center' 'bottom' ]+E * a new vert form pos
[34]   L←(.01×0 50 100 H×J-N)[ 'left' 'center' 'right' ]+F * a new horz form pos
[35]   A [w1]'where' K L M N                    * move form
[36]
[37] :case 'ontop'                               * ontop property
[38]   :if 0≠pD                                 * if not property value
[39]     #'[w1] 'Awres' (A [w1] 'Aontop')       * query property
[40]   :else                                     * else
[41]     S←(1+1≠+D)>'hwnd_notopmost' 'hwnd_topmost' * windows constant to use
[42]     S+S 0 0 0 0 'swp_nomove swp_nosize'      * other arguments
[43]     S←[w1]call'SetWindowPos' (A [w1]'hwnd'),S * call SetWindowPos
[44]     A [w1] 'Aontop' (1≠+D)                  * register property value
[45]   :end
[46]
[47] :else                                       * process all other properties/methods
[48]   :if (cC)≠A [w1]'properties'               * if a standard property ...
[49]   :orif (cC)≠A [w1]'methods'                * or a standard method
[50]     :if 0≠pD                                * when no value or arguments passed
[51]       #'[w1]'Awres'(A [w1] C)              * return current value
[52]     :else                                    * else...
[53]       A [w1] (cC),D                         * apply value or arguments
[54]     :end
[55]   :end
[56]
[57] :end

```

▽

Simplest Form of the *Qwi* Function

Qwi is the very chore of our Object Oriented approach to APL+Win development. Here is a simple first version of *Qwi* which is sufficient to run the examples we have shown before:

```

v R←A Qwi1 B;C;E;I
[1]  av R←A Qwi1 B -- [WI emulator
[2]  av [wself←'object_name' Qwi1 'New' 'class_name'
[3]  av [wself←'object_name' Qwi1 'New' 'class_name'('prop1' value1)...
[4]  av res← ['object_name'] Qwi1 'prop'
[5]  av      ['object_name'] Qwi1 'prop' value
[6]  av      ['object_name'] Qwi1 ('prop1' value1)...('propN' valueN)
[7]  av res← ['object_name'] Qwi1 'Get_Method' [arguments]
[8]  av      ['object_name'] Qwi1 'Set_Method' [arguments]
[9]  av Copyright (c) 1997 Eric Lescasse [8mar97]
[10]
[11]  :if 2≠nc'A' ∘ A←[wself ∘ :end  a [WSELF if no left arg
[12]  :if 1≠B ∘ B←,cB ∘ :end        a be sure B is nested
[13]  :if 'New'≠B                    a if method is 'New'
[14]      C←2>B                      a get specified class name
[15]      B←(c'New' C),2×B            a rebuild argument
[16]  :elseif 0≠pA [w1'self'          a else if object A exists
[17]      C←+A [w1'Ac1ass'            a get registered class name
[18]  :end
[19]  :if 2≠B ∘ B←,cB ∘ :end          a ensure B depth is ≥ 3
[20]  E←2>B←B ∘ (E/B)←c'E/B          a ensure each element is of depth 2
[21]  B←,B ∘ '#'[w1'Awres'(0 0p0)     a define default result
[22]  :for I :1n 1pB                  a for each element in argument
[23]      z'A 'C,' I=B'              a execute the class function
[24]  :end
[25]  R←'#'[w1'Awres'                a return result saved in Awres
v

```

On line 11, we set the left argument *A* to *[WSELF* if it has been omitted. Then, from line 12 to line 20 we normalize the right argument *B* so that it has a normalized depth 3 structure. For example, arguments such as:

```

      ]display 'ontop'
┌───┐
│ontop│
└───┘

```

are changed to:

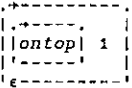
```

      ]display ,c,c'ontop'
┌───┐
│┌───┐│
││ontop││
││'ε'───┐│
││'ε'───┐│
└───┘

```

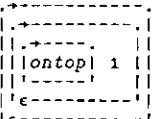
while arguments such as:

```
]display 'ontop' 1
```



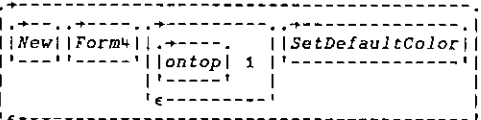
are changed to:

```
]display ,c'ontop' 1
```



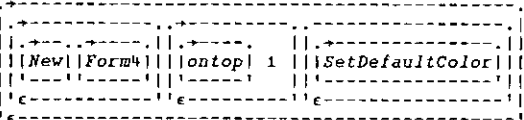
and finally arguments such as:

```
]display 'New' 'Form4' ('ontop' 1) 'SetDefaultColor'
```



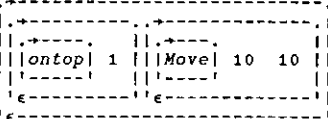
are changed to:

```
]display ('New' 'Form4') ('ontop' 1) (c'SetDefaultColor')
```



Finally, arguments such as:

```
]display ('ontop' 1) ('Move' 10 10)
```



already have the right structure and are left unchanged.

Lines 12 to 20 have a second role: they also are used to find out what is the object class name.

If we are constructing an instance of a new object (i.e. if the *New* method was invoked in the *Qwi1* right argument), then the second element of the *Qwi1* right argument is the class name (line 14).

If method *New* was not invoked, that means we are attempting to work on an existing instance of the class and in that case, the class name has been saved in this instance $\Delta class$ user-defined data property (line 17).

On line 20 we set the system object $\Delta wres$ user-defined data property to a default empty matrix value. This property will get any result that the class function needs to return.

Then a simple *:for* loop allows us to process each element of the normalized *Qwi1* right argument, one at a time.

For example, if we called:

```
'ff' Qwi1 'New' 'Form5' ('ontop' 1)
```

the right argument is first normalized to be:

('New' 'Form5') ('ontop' 1) and the *:for* loop is performed twice.

The 1st time it executes the following expression:

```
A Form5 'New' 'Form5'
```

and the second time it executes:

```
A Form5 'ontop' 1
```

where *Form5* is our class function.

Finally on line 25, any value saved in the $\Delta wres$ system object property is retrieved and returned by *Qwi1*. Since each call of *Form5* might have saved a result value into $\Delta wres$, *Qwi1* only returns the last one saved in $\Delta wres$.

This should not be a limitation since you can always decide to cut your *Qwi1* instruction into several *Qwi1* instructions in order to get all results.

For example:

```
'ff' Qwi1 'New' 'Form4' ('Move' 10 10)
192 256
```

and we get only the result of the *Move* method, losing the result of the *New* method. If we needed both results, we could always write:

```

      'ff' Qwi1 'New' 'Form4'
ff
      'ff' Qwi1 'Move' 10 10
192 256

```

Adding Error Handling to Qwi1

If you do a lot of trials from the APL session and make typos you may mis-spell the class name when creating a new instance of an object.

In that case you may have seen that you get the following error:

```

      'ff' Qwi1 'New' 'Test'
VALUE ERROR
Qwi1[24] * A ΔTest I>B
      ^

```

It would be much nicer to exit cleanly from *Qwi1*, signalling an error to the user, as in:

```

      'ff' Qwi1 'New' 'Test'
Unknown class: Test
      'ff' Qwi1 'New' 'Test'
      ^

```

There are 2 ways we could achieve this:

- first, we could add an APL test just before the *:for* loop to check if we do have a class function of the right name (here *Test*) available in the workspace
- second, we could rely on error handling to trap a *VALUE ERROR* on line 23

I am choosing to show you the second approach, since you all know how to perform the first, since the second approach has faster execution, and finally since it is another opportunity to use the remarkable *HANDLERFOR* utility which we already introduced in Chapter 9 (Error Handling).

In order to add error handling to our *Qwi1* function, we need to do 3 things:

- localize `⌈ELX`
- set `⌈ELX` to `'*HANDLERFOR ⌈DM'` at the beginning of our *Qwi1* function
- add an error handling comment on the line which could fail with *VALUE ERROR* (line 23)

Line 23 should become:

```
[24]      A 'C,' I>B' A{VALUE:Error'Unknown class: ',C}
```

If no error occurs when line 23 executes, our error handling comment will consume no CPU time. But if a *VALUE ERROR* occurs, the expression following the colon in our error handler will automatically execute and an error will cleanly be signalled to the calling environment.

Example:

```
'ff' Qw11 'New' 'Test'
Unknown class: Test
'ff' Qw11 'New' 'Test'
^
```

To finish with error handling, we could add a couple of other tests to signal errors to the calling environment. The *Qw11* function, dressed with error handling is renamed *Qw12* is:

```
V R+A Qw12 B;C;E;I;[elx
[1]  A R+A Qw12 B -- [WI emulator
[2]  A [wself+'object_name' Qw12 'New' 'class_name'
[3]  A [wself+'object_name' Qw12 'New' 'class_name'({'prop1' value1)...
[4]  A res+ ['object_name'] Qw12 'prop'
[5]  A [ ['object_name'] Qw12 'prop' value
[6]  A [ ['object_name'] Qw12 ('prop1' value1)...('propN' valueN)
[7]  A res+ ['object_name'] Qw12 'Get_Method' [arguments]
[8]  A [ ['object_name'] Qw12 'Set_Method' [arguments]
[9]  A Copyright (c) 1997 Eric Lescasse [8mar97]
[10]
[11] [elx+'HANDLERFOR [DM'
[12] :if 2=[nc'A' o A+[wself o :end A [WSELF if no left arg
[13] :if 1=[B o B+,cB o :end A be sure B is nested
[14] :if 'New'=[B A if method is 'New'
[15] C+2=B A get specified class name
[16] B+(c'New' C),2+B A rebuild argument
[17] :elseif 0=[A [wi'self' A else if object A exists
[18] :if 0=C+A [wi'aclass' A get registered class name
[19] Error'Use [WI instead of Qw12' A if none signal error
[20] :end
[21] :else A if not 'New' & not an object
[22] Error'Unknown object: ',A A signal error
[23] :end
[24] :if 2=[B o B+,cB o :end A ensure B depth is > 3
[25] E+2=[B o (E/B)+c'E/B A ensure each element is of depth 2
[26] B+,B o '#[wi'Awres'(0 0p0) A define default result
[27] :for I :in 1pB A for each element in argument
[28] A 'A 'C,' I>B' A{VALUE:Error'Unknown class: ',C}
[29] :end
[30] R+'#[wi'Awres' A return result saved in Awres
V
```

Here are a few examples showing the new *Qwi2* behaviour:

First example: trying to set a property with *Qwi2* on an object not created with *Qwi2*

```
'gg' Qwi 'New' 'Form'
gg
'gg' Qwi2 'font' 'Arial' 1
Use QWI instead of Qwi2
'gg' Qwi2 'font' 'Arial' 1
  ^
```

Second example: trying to use a method or set a property with *Qwi2* on a non-existent object:

```
'hh' Qwi2 'ontop' 1
Unknown object: hh
'hh' Qwi2 'ontop' 1
  ^
```

More information about the **Monthly APL+Win Training Program** can be obtained from the Lescasse Consulting's Web site at www.lescassee.com.

Please note: I have created a new company called "Lescasse Consulting": Uniware has transferred its whole APL business to Lescasse Consulting. The Monthly APL+Win Training Program is now distributed worldwide by Lescasse Consulting, except in the USA and Finland where we have local distributors.

Eric Lescasse
Lescasse Consulting (SARL)
18 Rue de la Belle Feuille
92100 Boulogne, France.

Tel: (33) 1.46.05.10.76; fax: (33) 1.46.04.60.23

APL and J (5): Operators – Inner Product, Each, Commute, Compose

by Chris Burke

This series of articles entitled *APL and J* will try to explain the J way of thinking, by contrasting APL and J coding examples. The term APL will be used generically to refer to facilities found in most commercial implementations of APL.

To conclude the discussion of APL's operators, we now look at *inner product*, *each*, and two operators found only in Dyalog APL: *commute* and *compose*.

Housekeeping

As usual, we use the letters *V*, *M*, and *A* for vectors, matrices and arrays of any dimension, optionally followed by numbers to denote their shape. Thus *V* for any vector, *V*5 for a vector of length 5, *M*4 5 for a 4-by-5 matrix, *A*2 4 5 for a 2-by-4-by-5 array. Where actual values are required, *V*5 will be defined as 15, *M*4 5 as 4 5 p 120.

APL terminology will be used, except where J terminology is required for clarity. Here, $\square i o + 1$ is assumed throughout.

Inner Product

Inner product in APL is called *dot product* in J, and both are denoted by the dot symbol, *f . g* for functions *f* and *g*. As may be expected, *f* is applied with reduce in APL, but not in J, where it must be given explicitly if required. Thus the following are equivalent:

<code>V3 +. × M3 4</code>	APL
38 44 50 56	

<code>V3 +/ . × M3 4</code>	J
38 44 50 56	

(Note that in J, a space is required before the dot – since otherwise the dot would be read as part of the previous symbol.)

Does *f . g* in APL always give the same result as *f/ . g* in J? Yes – if we look at typical APL uses of inner product, such as *+. ×* or *^.=*. However, the definitions

are quite different, and nicely illustrate the two approaches to array operations. J also allows more possibilities, though I have yet to use a dot product in J that could not be written in APL.

In the following discussion, we apply dot product as: $X \cdot f \cdot g \cdot Y$ where the shapes of X and Y are SX and SY .

In APL, $\neg 1 + SX$ must match $1 + SY$, and the computation is made by splitting X into vectors along the last axis, and splitting Y into vectors along the first axis, and applying g followed by $f/$ to each combination of vectors from X and Y , and enclosing the individual results.

Since the individual results are enclosed, they do not contribute to the result shape, which must therefore be $(\neg 1 + SX), 1 + SY$. The computation for each pair of vectors x from X and y from Y is:

$$c \leftarrow f/ \ x \ g'' \ y$$

For example, $M22 \ +. \times \ M23$ is evaluated as:

$$\begin{aligned} & (c \ +/ \ 1 \ 2 \ x'' \ 1 \ 4) \ (c \ +/ \ 1 \ 2 \ x'' \ 2 \ 5) \ (c \ +/ \ 1 \ 2 \ x'' \ 3 \ 6) \\ & (c \ +/ \ 3 \ 4 \ x'' \ 1 \ 4) \ (c \ +/ \ 3 \ 4 \ x'' \ 2 \ 5) \ (c \ +/ \ 3 \ 4 \ x'' \ 3 \ 6) \end{aligned}$$

and assembled into the matrix:

$$\begin{array}{ccc} 9 & 12 & 15 \\ 19 & 26 & 33 \end{array}$$

In J, the right argument Y is not split up at all, while the left argument X is split up into cells of rank 1 plus the left rank of g (you can think of *lists* of left argument cells). The number of items in the left argument cells must match the number of items in Y . Therefore, in the common case where the left rank of g is 0 (e.g. where g is $*$ or $=$), X is split up into vectors along the last axis just as in APL.

For example, $M22 \ +/ \ . \ * \ M23$ is evaluated as:

$$(+/ \ 1 \ 2 \ * \ M23) \ (+/ \ 3 \ 4 \ * \ M23)$$

which is:

$$(9 \ 12 \ 15) \ (19 \ 26 \ 33)$$

which is then assembled into the matrix:

```

  9 12 15
19 26 33

```

Note that this makes use of J's ability to multiply a vector by a matrix or arbitrary array, using prefix agreement.

The shape of the result of dot product in J is the shape of the frame of X (i.e. SX less the shape of the cells used in the dot product), catenated with the shape of the result cells. In the following example, let $f = (i.10)_{-}$ be the function that returns the list $i.10$, and $g = 0"2$ has left rank 2, then:

```

$ (2 3 4 5 6$'a') f . g 4 5 $'b'
2 3 10

```

The key changes J makes are: the right argument Y is not split up at all; the left argument X is split up according to the left rank of g ; and the results of each computation are not enclosed, but follow normal assembly rules.

Determinant and Permanent

In J, the dot product can be used monadically, when it is the *determinant* function, thus:

```

-/. * 2 2 $ 10 7 1 3
23

```

If $+$ is used instead of $-$, the function is the *permanent*. Any other verb could be used in place of $-$ or $+$, but I don't know of a useful example!

Each

The APL expression f^{**} (f each) means apply f to each scalar in an array, opening each scalar before applying the function, and enclosing the result. Since each result is enclosed, it must be that $(\rho f^{**}X) = \rho X$.

J has no built-in equivalent to the APL each operator. However, one can be constructed as a special case of the *under* conjunction $\&$, and this is defined in the J standard library. Given verbs u and v where the inverse v^{-1} is defined, then $u\&.v$ is equivalent to $v^{-1} u v$.

Now, if v is the verb $>$ (open), then its inverse is $<$ (box), so that $f\>$ means open, apply f , then box, which is exactly the same as APL's f'' , for example:

```
n=. 'winston';(i.3 4);10 20

$ &. > n
++-----++
|7|3 4|2|
++-----++
```

So if we define $each$ to be $\&. >$, then this is an adverb whose definition and behaviour is exactly like APL's $each$ operator:

```
each=. &. >

$ each n
++-----++
|7|3 4|2|
++-----++
```

J's rank operator $"$ and APL's $each$ operator $"$ are notationally similar, and should not be confused. For further discussion, see the first article in this series (Vector 13.1 pp. 96-101, July 1996).

Commute

In Dyalog APL, the *commute* operator $\ddot{}$ creates a new function which reverses its arguments, so that $X\ f\ddot{ } Y$ is equivalent to $Y\ f\ X$, for example:

```
5 -\ 7
2
```

The corresponding operator in J is $\sim$, which reverses the arguments when the resulting function is used dyadically:

```
5 -- 7
2
```

Where the resulting function is used monadically, the argument is used as both left and right argument, so that $u\sim y$ is $y\ u\ y$, for example:

```
#~5
5 5 5 5 5
```

Why bother with such a trivial operator? For one thing because it simplifies expressions by reducing the need for parentheses, and for another because it is useful in constructing tacit definitions, for example:

```
firstword=. {~ i.&' '
firstword 'In the beginning'
In
```

Compose

In Dyalog APL, the *compose* operator $\circ$ takes two arguments which are either a function and one of its arguments; or both functions.

In the first case, an argument is bound to a dyadic function, creating a new monadic function:

```
findblank← 1 ∘ ' '
findblank 'one two three'
4
```

The J equivalent is $\&$, for example:

```
findblank=. i. & ' '
```

Joining a function with one of its arguments to create a new function is called *currying* in honour of Haskell Curry.

In the second case, two functions are composed, creating a new function $f \circ g$ which applies first g , then f . The function g is used monadically, while f may be monadic or dyadic.

Where f is monadic, $f \circ g \ X$ is equivalent to $f \ g \ X$, for example:

```
f← +/ ∘ 1
f 100
5050
```

The J equivalent for the monadic case is given by @ or & (either will do), e.g.

```
int=. 1: + i.          origin-1 integers
```

```
f=. +/ @ int
```

```
f 100
5050
```

Where f is dyadic, $X \ f \ o \ g \ Y$ is equivalent to $X \ f \ g \ Y$, for example:

```
f+ x o i
```

```
10 f 5
10 20 30 40 50
```

This composition is the same as J's hook:

```
f=. * int
```

```
10 f 5
10 20 30 40 50
```

This example makes it clear that a hook could have been constructed as a normal operator, rather than a train of 2 verbs. The reason for making it a train is that it complements the fork, which is a train of 3 verbs, allowing resolution of trains of any length.

Note that where f is dyadic, then $f \ o \ g$ is not the same as J's $f \ @ \ g$ or $f \ & \ g$. In these cases:

$X \ f \ @ \ g \ Y$	is	$f \ X \ g \ Y$
$X \ f \ & \ g \ Y$	is	$(g \ X) \ f \ (g \ Y)$

Reference

Iverson, K.E. *J Introduction and Dictionary*, Iverson Software, 1996

At Play With J: Oh, No, Not Eigenvalues Again!

by Eugene McDonnell (eemcd@aol.com)

I can't explain why it is that I keep running into problems associated with eigenvalues. I don't seek them out, and have no interest in them, but there it is — they keep cropping up before me and I have somehow to find a way to drive a stake through their hearts before I can go on to something else.

When the eigenvalue problem last loomed before me, I found that I had to go back to basics in order to come to terms with it. In the course of doing so, I happened upon a technique in J for finding eigenvalues which is eminently satisfying pedagogically, since it can be used for small matrices to show all the theory of eigenvalues, even though it is staggeringly inefficient, becoming dreadfully slow for matrices of size 5 or 6 or larger. It is its pedagogical merit that I commend to you.

The basics are simple. If A is a square numeric matrix, we replace each diagonal atom a_{ii} with the two-atom list $a_{ii} _1$. What we are doing is replacing the problem of evaluating the determinant of a numeric matrix with that of evaluating the determinant of a matrix of polynomials. Of course, the non-diagonal terms are the simplest of polynomials, that is, constants, but the diagonal terms are all linear polynomials. Our problem is to evaluate the determinant of this polynomial matrix. In J we evaluate the determinant of a numeric matrix with the monad of the dot conjunction:

`det =: -/ . *` NB. (1)

For example,

```
m=: 2 3,: 5 8
m
2 3
5 8
det m
1
```

The verb `det` is not directly suited to our purpose, but with a few manoeuvres we'll get where we want to go. First, let's take a suitable matrix for our example (the example and much of the approach I have borrowed from Cornelius Lanczos's book, *Applied Analysis*, Prentice-Hall, 1956, chapter II):

```

]A =: 33 16 72 , _24 _10 _57 .: _8 _4 _17
33 16 72
_24 _10 _57
_8 _4 _17

```

We can obtain the diagonal atoms of A using the monad d [JP 3.B.m4, meaning phrase m4 from section B of chapter 3 of the book *J Phrases*].

```

]a0 =: d A
33 _10 _17

```

We append _1 to each of these using J's stitch (.,.):

```

]a1 =: a0 ., _1
33 _1
_10 _1
_17 _1

```

In order to be able to work with a matrix in which a single number is replaced by a list of two numbers, it will be necessary to box the rows of the 2-column matrix above, using monad B1 [JP 1.C.m12]

```

]a2 =: B1 a1

```

33	_1	_10	_1	_17	_1
----	----	-----	----	-----	----

In order to amend the diagonal atoms of A, we'll box the atoms of matrix A first, using monad B0 [JP 1.C.m11]:

```

]a3 =: B0 A

```

33	16	72
_24	_10	_57
_8	_4	_17

The amend adverb in J needs the indices of the places to be amended. We obtain these using the monad d again, modified by the adverb IR [JP 3.B.a3].

```

]a4 =: (d IR) A
0 4 8

```

We can use amend now to obtain the matrix we want:

```
Ja5 =: a2 a4 } a3
```

33 _1	16	72
_24	_10 _1	_57
_8	_4	_17 _1

For use later, we'll collect the steps taken so far to form the verb db:

```
db =: B10(d .. _1:) d IR } B0
```

Now we are in position to evaluate the determinant of this matrix. We can't use the det verb from above, since it can only deal with matrices whose atoms are simple numbers. My first thought was to use the definition of the Determinant conjunction $u \cdot v$ from the *Introduction and Dictionary*:

```
u=: -/ [ . v=: *
DET=: v/0,`({."1 u .v$:0minors)0.(1&<0{:0$)"2
minors=: )."10(1&([\.))
```

replacing the definitions of u and v by polynomial difference and polynomial product, dyads dif and ppr [JP 9.C.d1 and 9.C.d2]. However, I hadn't proceeded far when I suddenly realized that, since DET was the definition of the monadic form of the dot conjunction, I ought to be able to use dot itself directly, and not mess with its sybilline definition. The first thing I did was to provide myself with modified forms of dif and ppr , ones which operated on boxed atoms:

```
difb =: difa.>
pprb =: ppra.>
```

Taking this tack, I could now write, in direct analogy with (1),

```
detp =: difb/ . pprb
```

and then apply this to matrix a5:

```
detp a5
```

6 _11 6 _1

I wanted an open, not a boxed result, so modified the definition of $detp$:

```
detp=:>0(difb/ . pprb)
Ja6 =: detp a5
6 _11 6 _1
```

This looked promising, but was it indeed the characteristic polynomial of my matrix? One way to find out was to use the Cayley-Hamilton theorem, which says that a matrix satisfies its own characteristic equation. To see whether this was so, I found the first four powers of A, using the monad I [JP 9.A.m0] to form the 0th power of the matrix, that is, the identity matrix:

```
]a7=:A&ip^:(i.@>:@#`]) I A
```

```
1 0 0
0 1 0
0 0 1
```

```
33 16 72
_24 _10 _57
_8 _4 _17
```

```
129 80 240
_96 _56 _189
_32 _20 _59
```

```
417 304 648
_312 _220 _507
_104 _76 _161
```

then multiplied these by the supposed coefficients of the characteristic equation:

```
]a8=:a6 * a7
```

```
6 0 0
0 6 0
0 0 6
```

```
_363 _176 _792
264 110 627
88 44 187
```

```
774 480 1440
_576 _336 _1134
_192 _120 _354
```

```
_417 _304 _648
312 220 507
104 76 161
```

and lastly, summed a8, trusting that the result would be the zero matrix:

```
+ /a8
```

```
0 0 0
0 0 0
0 0 0
```

From here it was a short step to the eigenvalues: they are the roots of this polynomial, so I could use J's polynomial primitive (p.):

```
p. detp a6
┌─1 3 2 1┐
```

The result I wanted was the open (>) of the tail (:) of this one, so I made another modification:

```
a9 =: >@{: @p. @detp a6
3 2 1
```

The parts can now be assembled to give the eigenvalue finder cm:

```
cm =: > @ {: @ p. @ detp @ db
cm A
3 2 1
```

If these are the eigenvalues, their product should be equal to the determinant of the matrix.

```
*/ cm A
6
det A
6
```

So far, so good. Furthermore, if these are the eigenvalues, if any one of them is subtracted from the diagonals of A, the determinant of the result should be zero. Is this the case?

```
det A - 3 * I A
0
det A - 2 * I A
0
det A - 1 * I A
0
```

Yes it is.

The verb cm is a model of the monad of the c. primitive described in the *J Introduction and Dictionary*. It differs in that the roots are given in order of descending magnitude, which is how the polynomial (p.) primitive provides them, rather than the ascending order prescribed in the *Dictionary*. Since c. has not yet been implemented, it's anyone's guess how p. and c. will be reconciled. I've brought this matter to the attention of the proper authorities, so they do at least know that the problem exists.

API Problem as a Particular Recurrence Problem

by Andrei Buzin (buzin@ext1.ccas.ru)

The problem discussed in Robert Bernecky's letter in Vector 13.4 p.7 concerns the inconvenience of programming recurrence and iteration algorithms in APL. This problem is critical for APL for the following reason: a major advantage of APL is its conciseness, but the better part of recurrence and iteration algorithms *must* be coded in FORTRAN style, e.g. by loops. You must do this not on account of impossibility in principle, but because the concise APL expressions process the work extremely slowly or are extremely expansive in memory usage.

For example, you can solve the API problem, mentioned in the last three Vectors, with the following APL expressions (in DyalogAPL):

```
k 1''(10p)↑''c p
```

or

```
k 1'',\p
```

where k is 0.5 and p is 1 2 3 4 in Roger Stokes' example, but they are only quasi-solutions, because they give WS FULL or work too slowly if p is rather long.

I have faced this problem in teaching computational methods to students of applied mathematics. On the one hand APL is convenient for showing the essence of some method, to code it rapidly and to make experiments with it. On the other hand, if we use loops, the program runs slowly. And very often we can program the method in APL style, without loops, by using "each" or "scan"; such solutions are elegant, but they work extremely ineffectively!

For example, the Euler solution of differential equations can be programmed in modern APL as `Euler\N/x0`, where N is the number of points in the solution, $x0$ is the initial point (it can be a vector) and `Euler` is a simple function, which describes one step of the solution. This solution works, but is extremely slow! (As well as Robert Bernecky I had made some timing experiments for optimal loop organization; really the best is using the "pre-allocation" practice — this result was distressing for me).

Moreover, I have seen that all recurrent problems of type

$$x_i = f(x_{i-1}, x_{i-2}, \dots, x_{i-k}), \quad i=k+1, k+2, \dots, N \quad (1)$$

can be solved by the scan operator $F \backslash N / \subset x_1 \ x_2 \ \dots \ x_k$ (even in the case when x is an arbitrary array)! Function F depends on function f and has dummy left argument. But unfortunately, in practice the expression $F \backslash x$ is calculated as $F / \dots x$ except in the cases when F is a associative primitive function. You can see that in the case where the function F is associative, the resulting elements of $F \backslash$ can be calculated recurrently. The other case is where the function F does not depend on the left argument. But in the general case the resulting elements cannot be calculated recurrently because of APL conventions about the sequence of calculations and the Scan Operator.

A more general form of recurrent (and iteration) calculations is

$$x_i = f(a, i, x_{i-1}, x_{i-2}, \dots, x_{i-k}) \quad (2)$$

where a is the set of parameters, i is the index (or multi-index). The API problem is one example; in the API problem $k=1$ and $a = k \ p$ (here k and k differ). What is very important, is that *almost all classical computational methods can be represented in such form* (x can be a nested array).

The problem that is discussed in Vector is that we can not avoid loops in some recurrence problems and that loops work slowly in APL. Robert Bernecky says that "It is time for APLers to admit that iteration has its place in a programmer's toolkit". I don't think so.

Really, why is looping not usual for APLers? The reason is that many cycles are hidden into assembler codes automatically by the interpreter. The history of APL is the history of cycle-hiding (at first reduction, scan, inner and outer product, then each). Why must we make an exception for recurrence? It is inconsistent to have the eclectic mixture of styles in APL. And it is quite impossible to expect that any sober-minded person will credit that he must solve the recurrence (iteration) problem with APL.

If the function f in (2) is "compilable" (that is if it can be represented as an executive module), then the recurrent cycle, which calculates the recurrent sequence can also be written in computer codes — it is quite a simple cycle (it is the nested cycle if i is multi-index). Robert Bernecky proposes to make it in three steps: at first you write cycle in APL, then APEX translates the APL code into C, then compile it into an executive module. I think this procedure should be automated. My proposition is to introduce into APL a new operator — the recurrence operator.

I published this proposition in our Russian-language journal "APL" Club" in January 1997 and I hope to report it in Toronto. The main idea is to introduce a new primitive operator, for example, with the name "*recurrence*" (I suppose it can be a new symbol " $\perp$ " like $\perp$ swinging 90 degrees clockwise), with operand f . The arguments of the derived function are $a, x_1, x_2, \dots, x_k$. The structure of these arguments characterises the dependence or independence of i . Such an operator, not primitive but defined, is presented in my paper. It contains the cyclic analogues to Robert Bernecky's "*api 3*", but in general form. Note that my recurrence operator is ideologically close to Norman Thomson's operators for iterative computations (see, for example, *APL Quote Quad* 25.1).

The solution of the API problem would be the following:

```
(←k p) Api{recurrence}{1+p)(op)
```

where *Api* is defined as follows:

```
r←par Api a;k;p;i
  A One step of API calculation
  k p i←par
  r←p[i]+k×a
```

When I use my defined operator *RecIt* instead of "*recurrence*", the time of solution is 19 seconds for 10000 elements of p (on my 486/DX2-66). If the recurrence operator were realized internally we should solve the API problem (and all other recurrence/iteration problems) in APL-style and with a speed no slower than in FORTRAN.

Andrei Buzin
RusAPL
Calculating Centre of Russian AcSc.

Index to Advertisers

Causeway Graphical Systems Ltd	18
Dyadic Systems Ltd	4
Lingo Allegro	6
Renaissance Data (Booklist)	92
Soliton	2
Strand	10
Vector Back Numbers	9

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Compuserve: 100331,644.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (with diskette as appropriate) to the Editor:

Duncan Pearson,
Causeway Graphical Systems Ltd
The Maltings, Castlegate
MALTON, North Yorks, YO17 0DP
Tel: +44 (0) 1653-696760
Email: 100265.1564@compuserve.com

Authors wishing to use Word for Windows may contact Vector Production for a copy of the APL2741 TrueType font, and a suitable Winword template.

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO6 4JJ. Please also copy us with all electronically submitted material so that we have early warning of possible problems.

Tel: +44 (0) 1439-788385
Email: 100331.644@compuserve.com

British APL Association: Membership Form

Membership is open to anyone interested in APL. The membership year normally runs from 1st May to 30th April, but new members may join from 1st August, November or February if preferred. The British APL Association is a special interest group of the British Computer Society, Reg. Charity No. 292,786

Name: _____
 Address: _____

 Postcode / Country: _____
 Telephone Number: _____
 Email Address: _____

Category (please tick box) to run from: 1st May ☐ August ☐ Nov ☐ Feb ☐
 UK private membership £12 ☐
 Overseas private membership £14 ☐
 Airmail supplement (not needed for Europe) £4 ☐
 UK Corporate membership £100 ☐
 Corporate membership overseas £135 ☐
 Sustaining membership £430 ☐
 Non-voting UK member (student/OAP/unemployed only) £6 ☐

PAYMENT — in Sterling or by Visa/Mastercard/JCB only

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Mastercard, Visa or JCB number.

I authorise you to debit my Visa/Mastercard/JCB account

Number: _____ Expiry date: ____ | ____

for the membership category indicated above,

- ☐ annually, at the prevailing rate, until further notice
☐ one year's subscription only

(please tick the required option above)

Signature: _____ Send the completed form to:

British APL Association, c/o Rowena Small, 8 Cardigan Road, LONDON E3 5HU, UK

*Data Protection Act:
 The information supplied may be
 stored on computer and processed
 in accordance with the registration
 of the British Computer Society.*

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1997/98 Committee

Chairman	Anthony Camacho 0117-973 0036 100612.1057@compuserve.com	11 Auburn Road, Redland, BRISTOL, BS6 6LS
Secretary	Ajay Askoolum 01737 771643 106173.3347@compuserve.com	42 Hanworth Road, Redhill, Surrey RH1 5HT
Treasurer	Nicholas Small 0181-980 7870 treas.apl@bcs.org.uk	8 Cardigan Road, LONDON E3 5HU
Journal Editor	Duncan Pearson 01653-696760 causeway@compuserve.com	Causeway Graphical Systems Ltd, The Maltings, Castlegate, MALTON, North Yorks YO17 0DP
Projects and Publicity	Dr Alan Mayer 01792-205678x4274 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Webmaster	Ray Cannon 01252-874697 100430.740@compuserve.com	21 Woodbridge Road, Blackwater, Camberley, Surrey GU17 0BS
Activities	Jon Sandles 01904-612882 jon_sandles@compuserve.com	138 Burton Stone Lane, YORK YO3 6DF
Education	Dr Ian Clark 01388-526803 100021.3073@compuserve.com	9 Hill End, Frosterley Bishop Auckland Co. Durham DL13 2SX
Administration	Rowena Small 0181-980 7870 treas.apl@bcs.org.uk	8 Cardigan Road, LONDON E3 5HU

Journal Working Group

Editor:	Duncan Pearson	Causeway Graphical Systems Ltd (01653-696760)
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Support Team:	Jonathan Barman (01488-648575), Richard and Adam Weber (01302-539761), Anthony & Sylvia Camacho, Ray Cannon (01252-874697), Marc Griffiths (01653-691745), David Ziemann (0181-348 4039), Jon Sandles (01904-612882)	

Typeset by APL-385 with MS Word 5.0 and GoScript

Printed in England by Short-Run Press Ltd, Exeter

members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" — an interactive computer language noted for its elegance, conciseness and fast development speed. It is suited for most mainframes, workstations and personal computers.

SUSANING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Associated Companies and Members. In many cases these companies provide manpower and administrative assistance to the Association at the following addresses:

Causeway Graphical Systems Ltd
The Matings, Castlegate,
MALTON, North Yorks YO1 1AD
Tel: 01653-696760
Fax: 01653-697719
Email: causeway@compuserve.com
Web: www.causeway.co.uk

Dyadic Systems Ltd
Riverside View, Basin Road
Old Basing, BASINGSTOCK
Hants, RG24 0AL
Tel: 01256-811125
Fax: 01256-811130
Email: sales@dyadic.com
Web: www.dyadic.com

Insight Systems ApS
Nordre Strandvej 119
DK-3150 Hellebæk
Denmark
Tel: +45 49 76 20 20
Fax: +45 49 76 30 30
Email: info@insight.dk
Web: www.insight.dk

Softron Associates Ltd
Orford Bankway, 53
1062 NO 100, Huddersfield
West Yorkshire WF1 1JN
Tel: 01484 511175
Fax: 01484 511170

Compass Ltd
Compass House
60 Priestley Road
GUILDENFORD, Surrey GU2 8YU
Tel: 01483-514500

HMW Trading Systems Ltd
Hamilton House,
1 Temple Avenue,
LONDON EC4Y 0JH
Tel: 0171-353 8900
Fax: 0171-355 3325
Email: 1000202632@compuserve.com
Web: www.hmw.co.uk

MicroAPL Ltd
South Bank Technology
90 London Road
LONDON SE1 6LN
Tel: 0171-322 8986
Fax: 0171-322 1006
Email: microapl@bt.com
Web: www.microapl.co.uk

Digital APL Research
Fosbury 18
3480 Midway
San Jose, CA 95134
Tel: 415 938 2200
Fax: 415 938 2201